Reading Minds, Imagining, Then Acting: Simulating and Probing Other Agents' Latent World Models

Andrés Daniel Godoy Ortiz

MSc in Artificial Intelligence Applications and Innovation

Imperial College London

9 September 2026

Abstract

Any system that acts on another agent faces three problems: infer the latent model behind its behaviour, use that model to predict how the agent would act in situations not yet observed, and choose how to act optimally in response.

This thesis brings those three problems into a single framework. It builds a runnable model of another agent's mind, uses that model as a simulated environment, and trains LLMs and RL policies entirely within it, before any real interaction takes place. The framework is demonstrated on a learner: the system infers what the learner understands, simulates how they would respond, and actively selects the next question to diagnose them.

The system, ATOM, combines population-level learner modelling with the semantic capabilities of language models. A frozen semantic encoder represents questions by their text rather than by item identifiers or skill labels; a recurrent variational filter maps an interaction history to a Gaussian belief over a learner latent; and a hypernetwork turns each sample of that belief into a low-rank weight edit of a frozen 4B-parameter language model (Gemma 4). Several samples therefore instantiate several plausible models of the same learner, giving both personalised predictions and uncertainty, and the planner treats that belief as its state, valuing a question by how much its answer is expected to improve later predictions.

On FoundationalASSIST, semantic representations recover the curriculum's own grouping at 0.819 AUC without observing its skill labels, and on CodeWorkout they identify which of fifty problems a submission solves at 0.995 AUC against 0.917 for string similarity. ATOM predicts learner success there at 0.786 AUC, compared with 0.512–0.559 for frontier language models reading the same histories, and reproduces the learner's exact answer 46.0% of the time. Providing the correct outcome class raises exact reproduction to 57.3%, showing that inferring the learner state, rather than generation itself, is the principal bottleneck. On CodeWorkout, ATOM reaches 0.833 AUC against a best published 0.816, and 0.561 CodeBLEU on the TIKTOC benchmark against 0.554, on learners it never trained on.

The inferred world model is also usable for action. On CodeWorkout, belief-aware policies achieve 0.924–0.944 nats of predictive log-loss versus 1.072 for random questioning and approximately 1.03 for belief-blind heuristics. A policy trained entirely through imagined interactions in the learned simulator transfers in the favourable direction in five of six real-learner settings, although the study is underpowered to establish the effect statistically. More strongly, a large language model that performs worse than random at diagnostic question selection can be fine-tuned on the simulator's counterfactual value estimates to beat random in all six settings, significantly in five, recovering 25–74% of the available oracle headroom.

Together, these results show that a probabilistic model of another agent can serve not only as a predictor, but as a runnable environment in which actions can be evaluated and policies can be learned before interacting with the real agent.

Acknowledgements

Above all, I thank my mother, Linda Ortiz. I owe her more than I can put into words. Her sacrifices, perseverance, and constant support are behind every opportunity I have had and every result that has brought me to this point.

To my dear friends Carlos Ruiz, German Carrero, Camilo Quiroz and Paola Henry, for the encouragement that led me to this Master's and the support that carried me through it.

Finally, I am grateful to my supervisor, Kelly Zhang, who encouraged me to explore simulators for reinforcement learning. That idea became the starting point of the work developed in this thesis, and ultimately took me much further than I initially imagined.

I am deeply grateful to Colombia, the country that shaped both the questions I care about and the reasons I care about them. It gave me the people, the experiences, and the tools to understand socioeconomic problems, but also the motivation to want to solve them. In many ways, this thesis is part of that same journey: from understanding a problem to asking whether technology can help change it.

Contents

Abstract 1
Acknowledgements 2

1 Introduction **5**
1.1 Knowledge is the set of questions you can answer 5
1.2 Reading a mind backwards from what it wrote 6
1.3 What was built, and on what 7
1.4 Contributions 8
1.5 How to read this document 10
Notation 11

2 Background **12**
2.1 What is a world model, and what does it mean to infer someone else's? 12
2.2 How learner modelling does it today, and why its two families cannot see each other 13
2.3 Where does a system keep what it believes about a person? 15
2.4 How should a system choose what to observe? 16
2.5 What changes when interaction is meant to move a mind? 16
2.6 The gap 17

3 The Conceptual Framework **17**
3.1 Two agents, one history, and two kinds of move 18
3.2 The space of meanings and the space of minds 19
3.3 Why every solution has two layers, and why that is metacognition 20
3.4 Belief, and the two things one predicts from it 21
3.5 Acting on the belief 23

4 Data, Courts and Discipline **26**
4.1 What the framework asks of data 27
4.2 FoundationalASSIST: mathematics 27
4.3 CodeWorkout: programming 29
4.4 What a learner actually produces 29
4.5 Splits, courts, protocols, discipline 30

5 ATOM: the Architecture **32**
5.1 The shape of the machine 32
5.2 Reading meaning instead of looking up an index 34
5.3 From a history to a belief 35
5.4 From a location to a mind 37
5.5 What the person does: two heads 39
5.6 Training, and the two instantiations 40
5.7 ATOM-Act: the planner 41

5.7.1 The environment . . . 41
5.7.2 Six families of policy . . . 43
5.8 What the architecture computes, and what it does not . . . 45

6 Results I: Meaning in Place of a Curriculum 46
6.1 The geometry organises problems the way a curriculum does . . . 46
6.2 The geometry carries what makes an item worth asking . . . 47

7 Results II: Simulating a Learner 48
7.1 Stage one: locating the learner . . . 49
7.1.1 FoundationalASSIST: Mathematics . . . 49
7.1.2 CodeWorkout: Programming . . . 50
7.2 Stage two: writing what the learner would write . . . 52
7.2.1 FoundationalASSIST: The exact answer . . . 52
7.2.2 CodeWorkout: The whole program . . . 53
7.3 How much of this is *this* learner? . . . 55
7.4 Summary . . . 57

8 Results III: Acting on the Belief 58
8.1 Where a planner can matter at all . . . 59
8.2 Reading the belief beats asking at random . . . 60
8.3 A policy trained without a single real interaction . . . 61
8.4 Teaching a language model to ask . . . 63
8.5 Summary . . . 66

9 Discussion and Conclusion 67
9.1 What was shown . . . 67
9.2 Limitations . . . 68
9.3 Conclusion . . . 69

A Ablations and Retired Variants 70
A.1 The representation . . . 70
A.2 The belief . . . 70
A.3 The heads and the binding . . . 71
A.4 The planner . . . 72
A.5 Data defects found and closed . . . 73

B Notation, for reference 73

1 Introduction

1.1 Knowledge is the set of questions you can answer

When we say a friend knows medicine, we are not claiming that a fact is filed somewhere in his head. We are making a prediction: if you asked him about the heart, he could answer. Humanity has always described what a person knows by naming the questions they could handle — ask her about the engine, ask him about the treaty — and every formal apparatus we have built since is a compression of that list. A degree says a person once answered enough questions of a certain kind. A skill tag says a problem belongs to a family. A certificate says an institution vouches for the compression. All three are coarse proxies for the thing they stand in for, and all three lose the same information: they cannot tell you what *this* person would do on *that* question.

This thesis builds a machine that does the thing the proxies stand for. From the trace of what a person has done, it forms a belief about the mind that produced the trace; it uses that belief to simulate the person on any new task — whether they would succeed, and what they would actually write, right or wrong; and it acts on the belief, choosing what to ask next so as to learn about them as fast as possible.

Stated generally, the problem is this: **infer another agent's latent world model from its behaviour, and act on the inference**. A world model, in the sense this thesis uses throughout, is the internal structure through which an agent represents its domain and generates its responses. It is never observed. All that is ever observed is what the agent does, and the task is to recover enough of the structure to predict what it would do next, on something it has not yet been asked.

Psychologists call the capacity to do this *theory of mind*: representing another agent's mental states well enough to anticipate their behaviour. Recent work usefully splits it in two (Riemer et al., 2025). *Literal* theory of mind is predicting what the other agent will do. *Functional* theory of mind is adapting one's own behaviour in response — and strong literal performance does not transfer to functional. The split maps onto the two halves of this thesis. The simulator is the literal half: it says what the learner would produce. The planner is the functional half: it changes what the system does next, in light of what it now believes. Chapter 3 treats them as one agent with a sensing side and an acting side.

The problem is not confined to a classroom, and where else it appears is worth naming before where it is measured. A health application reading a wearable is inferring a latent state from behavioural emissions and choosing when to prompt. A recommender under expensive exploration is choosing which item will most sharpen what it knows about a user. And an evaluation harness for an artificial agent is trying to find out what a model can do, given that prompting frequently under-elicits and that the model may not cooperate. In each case the object is a latent structure inside another agent, the evidence is behaviour, and the interesting decisions are about which observations to spend.

It is also the thing today's language models most conspicuously do not do. They generate text extremely well and they model nobody in particular while doing it. Asked to personalise, they are given a transcript and told to infer; asked how confident they are about a specific person, they have no representation to be confident about. Genuine personalisation requires a model *of the other mind* — an object that persists, that accumulates, and that carries its own uncertainty — and not merely a generator with a longer prompt.

Education is the testbed, for one reason: it is the domain where what a person can do is cheapest to observe and to grade. A learner's answer is written down, it is marked, and there are millions of such records. Nothing in the method is about education, and Chapter 9 returns to the settings

above once the machinery has earned the right to be discussed there.

1.2 Reading a mind backwards from what it wrote

What we observe of another mind is never the mind. A world model, in the sense fixed in the previous section, is the structure through which an agent represents its situation and generates what it does next; it lives inside the agent and produces latent states — how things stand, what follows, what to say. What reaches us is only the last step of that production: an answer, written down. And the writing is loose. The same state can be emitted as many different strings, and two problems with different labels can be about exactly the same thing, so the text we see is a noisy encoding of a meaning, and the meaning is a trace of a state the world model was in. Reading a mind from its answers is therefore that chain run backwards: from the *text*, through the *meaning* it encodes, to the *mind* that generated it. Every layer of that chain reappears in this thesis as a space, and the machine built here is the inverse chain made literal.

Humans run the inverse chain constantly, and the way they do it shows why it is hard. When we simulate a colleague, we do not reason from their words alone; we bring a scaffold of how minds in general work, and use the words to place this one within it. The scaffold cannot come from the person in front of us — their history tells us where they sit, not what the axes are — so it has to be learned from a population. Predicting a person is therefore two inferences and not one: *locate* the mind among all minds, then read what the population knows at that location. The first is inference about an individual; the second is knowledge that no individual could supply.

Two research traditions each solve one half of this and structurally cannot solve the other. Knowledge tracing learns from populations and cannot read a new question. Fifty years of psychometrics and its neural successors model a learner's latent state from sequences of graded responses, trained across thousands of learners answering the same items, which is how they come to know what a learner who fails this item typically also fails. But their knowledge of an item is a lookup: each problem, or each skill tag it carries, indexes a row estimated only from responses to that problem. A problem written this morning has no row. Two problems are related only if a human curriculum says they are. The model never reads a word of the question it is predicting about — it works on the text layer's labels and never reaches the meaning layer at all. Language models have the opposite shape: they read everything and keep no state. A frontier model given a learner's history as text has no explicit representation of that learner. It has also never been shown another learner in a way it could aggregate, so it has no population to compare anyone against. It lives in the meaning layer and never arrives at the mind. On the primary corpus of this thesis, four models of 70 to 120 billion parameters, given full learner histories, predict correctness at between 0.512 and 0.559 AUC, where chance is 0.500.

The bet of this thesis is that these are not rivals but two stages of one computation, and that they can be joined by a specific mechanism: **make the language model's weights a function of a belief about the person.** The belief is built from what the learner has been asked, what they answered and how they fared, and it is built without any part of the system ever reading an item identifier or a skill tag.

Concretely, the chain becomes three spaces and two learned maps. A frozen encoder takes each problem's text to a point in a **space of meanings**, where problems about similar things land near one another regardless of how they are phrased or labelled; a learner's history is a path through that space, annotated with what happened at each step. A recurrent filter walks the path and returns a location in a second space, the **space of minds**, together with a region around it saying how sure it is; this is where the semantic trace of a person's answers becomes a position among all persons.

A small network then turns that location into an actual world model, by editing the weights of a large frozen language model along a set of axes learned from the whole population — the axes along which minds in this domain differ. The edited model is the simulated person: prompted with a new problem, it says whether they would succeed and writes what they would write. The population supplies the scaffold; the filter supplies the fix; the edited model is the mind, made runnable.

Representing questions by their meaning rather than by their identity is what makes the system curriculum-free. No skill labels enter it; no prerequisite graph is asserted; a problem written today is handled the moment its text exists. Chapter 6 shows that the curriculum's own grouping is recoverable from problem text alone, and that a human-authored skill taxonomy is worth six thousandths of AUC to a model that has one.

The region around the location is the hinge of the design, and it is the reason one object can do two jobs. A belief that is a bare point can be turned into a model and nothing else. A belief that carries its own width can also be read for how much is still unknown about this person, and therefore for what would be worth finding out.

That is what turns a model into an agent. Once a person can be simulated, interactions can be tried in imagination before they are spent in reality: given a belief and a menu of possible questions, the system asks which answer would most sharpen what it can predict — and asks that question. The state it plans over is not the history but the belief, and the width is why. A planner that saw only the centre could never tell a confident estimate from an ignorant one, and would have no idea which question is worth asking. That is a reinforcement-learning problem with an unusual state, and this thesis takes it seriously as one. It trains a policy that reads the belief and prices questions; it trains a policy *entirely inside the simulator*, with not one real interaction in its training, and then examines it on real learners; and it fine-tunes a language model on the prices the simulator itself computes, turning a model that cannot do the task at all into one that can.

Teaching — changing the mind rather than reading it — is the same machinery with a different reward, and Chapter 3 states it in the same currency. It is specified in this thesis and not measured, because no public educational corpus records who assigned which lesson to whom by a known mechanism, and without that a teaching policy cannot be validated rather than merely reported.

1.3 What was built, and on what

The system that does all of this is called **ATOM**, and it has two halves: **ATOM-Sim**, the simulator, comprising the encoder, the filter, the hypernetwork and the prediction heads; and **ATOM-Act**, the planner, which prices questions by what their answers would reveal, trains a policy whose state is the belief, and turns the simulator on a language model — training it, against prices ATOM-Sim computes, to select better than it can cold. Figure 1 is the whole machine in one picture; Chapter 5 is the same picture with the mathematics in it.

It was measured on two corpora, chosen for complementary reasons. **FoundationalASSIST** is five thousand middle-school mathematics learners and 1.75 million interactions, with the full text of every problem and of every answer a learner typed, plus a record of when they asked for a hint or gave up. It supplies scale, the cleanest available success label, and the population from which the shared laws are learned. **CodeWorkout** is 919 first-year Java learners and 195 thousand code submissions, each compiled and run against a fixed test suite, so a single question yields a whole loop of attempts and feedback. It supplies rich productions — a whole program rather than a two-token answer — and the one setting in either corpus where the feedback a learner receives is a fully known function of what they did. The same architecture runs on both; every difference between the two instantiations is a consequence of what the corpus records.

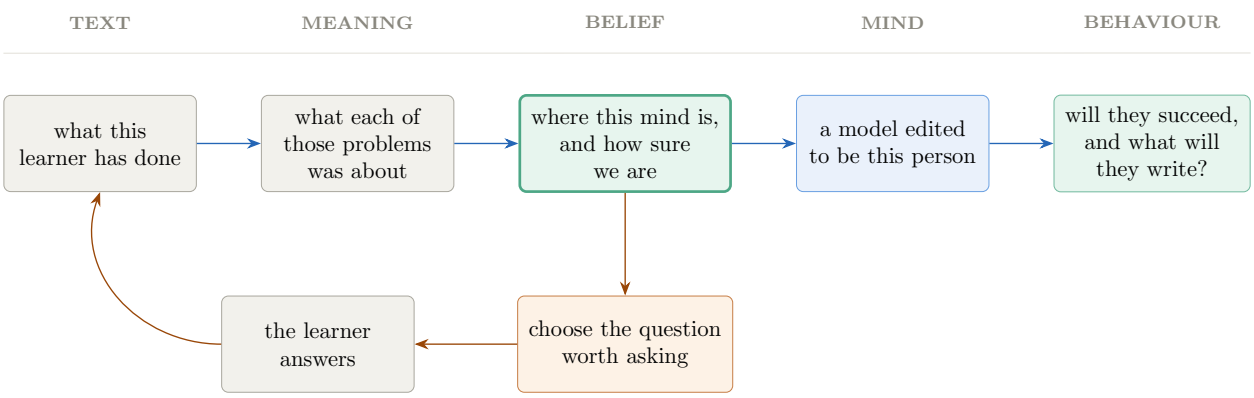


Figure 1: *ATOM in one picture. Text becomes meaning; a history of meanings and outcomes becomes a belief about the person; the belief becomes a model that behaves as that person would; and the planner, reading the belief, decides what to ask next, so that the answer sharpens the belief. Grey is frozen, blue is trained once on the whole population, green is the only thing that is personal, amber is the planner.*

1.4 Contributions

One asymmetry runs through all of the numbers below and is stated once here rather than repeated. ATOM's backbone is frozen and comparatively small: about four billion parameters are active per token, and 74 million are trained. Every comparator is larger. The mathematics baselines are frontier models of 70 to 120 billion parameters; the strongest programming comparator is built on an eight-billion-parameter instruction-tuned Llama-3 together with a code parser, and is trained on the very cohort it is scored against, where ATOM reads those learners for the first time at evaluation.

C1. A conceptual framework for reading and acting on other minds (Chapter 3): two spaces, competence as attainable success, a belief that carries a region, information priced by the decision it changes, and discovery and extension held apart. To the author's knowledge no prior work assembles these into one specification. Its specific yield is that the personalisation literature's three mechanisms become coordinate choices for one operation, and that Table 5 can state, object by object, what the built system does not compute.

C2. Meaning can do the work a curriculum did, which is what lets the two traditions become one machine (Chapters 5 and 6). On **FoundationalASSIST** the semantic encoder recovers the curriculum's own grouping at 0.819 AUC without ever being shown it, and judges which of two problems is more discriminating at 65.4% where every frontier model tested is below chance; stripping all 462 skill tags from a baseline trained on the same corpus costs it 0.0060 AUC, which is what the curriculum was worth to a model that had one. On **CodeWorkout** the same frozen representation identifies which of fifty problems a submission solves at 0.995 AUC, and cannot tell whether two submissions pass the same tests at all, at 0.523 — a failure that shaped the architecture, since behaviour is therefore carried by a supervised head rather than read off the geometry. The combination this licenses — a frozen semantic encoder, a variational filter over it, and a hypernetwork-generated low-rank edit of a frozen language model — is, to the author's knowledge, the first of its kind in learner modelling.

C3. On FoundationalASSIST, a frozen backbone made personal by a weight edit generated from a belief reads a learner better than models many times its size (Chapters 5 and 7). It reaches 0.786 AUC and 73.7% accuracy against 0.512–0.559 for four

frontier models given the same history as text; and it reproduces the exact string a learner typed 46.0% of the time against 44.3% for the best of those models, and 21.4% against 14.5–19.8% on the events where the learner was wrong. The one model ahead of it is attentive knowledge tracing, at 0.810 AUC, trained here on this corpus — to the author's knowledge for the first time — and given the skill tags ATOM is denied. The two are also not returning the same object: a knowledge-tracing model returns a point estimate of a probability, where ATOM returns a mean and a standard deviation over the person and marginalises every prediction over K draws from it — noisier by construction, and the only one of the two that a planner can read.

C4. On CodeWorkout the same architecture holds against systems built for that corpus, on learners it never trained on (Chapters 5 and 7). ATOM reaches 0.833 AUC on a strictly learner-held-out cohort, above the best published CodeWorkout result of 0.816 under KCGen-KT's five-split protocol. The comparison is not row-for-row: KCGen-KT is evaluated on random splits of its population with all fifty problems represented in training, whereas ATOM is evaluated on learners excluded entirely from training and its aggregate additionally includes seven problems withheld from all training. On generation it writes code at 0.561 CodeBLEU on TIKTOC's own benchmark — the same seventeen problems and the same 246 students — against TIKTOC's 0.554, zero-shot across a semester boundary; across all fifty problems it reaches 0.526 and loses to KCGen-KT's 0.580.

C5. The two traditions are two stages of one computation, and the gap between them is measurable (Chapter 7). On **FoundationalASSIST**, supplying the correct outcome class and changing nothing else raises exact answer reproduction from 46.0% to 57.3%, and from 21.4% to 30.5% on the events where the learner was wrong. The generator was never the bottleneck; locating the mind was. This is the thesis's cleanest evidence for the bet of Section 1.2.

C6. A simulator of a person is a workable environment for reinforcement learning, and there is room in it (Chapters 5 and 8). On **CodeWorkout**, a trained budget-conditioned actor whose state is the belief beats every blind heuristic and matches greedy value of information; and a policy trained entirely inside the simulator, with not one real interaction anywhere in its training, transfers in the right direction on real learners in five of the six scenarios it was tried on. Neither clears its strongest bar — the actor never separates from greedy value of information, and no imagination-trained scenario clears its pre-registered interval — and that is the honest shape of the claim. What the two establish together is the setting rather than a method: a belief with a width can be planned over at all, policies trained inside a learned person behave on real learners much as they behaved in imagination, and the room left between those policies and the ceiling is where the work goes next.

C7. A learned simulator of a person can supply a training signal that a language model has no other way to get (Chapter 8). On **CodeWorkout**, given the diagnostic task cold, a frontier model reads position rather than content and loses to a random policy in every scenario tested. Fine-tuned on the exact pricing the simulator computes over real events, the same model beats random in every scenario, interval-clear in five out of six, recovering between 25% and 74% of the available headroom and separating from the strongest blind heuristic in three; and a small utility model trained on the same signal prices questions it has never scored as well as the principled calculation does. A smaller extension pushes the same signal one step further, teaching the model to *write* a question rather than choose one:

it learns to write questions that belong on the course — moving them onto the manifold of real problems, from 27.2% to 95.6% — without yet making them more informative than a random pick, at a tenth of the training scale that was registered for it. This is the least finished of the results and, on the evidence of that trend, the one with the most room ahead of it: a direction shown to work, not a method delivered.

C8. An evaluation discipline (Chapter 4): two courts never compared with each other, a four-arm decomposition of personalisation, a sealed lockbox, learner-clustered intervals, and pre-registered gates recorded as failed when they failed. It is a contribution because it changed the conclusions: without it, most of the apparent person signal on FoundationalASSIST would have been reported as knowledge when it was help-seeking.

1.5 How to read this document

Chapter 2 reviews the literatures this problem touches, organised around five questions the thesis needs answered rather than around fields; it hands on a specific gap. Chapter 3 states, without reference to any implementation, what an ideal mind-reader computes, and hands on a list of demands on data. Chapter 4 presents the two corpora and the evaluation discipline, and hands on the facts that shape every design choice. Chapter 5 specifies ATOM and closes with an itemised statement of how much of the ideal it computes. Chapter 6 measures whether meaning can replace a curriculum, and hands the representation to the simulator. Chapter 7 measures locating and simulating on both corpora, and hands on the finding that locating is the bottleneck. Chapter 8 measures acting on the belief. Chapter 9 takes the contributions in turn, states the limitations once, and returns to the settings beyond education.

A reader without a background in machine learning can read this chapter, Chapter 2, the first section of Chapter 3 and Chapter 9, together with the summary section that closes each results chapter, and will have the whole argument. A reader who wants only the results can read the tables: they are self-contained, and each caption says what the number is and what it is compared against.

Notation

One notation is used throughout, and it is fixed here. Nothing in the document deviates from it; the same table is repeated as Appendix B for reference while reading the results.

OBJECT	SYMBOL	IN PLAIN WORDS	INTRODUCED
Meanings			
Space of meanings	$\mathcal{S}$	where the meaning of any piece of text lives: a problem, an answer, a program	§3.2
Meaning of a text	$s(x) \in \mathcal{S}$	what this text is about; in the implementation, 64 numbers	§3.2, built §5.2
Raw text	$x \in \mathcal{X}$	the characters themselves, before encoding	§3.2
Idea, thought, target	z, τ, g	a point of $\mathcal{S}$; a path through $\mathcal{S}$; where a path is meant to end	§3.2
Minds			
Person latent	$u \in \mathcal{U}$	the coordinates of one mind: what it knows and how it tends to work	§3.2
Space of minds	$\mathcal{M}$	every configuration the personalised model could take; a person is a point $W(u)$ in it	§3.3, built §5.4
Shared laws	$\Lambda = (\omega, \phi, \psi)$	what all minds have in common: the prior over persons, how a person moves, how a person shows in behaviour	§3.3
Engagement	η_t	how hard the person is actually trying; separates *cannot* from *does not*	§3.2
Competence	$K_g(u)$	the probability that best-effort thinking reaches target g	Def. 3.3
Evidence and belief			
History	h_t	everything observed about this person up to round t	§3.1
Belief	$b_t = p(u \mid h_t)$	where the mind probably is, and how sure we are; realised as $\mathcal{N}(\mu_t, \mathrm{diag}(\sigma_t^2))$	Def. 3.4, built §5.3
Belief samples	$u^{(1)}, \ldots, u^{(K)}$	K plausible versions of the person, drawn from b_t	§5.3
Questions, answers, moves			
Question	q	what the person is asked; its meaning is $s(q)$	§3.2
Outcome class	c	which of the corpus's outcome categories occurred (5 on mathematics, 4 on programming)	§4.2
Production	w	what the person actually wrote: the typed answer, or the program	§5.5
Probe	$e \in \mathcal{E}$, cost $c(e)$	a move that only informs the modelling agent — the only kind measured here	§3.1
Control action	$a \in \mathcal{A}$	a move that changes the person: teaching. Specified, never measured	§3.1
Deciding			
Value of information	EVOI$(\cdot)$	how much better a decision gets because of what a move reveals	Def. 3.6
The estimator used	EPIG(e)	the special case actually computed: how much a question's answer would sharpen the current prediction	Eq. (10), built Eq. (20)
Weight edit	$\Delta W(u)$	the per-person change to the frozen model's weights	Eq. (15)
Mixing weights	$c(u) \in \mathbb{R}^{32}$	how far to press each of the 32 shared adapter axes for this person	§5.4

2 Background

The problem of this thesis — inferring another agent's latent world model from its behaviour, and acting on the inference — touches research communities that rarely cite one another. This chapter reviews them, and it is organised around the questions the thesis needs answered rather than around the fields that answer them. Each section opens by saying which part of the problem it serves and closes by saying what it leaves open, so that the gap of Section 2.6 is assembled from five specific absences rather than asserted at the end.

The five questions are: what a world model is and what it means to infer someone else's (§2.1); how learner modelling does it today, and why its two families cannot see each other (§2.2); how a shared model is personalised to many people, and where the belief about a person is kept (§2.3); how a system that can choose its observations should choose them (§2.4); and what changes when interaction is used to move a latent state rather than to read it (§2.5).

2.1 What is a world model, and what does it mean to infer someone else's?

This section serves the framework of Chapter 3: it establishes what kind of object is being inferred, and what the existing formalisms for inferring it can and cannot do.

The idea that an agent acts from an internal, compressed model of its environment is now standard in both machine learning and cognitive science. In model-based reinforcement learning the agent learns a latent generative model of its environment's dynamics and plans inside it: the world models of Ha and Schmidhuber (2018), the latent imagination of the Dreamer family (Hafner et al., 2019, 2020, 2023), value-equivalent models that predict only what matters for planning (Schrittwieser et al., 2020; Grimm et al., 2020), and the joint-embedding predictive architectures advocated by LeCun (2022). The cognitive-science counterpart is predictive processing and the free-energy principle, in which perception and action are unified as the minimisation of one variational bound over a generative model (Friston, 2010; Da Costa et al., 2020; Parr et al., 2022). What these share is that the object governing behaviour is a latent structure never observed directly.

That lineage matters twice here. The thing being inferred about a learner is a world model in this sense — a structure that generates their answers rather than a score attached to them. And the system doing the inferring is itself a world-model agent of the same kind: it learns a simulator and then trains a policy inside it, which is what Chapter 8 measures.

Recovering such a structure for *another* agent, from behaviour alone, is the computational problem of theory of mind. The dominant formalism casts it as inverse planning: assume the observed agent acts approximately rationally toward its goals given its beliefs, then invert one's own planning process to find the hidden mental variables that best explain what it did (Baker et al., 2009, 2017), an inversion identified with inverse reinforcement learning (Jara-Ettinger, 2019). Learned variants replace the explicit Bayesian inversion with a neural estimator meta-trained across a population of agents (Rabinowitz et al., 2018), and opponent-modelling methods in multi-agent reinforcement learning infer latent representations of other agents, including recursive formulations (Wen et al., 2019; Moreno et al., 2021). A complementary line organises interactive systems by the depth of their user model, separating systems that treat the user as a reactive process from those that treat the user as a planning agent whose tacit knowledge can be recovered by inverting its belief updates (Çelikok et al., 2019).

Language models intensified the interest from two directions. Empirically, they do well on first-order false-belief tasks and, in some studies, on higher-order recursive beliefs, which prompted a wave of increasingly adversarial benchmarks (Gandhi et al., 2023; Xu et al., 2024; Chen et al.,

2024; Kosinski, 2024; Strachan et al., 2024). The literature then turned critical, and one distinction from it is used throughout this thesis: most benchmarks measure *literal* theory of mind, which is predicting what another agent will do, rather than *functional* theory of mind, which is adapting one's own behaviour to a partner in context — and strong literal performance does not transfer to functional (Riemer et al., 2025). Methodologically, the most recent work fuses the Bayesian and neural traditions, using language models as priors or proposal mechanisms inside Bayesian inverse planning (Zhang et al., 2025; Jha et al., 2025).

What is missing. This literature supplies the formalism and not the two things the present problem needs together. The latent it infers is typically a belief-desire tuple sufficient for one episode rather than a *full generative model* of the other agent — one that can be run forward to produce what the agent would actually say. And the observer is a spectator: it explains behaviour it was given, rather than *choosing* the observations that would identify the agent fastest. Chapter 3 takes the formalism and adds both.

2.2 How learner modelling does it today, and why its two families cannot see each other

This section serves the architecture of Chapter 5, and it is where the thesis's central engineering claim comes from. A parallel tradition models one specific latent — a learner's knowledge state — from sequences of graded responses, and it is the state of the art the results of this thesis are measured against.

The classical family begins with psychometrics. Item response theory relates the probability of a correct response to a latent ability and a small number of item parameters (Lord, 1980); cognitive diagnosis models decompose mastery across fine-grained skills, with neural variants relaxing linearity while keeping interpretability through monotonicity constraints (Wang et al., 2020). Knowledge tracing makes the latent dynamic: Bayesian knowledge tracing models it as a hidden Markov state per skill (Corbett and Anderson, 1994); deep knowledge tracing replaces that with a recurrent network over interaction sequences (Piech et al., 2015); attentive variants introduce monotonic attention with exponential decay to encode forgetting, tying the architecture back to psychometric quantities (Ghosh et al., 2020). The deep variants have been examined critically for limited interpretability and for modelling forgetting and inter-concept dependence only coarsely (Ding and Larson, 2019; Bai et al., 2024), and benchmark infrastructure has consolidated around standardised evaluations (Liu et al., 2022, 2023).

Two properties of this family are worth isolating, because Chapter 5 is built to keep one and remove the other.

The strength to keep. Every one of these models is trained on responses from hundreds or thousands of learners to the *same* items. It therefore learns population-level regularities — what a learner who fails this item typically also fails, how mastery is distributed, which items separate learners and which do not — that no single learner's history could ever reveal. This is the metacognitive layer of Section 3.3 arriving in practice, and it is the reason a two-scalar psychometric model remains a strong baseline sixty years on.

The limitation to remove. The model's knowledge of an item is a *lookup*. Each item, or each skill tag it is assigned, indexes a row of an embedding table — or a parameter vector, in the psychometric case — and that row is estimated only from responses to that item. Attentive variants learn richer item embeddings, but they learn them from the interaction data, not from the item's words: the model never reads the question. Four consequences follow, and they are the four the

encoder of Section 5.2 removes. An item that has never been answered has no row at all, so a problem written this morning cannot be scored until learners have answered it. A rarely answered item has a badly estimated row. No relation between two items exists unless a human-authored curriculum asserts it. And capacity is spent per item, so the data requirement grows with the catalogue rather than being amortised across it.

Recent work reformulates competence modelling around pretrained language models, and inherits the mirror-image problem. Next-token knowledge tracing recasts the task as next-token prediction over question content and response histories (Lee et al., 2025); language-bottleneck models force predictive information through a concise natural-language summary of the learner's state (Yeung et al., 2025); and a strand of generative learner simulators fine-tunes or prompts language models to reproduce learner-specific, imperfect reasoning — characteristic errors, partial understanding, self-correction — evaluated as behavioural digital twins (Xu et al., 2025). Closest to the simulator of this thesis is test-case-informed knowledge tracing for open-ended coding (Duan et al., 2025), which jointly predicts a student's code and its per-test-case correctness from a recurrent knowledge state injected into a language model, and its successor, which generates the knowledge components automatically rather than taking them from a human (Duan et al., 2026); both are direct comparators in Chapter 7.

The clearest published statement of the gap comes from a zero-shot evaluation of frontier models on full-text learner histories, which finds them barely exceeding trivial baselines at predicting performance, with a pronounced optimism bias (Worden et al., 2026). The authors' diagnosis is worth restating because Chapter 5 is an answer to it point by point. A language model reading a history as text has no explicit student representation: nothing in its architecture maintains or updates a learner-specific state, and its objective was mainly shaped to model surface linguistic patterns rather than to infer latent cognitive variables from response sequences. It has no longitudinal, population-level capability: it sees one learner at a time, has no mechanism for aggregating what other learners did on the same items, and cannot be shown enough of them in context to approximate what a knowledge-tracing model learns from its training set. It seems to lack metacognition about learning — the ability to reason about how *different* learners would approach an item, which is what makes an item discriminating rather than merely hard.

The two families are therefore complementary in their failures, and the split is a split on the chain of Section 1.2. Knowledge tracing works on the text layer's labels — item identifiers and skill tags — and never reaches the meaning layer at all; a language model reads meaning fluently and never arrives at the mind, because it maintains no person to arrive at. Neither traverses the chain, and the two failures are the two halves of one traversal. The same point can be put as two ingredients rather than two schools. Predicting what a learner will do requires *student dynamics* — how a person's state moves and how it shows in behaviour, learned across a population — and *question semantics* — what this particular question is asking. Classical knowledge tracing has the first and no access to the second; a language model has the second and none of the first. The distinction matters because the three kinds of generalisation a deployed system needs depend on them differently: generalising to a *new learner* needs the dynamics; generalising to a *new question* needs the semantics; and transferring what was learned on one question to another because they test related knowledge needs both at once, since it requires knowing that the two are related without a curriculum to say so.

What is missing. Nothing joins them. The position of this thesis is that this is not a trade-off to be chosen between but a division of labour to be engineered: an explicit, population-trained belief over the learner, and a frozen language model that reads content, joined by a mechanism that

conditions the second on the first. Chapter 3 argues that the division is forced by the structure of the problem; Chapter 5 builds it.

2.3 Where does a system keep what it believes about a person?

This section serves Section 5.4, and it asks a question every system serving many people with one model must answer, whatever its domain: where does it keep what it currently believes about *this* person? The literature answers in three ways, and they are best read not as three theories but as three coordinate systems for one operation — locate the person, then condition the shared law on the location, which is the hierarchical structure of Section 3.3.

In activations. Context-based methods compress a user's history into a vector injected into the model's input or hidden states: recurrent knowledge states injected as soft prompts (Duan et al., 2025), prefix and soft-token conditioning generally, and, at the limit, raw in-context learning over the transcript. Theory explains why this works at all: attention is kernel regression over the context, in-context learning implements implicit Bayesian inference over the pretraining mixture (Xie et al., 2022) and can implement gradient-descent-like adaptation (von Oswald et al., 2023; Dai et al., 2023), and sequence models trained across many tasks converge to the posterior predictive of the prior their training population embodies (Müller et al., 2022). The carrier is fast and end-to-end; the posterior is collapsed to a point, and histories outgrow the window.

In neighbours. Retrieval-based methods represent the person by similar others: collaborative filtering, user-graph retrieval, message passing over user-item graphs. The carrier is nonparametric — the empirical population is the prior — and hierarchical pooling has an exact graph reading, since Laplacian smoothing on a user graph is partial pooling under a Gaussian Markov random field prior, with the graph Laplacian converging to the manifold's own operator as samples densify (Belkin and Niyogi, 2003). Neighbour disagreement is a free epistemic signal. The carrier has no dynamics of its own, and its validity is bounded by whether the embedding distance used matches the behavioural distance meant — the point of Remark 3.1.

In weights. Adaptation-based methods edit the model itself: meta-learned initialisations whose inner loop is empirical-Bayes inference (Finn et al., 2017; Grant et al., 2018), hypernetworks that generate parameters from a conditioning code (Ha et al., 2017), parameter-efficient adaptation through low-rank updates (Hu et al., 2022), and hypernetwork-generated adapters mapping a profile to a personal edit (Phang et al., 2023). The carrier is powerful and costs the same at inference however long the history is; a single generated adapter is again a point estimate.

The three are interconvertible in simplified regimes — attention as kernel smoothing, in-context adaptation as an implicit weight update, meta-learning as hierarchical Bayes — so choosing among them is a design decision rather than a commitment. What ATOM does is use the first as its *filter*, compressing a history into a belief, and the third as its *binding*, turning the belief into a personal model, while treating the second as a source of priors and audits rather than of predictions.

What is missing. All three habitually discard the same property: the *region*. Each keeps a point estimate of where the person is and throws away how sure it is. Meta-reinforcement-learning treatments that maintain a belief inside the rollout are the closest relatives (Zintgraf et al., 2020; Duan et al., 2016), and the deep variational filtering lineage supplies the training machinery (Krishnan et al., 2015; Hafner et al., 2019). This thesis keeps the width from the first observation through to the rule that decides what to ask next, and it keeps it because the planner of Section 2.4 cannot function without it.

2.4 How should a system choose what to observe?

This section serves Section 3.5 and the planner of Chapter 8. When the measuring system can choose its observations, the question becomes which probe to administer.

Computerised adaptive testing answers it by selecting each item to maximise information about the latent ability: through Fisher information, through Bayesian criteria that minimise expected posterior variance, and through mutual-information criteria in the multidimensional case (Owen, 1975; van der Linden, 1998; Mulder and van der Linden, 2009). These rules are instances of two more general ideas. Bayesian experimental design values an experiment by the expected information it yields about a quantity of interest (Lindley, 1956; Chaloner and Verdinelli, 1995). The decision-theoretic value of information values an observation by the improvement it produces in a subsequent *decision*, which is a strictly different and generally smaller quantity (Howard, 1966). Modern instantiations learn the selection policy directly: deep reinforcement learning produces non-myopic item-selection policies that improve on one-step information maximisation (Liu et al., 2026), and bandit formulations select items while calibrating item parameters from text (Ghosh et al., 2024).

The general form of acting under uncertainty about a latent is decision-making over beliefs. Partially observable Markov decision processes plan over posterior beliefs, so that an action's value includes what it reveals (Kaelbling et al., 1998); Bayesian reinforcement learning and information-directed sampling make the exploration-exploitation trade-off explicit by valuing actions partly for their information (Russo and Van Roy, 2018); active inference derives the same drive by decomposing expected free energy into instrumental and epistemic terms (Da Costa et al., 2020; Parr et al., 2022), with recent posterior-consistency and no-regret results connecting it back to Bayesian design (Li et al., 2026). Two practical concerns recur. Greedy one-step information maximisation is generally suboptimal, because the value of a probe includes the probes it later enables, which motivates non-myopic acquisition (Jiang et al., 2020). And per-step optimisation of an information objective is typically intractable, an obstruction removed by amortisation: training a design network once to map an interaction history to the next probe in a single forward pass (Foster et al., 2021).

What is missing. This thesis adopts the value-of-information currency wholesale and adds two things. The value of a probe is targeted at the current *prediction* — at what the modelling agent will say next about this person — rather than at the posterior as a whole, so a question that would move the belief without changing any prediction is worth nothing. And the probe space is the space of meanings rather than a catalogue of items, so a question is chosen by what it asks about, and in principle could be constructed rather than only selected.

2.5 What changes when interaction is meant to move a mind?

This section serves the extension operation of Section 3.5, which this thesis specifies and does not measure, and it also explains why the framework is claimed to be domain-agnostic.

A final body of work uses interaction not to infer a latent state but to move it. In reinforcement-learning personalisation, just-in-time adaptive interventions choose actions — typically notifications or prompts — to steer a user's latent state toward a target behaviour, as in the HeartSteps physical-activity programme (Liao et al., 2020), with recent work mitigating data scarcity over a high-dimensional user state using language-model priors and contextual-bandit formulations (Karine and Marlin, 2025; Gao et al., 2026). Machine teaching and knowledge distillation supply the analogous machinery for instructing a model: adaptive chain-of-thought distillation tailors supervision to the student model's current competence (Zhao et al., 2025), and trajectory-selective

distillation chooses which demonstrations a given student can learn from (Gen-SSD, 2026). Offline model-based reinforcement learning contributes the safety discipline this thesis adopts at the moment of commitment: pessimism in proportion to model uncertainty (Yu et al., 2020; Janner et al., 2019).

The mirror image of the inference problem arises for machine agents themselves. Capability elicitation studies the conditions under which a model's latent abilities can be recovered, with prompting frequently under-eliciting where fine-tuning recovers hidden capability (Greenblatt et al., 2024), complicated by strategic underperformance (van der Weij et al., 2024) and by model organisms designed to resist elicitation (Hofstätter et al., 2025). Interpretability attacks the latent directly, from probing classifiers (Alain and Bengio, 2017) to the eliciting-latent-knowledge programme (Christiano et al., 2021; Burns et al., 2023; Marks and Tegmark, 2023; Mallen et al., 2024).

What is missing. These are not analogies. Choosing an intervention to move a person's latent state, choosing a demonstration a student model can learn from, and choosing a prompt that reveals what a model can do are the same formal problem: recovering, and acting on, a latent model from behaviour. That is the sense in which the framework of Chapter 3 is domain-agnostic and education is a testbed rather than a theme. What is absent from all of it, for the educational case, is data: no public corpus records who assigned which lesson to whom by a known mechanism, which is why Chapter 4 closes the extension operation off empirically.

2.6 The gap

Across these literatures two devices recur with increasing frequency: a latent state inferred from behaviour, and a large pretrained model as a prior over that state and over the structure relating its components. No prior work, to the author's knowledge, assembles the whole organism: a hierarchical, region-carrying belief over another agent's world model, maintained by a filter over a semantic, curriculum-free, index-free representation of interactions; a population layer that serves unbounded individuals through one frozen language model, conditioned on that belief through a generated weight edit rather than through its context window; and a planner that treats the belief as its state, prices a question by the prediction it would improve, and is trained and evaluated inside the simulator that belief defines.

Each seam is separately named as open in its own literature: decision-targeted acquisition over a semantic probe space; a population-trained student state joined to a content-reading language model; one uncertainty object carried from the first observation through to the asking rule; amortised experimental design with a person-level filter in the loop. The next chapter states what the assembled object would have to compute.

3 The Conceptual Framework

This chapter states, without reference to any implementation, what an ideal mind-reader computes. It is a specification of a problem rather than of an algorithm: it names the objects an agent must maintain if it is to predict another agent's behaviour and then choose its interactions well, and it says what each object is for. Nothing here is a description of the system built in this thesis. That system is Chapter 5, and it is judged against this chapter — literally, by a table at the end of Chapter 5 that states object by object what is computed in full, what is approximated, and what is specified and left unbuilt.

The chapter is needed now because the vocabulary of everything that follows is fixed here.

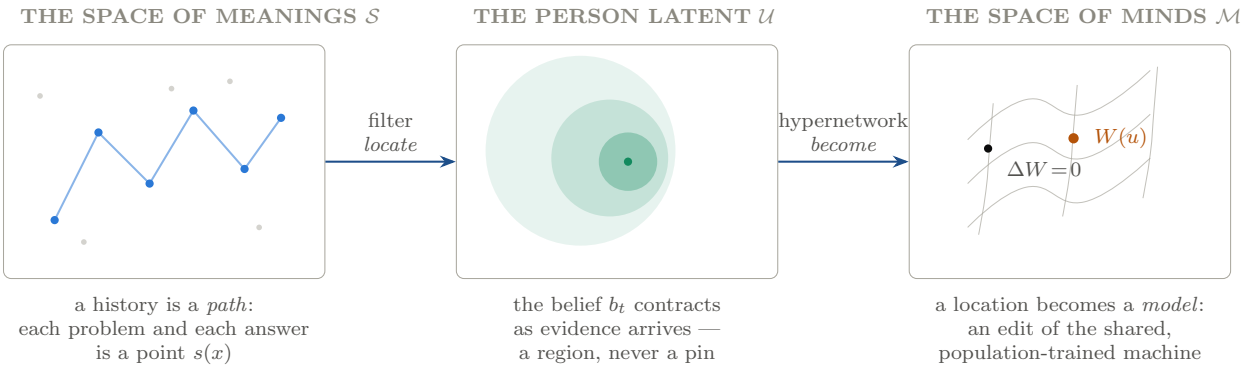


Figure 2: *The chain of spaces. Reading left to right: text is encoded to meaning, so a learner's history is a path through the space of meanings; a filter turns that path into a belief about where this person sits among all persons; and reading the population's shared laws at that location yields an actual model that behaves as the person would. The transition from an* idea *to a* knowledge state *happens in the first arrow: the filter is the object that consumes meanings and outcomes and emits coordinates of a mind. Uncertainty is carried throughout — the middle panel is a region, and it is what the planner of Section 3.5 reads.*

Chapter 2 left two families of learner model with complementary failures and no account of why they should be joined; the account is the two-layer structure of Section 3.3. The chapter is organised as a single chain, drawn in Figure 2: from text to meaning, from a history of meanings to a location in a space of minds, from that location to a prediction, and from the uncertainty in that prediction to a decision about what to ask next. Section 3.1 sets up the two agents and the two kinds of move; Section 3.2 builds the two spaces; Section 3.3 explains why any solution has two layers; Section 3.4 defines the belief and the two things one predicts from it; and Section 3.5 prices a question and states the two operations the framework supports.

3.1 Two agents, one history, and two kinds of move

The framework has exactly two agents. The **subject** is the agent whose latent structure is to be modelled and, in principle, influenced; in this thesis the subject is always a learner. The **modelling agent** is the agent that interacts with the subject in order to find out what it is like. Only the second is being designed. The subject is treated as a piece of the world whose behaviour must be predicted, in the same sense in which a physical system is a piece of the world whose trajectory must be predicted.

Interaction proceeds in rounds. At each round the modelling agent makes a move, the subject responds, and the pair is appended to the record. Everything the modelling agent has observed by round t is its **history** h_t: a chronological sequence of moves and the responses they drew. The history is the only channel through which the subject's internal structure is ever visible; there is no privileged access, and no self-report is treated as ground truth — a report is another response, informative because the modelling agent has learned how reports relate to subsequent behaviour, not because reports are true.

Moves come in two kinds, and the distinction is worth an example of each. A **probe** $e \in \mathcal{E}$ exists to move the modelling agent's belief and nothing else: asking a learner to solve a problem in order to find out whether they can. A **control action** $a \in \mathcal{A}$ is a move that changes the subject: teaching them the method the problem requires. A probe carries only a cost $c(e)$ — the learner's time, their patience, the opportunity cost of the question not asked. A control action carries the domain's consequences. The types are semantic rather than exclusive, and in many real settings one move

both treats and reveals; the framework keeps them apart because the two are priced differently, not because a given move must be one or the other.

This thesis measures probes. Section 3.5 states the control problem in the same currency, and Chapter 4 explains, with the data, why it is not measured on real learners: no public educational corpus records who assigned which lesson to whom by a known mechanism, and without that a teaching policy cannot be validated. Saying so here rather than in a caveat later is deliberate. Everything the results chapters report about acting is about asking.

One assumption is needed and is stated once.

Assumption 3.1 (Oblivious subject)**.** The subject responds as it would outside the interaction, and does not adapt strategically to being modelled. This excludes test-wiseness, gaming, and any incentive to misrepresent. Relaxing it turns every answer into a strategic report and changes the emission law rather than the framework's shape; the consequences are noted in Section 3.5.

3.2 The space of meanings and the space of minds

Everything the modelling agent observes arrives as text — a problem statement, a typed answer, a program — and everything it needs from that text is its meaning. The substrate of the framework is therefore a space in which meaning is a location.

Definition 3.1 (The space of meanings)**.** Let $\mathcal{S}$ be a measurable space whose points represent meanings independently of the language or syntax that expresses them, and let $\mathcal{X}$ be the space of raw texts. Utterances that say the same thing, in different phrasings or different languages, map to the same region of $\mathcal{S}$; distance and neighbourhood in $\mathcal{S}$ are taken to be semantically meaningful.

Assumption 3.2 (An encoder exists)**.** There is a measurable map $s : \mathcal{X} \to \mathcal{S}$ carrying a text to its meaning, and a map back. The framework uses only that such maps exist and are semantically faithful. Constructing one is an engineering problem, and it is solved in Chapter 5 by adopting a frozen encoder rather than learning one.

Two objects are built on this space. An **idea** is a point $z \in \mathcal{S}$: what a single question, or a single answer, is about. A **thought** is a finite path $\tau = (z_1, \ldots, z_L)$ through $\mathcal{S}$ whose endpoint is the produced answer and whose intermediate points are the reasoning that produced it. A **target** g is an idea, or a region of ideas, that an agent may be required to reach.

The distinction between the endpoint and the path is recorded because it is the reason two of this thesis's measurements are different tasks rather than two views of one. Whether an answer is *correct* can be read off the endpoint alone. *How* the answer was reached is a property of the whole path. Predicting what a learner will write is therefore strictly richer than predicting whether they will be right, and Chapter 7 measures both.

The subject's own structure is what generates these paths.

Definition 3.2 (World model and person latent)**.** An agent's **world model** is the internal structure through which it represents its domain and generates its thoughts. Every corpus in this thesis is text, so the domain so represented is throughout a world of sayable things rather than one seen or acted in. Its content at time t is summarised by the **person latent** $u \in \mathcal{U}$: the coordinates of one mind, comprising both stable dispositions and whatever is currently known, believed and available. The person latent is never observed.

Given a question q, the subject's world model induces a distribution over thoughts, $\tau \sim \pi(\cdot \mid q, u)$. Observed behaviour is a sample from that distribution, and recovering u from such samples is an

inverse problem: the inversion of the subject's own generation of behaviour from its model. This is the shape theory of mind takes when it is cast as inverse planning (Baker et al., 2017; Jara-Ettinger, 2019).

One component of the person latent is singled out. **Engagement** $\eta_t \in [0, 1]$ governs the precision with which the subject pursues whatever target it has been set. At full engagement the subject follows its *best-effort* policy $\pi^\star(\cdot \mid q, u)$; as engagement falls, behaviour departs from best effort with no change whatever in what the subject could do. The separation is what allows a failure to be attributed either to the person latent — *cannot* — or to engagement — *does not*. Any account that reads behaviour as a direct index of ability collapses the two. On the primary corpus of this thesis the collapse is not a theoretical worry but the dominant statistical fact: a learner who asks for the answer forfeits credit, so the recorded label is a product of two very different events, and Chapter 4 shows that between-learner variation in the first exceeds variation in the second.

With engagement separated out, the thing worth calling knowledge can be defined.

Definition 3.3 (Competence)**.** The **competence** of an agent with person latent u at target g is the expected success of its best-effort thought,

$$K_g(u) \;=\; \mathbb{E}_{\tau \sim \pi^\star(\cdot \mid q_g,\, u)}\, \mathbb{E}_{R \sim p(R)} \big[\, R(\tau, g) \,\big], \tag{1}$$

where R scores how far a thought attains its target and is itself uncertain, drawn from $p(R)$ to capture noise and disagreement in marking.

Competence is not a stored object. It is a property of a stochastic process: the probability that a chain of thought, generated at best effort from this person latent, terminates in success. This is the everyday practice of Chapter 1 made formal. When one says a friend knows medicine, one is not asserting that a fact is filed somewhere in their head; one is asserting that if the question were asked, the answer would arrive. Knowledge is the set of questions that can be answered, and (1) is that sentence written down.

3.3 Why every solution has two layers, and why that is metacognition

The modelling agent's theory of a person is a two-layer generative story. Nature draws who the person is from a population; their internal state moves with time and with what happens to them; and the state leaks into observables:

$$u_0^i \sim p_\omega(u), \qquad u_t^i \sim T_\phi(\,\cdot \mid u_{t-1}^i,\, m_{t-1},\, q_{t-1},\, \Delta t\,), \qquad y_t^i \sim O_\psi(\,\cdot \mid u_t^i,\, q_t\,), \tag{2}$$

for persons indexed by i, where m ranges over the moves of Section 3.1. The triple $\Lambda = (\omega, \phi, \psi)$ — a prior over persons, a transition law, an emission law — is what this thesis calls the **shared laws**. Emissions include the response itself, its correctness, and behavioural side-channels such as help-seeking and latency, because a world model of a person must predict the domain outcome jointly with the behaviour that surrounds it.

The structural bet carries the entire design: the laws are shared across everyone, and only the location is personal. How minds move is assumed universal enough to be learned from a whole population; where a particular mind is, is individual. That single assumption buys three things. It buys transfer, because two people at nearby locations inherit each other's evidence. It buys sample efficiency, because every person observed is evidence about the laws. And it buys the possibility — taken up in Section 3.4 — of being legitimately confident about an interaction this particular individual has never had.

The two layers are not a modelling preference; they are forced by a symmetry. Persons are *exchangeable*: nothing in the problem depends on the order in which they are indexed. De Finetti's representation theorem states that any exchangeable collection is necessarily a mixture — independent draws given a latent per-person parameter, itself drawn from a population measure (de Finetti, 1931). Within a person, time is not exchangeable, but the sequence is Markov-exchangeable, and the Diaconis–Freedman extension states that such processes are necessarily mixtures of Markov chains: a latent state with a transition law (Diaconis and Freedman, 1980). Any architecture that succeeds at this problem is therefore approximating the mixing measure and the conditional laws of (2), whether or not its designers describe it that way. Chapter 2 reads three families of personalisation mechanism through exactly this lens.

The two layers also fix the **order of computation**, and this is the sentence the rest of the thesis is built on. Predicting what a person will do is not one inference but two, performed in sequence. First, from the history, infer *where* this person sits among all persons. Second, at that location, read the shared laws to obtain the behaviour a person there would produce. The first stage is inference about an individual; the second is the population's knowledge, evaluated at a point. Think of it as a satellite fix and a map: the filter is the fix, the shared laws are the map, and reading the map at the fix gives the traveller. Handing a stranger's fix to the same map gives a plausible traveller, but not this one — which is precisely the control Chapter 7 uses to isolate what knowing *which* learner is worth.

This is the sense in which the framework describes *metacognition* on the modelling agent's part. To predict one mind it must first reason about how minds differ from one another, and decide which mind it is facing, before it can reason as that mind would. A system with no population layer has nothing to decide between; a system with no personal layer has nobody to be.

Remark 3.1 (The manifold of minds). The mixing measure has a picture worth keeping in view. Among all conceivable laws that could turn questions into answers, the laws of *actual* minds occupy a thin, low-dimensional surface. Learning the shared laws is surveying that surface; locating one person is contracting a region on it as evidence arrives. Three consequences recur through this thesis. *Transfer is smoothness*: two people help predict one another to precisely the extent that the map from latent to behaviour changes gently between them. *Difficulty is intrinsic dimension, not ambient dimension*: how much data the shared laws need scales with how many independent axes minds actually vary along, not with the size of the representation used to describe them — which is why the low-rank population layer of Chapter 5 is a substantive bet rather than a convenience. *Distance must be behavioural*: the coordinates are gauge, since any relabelling of the latent space, compensated in the laws, leaves every prediction unchanged, so "similar people" can only mean people who are hard to tell apart *by what they would do*. Two people can be neighbours in an embedding of their histories and strangers in their behaviour.

Smoothness is more than a fact about minds; it is a *requirement on the coordinates*. A belief that is a distribution over the person latent is meaningful only if nearby coordinates denote nearby minds — only if the region between two plausible locations is itself populated by plausible people, so that averaging predictions across the region is averaging over people who could exist. Were the population to occupy isolated islands, the width of a belief would be uninterpretable and its samples would be fictions. Chapter 5 therefore treats the coherence of the person latent as a constraint to be enforced by the training objective, not as a property to be hoped for.

3.4 Belief, and the two things one predicts from it

The modelling agent does not know where the subject is, and reasons over a distribution instead.

Definition 3.4 (Belief)**.** The modelling agent's **belief** at round t is the posterior over the subject's latent given everything observed,

$$b_t \;=\; p\big(u_t \mid h_t\big), \tag{3}$$

maintained by the Bayesian update that predicts forward through the transition law and corrects by the emission law,

$$b_{t+1}(u) \;\propto\; O_\psi\big(y_{t+1} \mid u,\, q_{t+1}\big) \int T_\phi\big(u \mid u',\, m_t,\, q_t,\, \Delta t\big)\, b_t(u')\, du'. \tag{4}$$

Two properties of (4) do real work later. The first is that when the subject is not observed, the belief is still pushed forward through the transition law without any correcting observation, so it *diffuses*. A stale belief is automatically a wide belief, which raises the value of a cheap question without any re-engagement heuristic having to be written down. The second is stated separately because a planner is incoherent without it and it is easy to violate in an implementation:

> **The belief is a function of the history only.** It does not depend on the question about to be asked.

A weather forecast is made before looking out of the window. A "forecast" that peeks first will score better and has stopped being a forecast. The same holds here: a belief recomputed with the next question in view is a prediction in disguise, and it destroys the quantity the planner exists to compute, because the value of asking a question is the change it would produce in a belief that did not already know it was coming.

Because its argument is itself a model of an agent, the belief is a belief about a believer, and the modelling agent is a reinforcement-learning agent whose state is this object, whose actions are interactions, and whose transitions are updates to it (Kaelbling et al., 1998).

From a belief, the modelling agent predicts two different things, and the framework's first commitment is to hold them apart.

Definition 3.5 (Simulator and success map)**.** Under belief b, the **simulator** is the posterior-predictive distribution over what the subject would produce,

$$\hat{\pi}(\tau \mid q, b) \;=\; \int \pi(\tau \mid q, u)\, b(u)\, du, \tag{5}$$

and the **success map** is the posterior-predictive competence at a target, together with the **epistemic uncertainty** that remains about it,

$$\hat{K}_g(b) \;=\; \int K_g(u)\, b(u)\, du, \qquad U_g(b) \;=\; \mathrm{Var}_b\big[K_g(u)\big]. \tag{6}$$

The simulator is the modelling agent's stand-in for the subject's own world model of Definition 3.2. It is not that structure — which is unobservable, and which Section 3.5 records can never be identified from behaviour alone — but a runnable object with the same output distribution, and it is the object Chapter 5 builds.

Proposition 3.1 (Success is easier to predict than the answer)**.** *If the simulator is concentrated — the modelling agent can predict the thought the subject would produce — then it can also predict success, by scoring that thought. The converse fails: the success map may be confident, $U_g(b)$ small, while the simulator remains diffuse. The targets whose* answer *can be predicted are therefore a strict subset of those whose* success *can be predicted.*

The everyday form is familiar. A layperson cannot reproduce the answer a physician friend would give to a question about the heart, yet is confident the friend would answer correctly. Definition 3.3 licenses the gap: competence is a scalar functional of the person latent, and a scalar can be read off without simulating the trajectory that would realise it. This is also where modelling is cheap. Estimating a probability of success requires only that a reliable route to the target exist; reproducing the subject's thought requires its whole generative process. The two predictions are evaluated separately throughout this thesis — the success map against the knowledge-tracing literature, the simulator against the behavioural-twin literature — and the gap between them is itself reported.

Finally, the uncertainty in any prediction separates into three parts, and only one of them can be bought with a question. Reading an observed outcome Y for a target g through the hierarchy gives

$$\operatorname{Var}\big[Y \mid g, h_t\big] \;=\; \underbrace{\mathbb{E}_{\Lambda}\,\mathbb{E}_{u|\Lambda}\big[\operatorname{Var}(Y \mid u, \Lambda)\big]}_{\text{behavioural}} \;+\; \underbrace{\mathbb{E}_{\Lambda}\big[\operatorname{Var}_{u|\Lambda}\big(K_g(u)\big)\big]}_{\text{person}} \;+\; \underbrace{\operatorname{Var}_{\Lambda}\big(\mathbb{E}_{u|\Lambda} K_g(u)\big)}_{\text{law}}. \tag{7}$$

The behavioural term is irreducible and must be planned around. The law term is reduced by observing *many* subjects, or by trusting the pretrained prior that supplies the laws. Only the person term can be reduced by interacting with *this* subject, and it is therefore the only one a planner can act on within one interaction: the belief's width along the directions a prediction depends on is what makes a question worth asking, and that is why the width is part of the planner's state. The decomposition also explains why confidence about the novel can be legitimate. Both epistemic terms can be small on an interaction this person has never had — their location is pinned down by their other behaviour, and the population has visited that regime — so calibration and intuition need not conflict.

3.5 Acting on the belief

The decisive design choice of the framework is that the planner's input is the belief, not the raw history. Everything the modelling agent knows about the subject is in b_t; everything it decides is a function of b_t; and the value of a move is defined by what it does to b_t.

Write the decision problem over beliefs, closed over both kinds of move. When the modelling agent moves, its own model predicts the subject's response, the update of (4) produces a new belief, and a reward or a cost arrives:

$$y \sim \hat{p}(y \mid b_t, q_t, m), \qquad b_{t+1} = F(b_t, q_t, m, y), \qquad Q^{\star}(b, q, m) \;=\; \mathbb{E}\Big[\, r \;+\; \gamma \max_{m'} Q^{\star}(b', q', m') \,\Big]. \tag{8}$$

Every symbol in (8) has been defined: b is the belief of (3), q the question in front of the subject, m a move of either type, y the subject's response, F the update, r the reward the move earns, and γ the discount. The recursion should be read carefully, because it is the belief that transitions. A control action's value is its consequences plus whatever its outcome teaches. A probe's immediate reward is just $-c(e)$, so its entire value lives in the continuation: the sharper belief it produces and the better decisions that belief enables, denominated in reward and discounted automatically. Two things follow.

First, "ask or act" is not a special rule bolted onto the agent. It is the same arg max, reading off which type of move wins now. Second — and this is the property the whole design turns on — **a bit of information that changes no downstream decision contributes exactly zero to $Q^{\star}$.** Information is worth precisely the reward it releases. Nobody has to set an exchange rate between nats and outcomes, because the recursion sets it.

Definition 3.6 (Expected value of information)**.** Let the quality of the modelling agent's decision under belief b be $V(b)$. For a move m inducing posterior belief b',

$$\mathrm{EVOI}(m) \;=\; \mathbb{E}\big[V(b')\big] - V(b), \tag{9}$$

the expected improvement in decision quality produced by making it (Howard, 1966).

The ideal probe is the one whose answer is most informative about which course of action is best. This thesis runs the simplest well-posed special case of that ideal, and stating it here makes the planner of Chapter 5 recognisable as the same object. Suppose the "decision" is a prediction: the modelling agent must forecast the subject's outcomes on a set of *target* interactions, and the quality of that forecast is its negative expected predictive entropy. Then the expected value of information reduces to the mutual information between the outcome of the candidate probe and the outcomes of the targets, under the current belief:

$$\mathrm{EVOI}(e) \;=\; I\big(y_e;\, y_{\text{targets}} \mid b\big), \tag{10}$$

the **expected predictive information gain** of the experimental-design literature. Equation (10) is not the expected information gain about the person latent itself: a probe whose answer would move the belief without changing any target prediction scores zero, as the recursion demands. Computing it needs only samples of the person latent from the belief and the emission law evaluated at each — the two things Chapter 5 provides — and it obeys a useful invariant, that a question cannot be worth more information than its own answer contains.

Two disciplines complete the machinery. *Non-myopia*: the value of a probe includes the probes it later enables, which is why acquisition is posed as a policy over beliefs rather than a one-step selection rule (Jiang et al., 2020; Foster et al., 2021). *Pessimism at commitment*: an optimiser let loose inside a learned model becomes a search engine for that model's errors (Yu et al., 2020), so committing uses conservative value estimates, while probes — whose downside is bounded by their cost — are exempt. The stopping rule falls out as the exhaustion of questions worth their price: probe while $\max_e[\mathrm{EVOI}(e) - c(e)] > 0$, then commit.

Remark 3.2 (When the type of a move matters). For an agent that only asks, the typing of Section 3.1 is nearly vacuous: every move it is allowed to make leaves the subject unchanged, so every admissible move is already a probe. The typing becomes substantive where asking and changing interleave, and the agent must choose between advancing and verifying. It is introduced because it is a general feature of any agent that both teaches and checks its work.

The two operations. The framework supports two, and they must not be collapsed into one objective.

Problem 3.1 (Discovery)**.** Given a prior belief, a downstream decision of quality $V(b)$, and a horizon, find the policy maximising the expected cumulative value of information, $\pi^{\text{disc}} = \arg\max \mathbb{E}[\sum_t \mathrm{EVOI}(m_t)]$, each move chosen from the current belief. Discovery is *inference*: it reduces uncertainty about the person latent without changing it.

Problem 3.2 (Extension)**.** Given a belief and a goal expressed as preferred competence outcomes over a target set G, find the policy maximising expected post-intervention competence net of cost,

$$\pi^{\text{ext}} \;=\; \arg\max\; \mathbb{E}_{u'}\Big[\sum_{g\in G} \tilde{p}(g)\, K_g(u')\Big] \;-\; \lambda\, \mathrm{cost}(\pi) \quad \text{subject to the simulator remaining accurate,} \tag{11}$$

where u' is the subject's latent after the interventions and $\tilde{p}$ weights the targets. Extension is *control*: it changes the person latent.

The constraint in Problem 3.2 is the deep point. One can teach well only where one can predict: if instruction pushes the subject into a region where the simulator is inaccurate, the modelling agent loses the ability to anticipate the response, and teaching degrades. "The better I can predict your behaviour, the better a teacher I am" is, in this framing, a constraint in an optimisation rather than a slogan. Discovery and extension share one state — the belief — and each produces information the other uses, but they have different rewards and there is no common currency in which both are spent. The natural schedule alternates them: probe to locate the frontier, intervene to move it, probe to verify the move.

Education as an instantiation. Nothing above names a domain. Education instantiates every object with the cleanest available reading. The subject is a learner; the person latent is what they currently know and how they tend to work; a thought is a chain of reasoning whose score is its correctness, which is cheap and explicit, so the success map is well calibrated. Relative to a goal, a learner's targets fall into three regions: the *known*, which the modelling agent can confidently call successful — and which, by Proposition 3.1, is strictly larger than the set whose answers it could reproduce; the *gap*, confidently failed; and the *unknown*, where evidence is insufficient. Crucially, the structure that carries inference between targets is learned geometry rather than a curriculum: relatedness between problems is proximity in $\mathcal{S}$, so no human labelling of prerequisites enters the model.

The operations specialise directly. *Evaluation* fixes a set of goals and administers items to resolve each into known or gap, selecting $m^\star = \arg\max_e \mathrm{EVOI}(e \mid b_t) - \lambda c(e)$ and halting when the largest remaining uncertainty falls below a threshold. Two nested special cases locate this against the literature: when $V(b)$ is the negative entropy of the success map, the criterion reduces to expected information gain; when, further, the person latent is a one-dimensional ability at full engagement with logistic item-response competence, it reduces to Fisher-information item selection — classical computerised adaptive testing. The framework generalises all three, by admitting a structured multi-competency latent, by letting information be carried by a whole production rather than a single bit, by separating competence from engagement so that a wrong answer is not read as inability, and by weighting questions by the decision they would change. *Diagnosis* is the same operation made directional: a confirmed gap raises the uncertainty of the semantically related, more elementary targets beneath it, and the search descends until the frontier between what the learner can and cannot do is sharp. *Teaching* enters the transition law: a good lesson introduces material the learner can assimilate — likely under their current model — that nonetheless brings the goal within reach. One refinement falls out rather than being imposed. If a lesson simply hands over the answer, nothing in the person latent need change; for the latent to move, the learner must perform the inferential step themselves. Competence gain is therefore largest at *intermediate* novelty, which is the formal shape of desirable difficulty and the reason a good tutor elicits an idea rather than supplying it.

Four commitments. The framework carries four consequences that constrain what may be claimed from it, and they are stated once.

Behaviour is the only ground truth. Two models with different internal coordinates but identical behavioural predictions cannot be told apart by any dataset, yet they extrapolate differently off-distribution. What breaks the tie is the prior and the structural assumptions, so the hierarchy does

epistemic work and that choice determines counterfactual validity. The prior is not a regulariser; it is the system's actual theory of mind. On the question of whether the latent deserves to be called a mind, the framework is deliberately agnostic: the criterion is predictive sufficiency, and the system becomes theory of mind insofar as belief-desire-structured latents win that competition.

Logged data is confounded. Historical logs were generated by a process reacting to variables not in the record. A transition law fitted by maximum likelihood on such data is associative, and planning with it inherits off-policy bias; the modelling agent's own moves are the only unconfounded data it will ever possess. A subtler form of the same problem recurs whenever a policy is evaluated by replaying a logged history: the set of questions it may ask was chosen by the platform, not by the policy, so the geometry of that set — how recent the askable questions are, how far they sit from the targets — must be reported alongside any result. A comparison can be perfectly fair and still answer the wrong question.

A searched simulator becomes a search engine for its own errors. The simulator is learned and therefore wrong somewhere, and optimisation amplifies model error. This is the jurisdiction of pessimism: discipline inside the imagination and at the moment of commitment, with questions, not optimism, doing the work at the boundary with the world.

The manipulation dual. The same machinery, under a misspecified reward, is a manipulation engine with a belief state — and probes sharpen rather than soften the point, because asking is itself an intervention on a person, and there are questions one should not ask. The constraint set on moves, namely the interventions and questions a person would *endorse*, is itself a latent to be inferred, with its own uncertainty and its own reasons to ask. It belongs in the formalisation, not in an appendix, and Chapter 9 returns to it.

What this hands to the next chapter. The framework has now named everything an ideal mind-reader needs: an encoder from text to meaning, a history, a belief with a width, shared laws learned across a population, a simulator and a success map read from the belief, and a rule that prices a question by the decision it would change. Each of those is a demand on data. Chapter 4 states what two real corpora actually record, which of the demands they meet, and which single absence removes one of the two operations from the empirical part of this thesis altogether.

4 Data, Courts and Discipline

Everything the system of Chapter 5 does is shaped by what two corpora happen to record. This chapter presents them first, so that each design choice in the next chapter can be read as a response to a concrete fact stated here rather than as a preference. It also fixes the terms in which every later number is reported: which learners and which problems a model was allowed to see, which population a score is computed over, and what discipline governed the runs that produced it.

Section 4.1 lists what the framework of Chapter 3 asks of data and identifies the one thing no public corpus supplies, which is why one of the framework's two operations is specified and never measured. Sections 4.2 and 4.3 describe the two corpora. Section 4.4 shows what a learner actually produces on each, because that single difference explains most of the contrast between the two halves of Chapter 7. Section 4.5 states the splits, the courts, the protocols and the evaluation rules.

4.1 What the framework asks of data

An ideal corpus would supply seven things per interaction. **R1**, the question's content rather than an identifier, because Chapter 3 represents an item by its meaning. **R2**, the learner's actual production — what they typed, wrote or submitted — because the simulator predicts a thought and not only its score. **R3**, a success signal, so the success map has ground truth. **R4**, timestamps, so histories are ordered. **R5**, breadth: many learners with long histories, because the shared laws are learned across a population. **R6**, auxiliary behavioural channels — help-seeking, attempt loops, per-test feedback — because engagement must be separable from competence. And **R7**, logged interventions with a known assignment mechanism: the teaching channel, with causal identification.

No public educational corpus supplies R7. That single absence is why the extension operation of Problem 3.2 is specified in this thesis and never measured on real learners. A teaching policy can only be validated where lessons were assigned by a process that is known, because otherwise every apparent effect of a lesson is confounded with whatever made the learner or the platform choose it; and in every available log, the learner chose what to attempt next, or the platform did by a rule that was not recorded. Stating this here rather than in the discussion is the point of putting data before architecture: the architecture has no teaching component, and the reason is on this page.

Two corpora supply everything else, and they are complementary rather than redundant. FoundationalASSIST is broad and shallow per event: thousands of learners, hundreds of events each, one short typed answer per event. CodeWorkout is narrow and deep: hundreds of learners, fifty problems, and for each problem a whole loop of code submissions with compiler and test-case feedback. The first is the statistical substrate on which the shared laws are learned at population scale, and the corpus on which a questioning policy can be replayed with statistical power. The second is where the production channel is at its richest, and where a single question reveals a whole attempt rather than one bit. Table 1 states, requirement by requirement, what each supplies.

Component	Needs	Mathematics	Programming	Note
Items by meaning	R1	problem text	problem statement	the same frozen encoder on both
Production channel	R2	typed answer	whole program	one is short, one is rich
Outcome classes	R3, R6	5, help-seeking split out	4, compile and test outcome	§4.2, §4.3
Per-test-case outcomes	R6	—	✓	one scored answer per item on maths
Attempt loop within an item	R6	—	✓	one attempt per item on maths
Help-seeking	R6	✓	—	carried by the outcome classes
Time	R4	✓	✓	order only; no forgetting model
Population scale	R5	5,000 learners	919 learners	the shared laws are thinner on code
Identified teaching	R7	—	—	extension specified, not measured

Table 1: *What each component of the framework needs from data, and what each corpus supplies. The last row is the one that removes an operation from the empirical programme.*

4.2 FoundationalASSIST: mathematics

FoundationalASSIST is a release of middle-school mathematics interactions that carries the full text of every problem *and* of every answer a learner actually typed (Worden et al., 2026). It was released

by ASSISTments and Worcester Polytechnic Institute, with problems drawn from the Illustrative Mathematics curriculum for grades 6–8, and is licensed CC-BY-NC-4.0 under gated access, granted for this research and processed under its responsible-use terms. The raw and derived text never leaves the machines it is processed on and is never committed to any repository.

The release holds 1,753,384 interactions from 5,000 learners, each of whom completed between 211 and 421 problems, a mean of about 350. It covers 3,395 problems with full HTML and MathML text — 2,188 fill-in-the-blank, 795 multiple-choice, 380 select-all and 32 ordering — of which 1,175 carry the full text of their distractors. Each interaction records the problem, the learner's first typed answer verbatim, the number of hints requested, whether the answer was revealed, a strict first-attempt correctness score, and an end-time stamp. Help-seeking is substantial: 5.2% of interactions used a hint and 21.7% revealed the answer. A skills file tags problems with curriculum codes. Those tags are available to every baseline in this thesis and are withheld from the system it proposes, which is a deliberate asymmetry rather than an oversight.

The release was rebuilt once into a canonical manifest before any result reported here was produced. Exact-duplicate rows — 31,441 of them, in 15,677 groups, which would otherwise have allowed an identical event to appear on both sides of a split — were removed; 5,104 rows without a valid timestamp were excluded from every history; and missing scores were masked rather than imputed. The stream used throughout is therefore 1,716,839 timed events over the 3,323 problems that survive the item split of Section 4.5.

The label is a mixture, and this shapes every correctness number in the thesis. A learner who requests the answer forfeits credit, so the probability that a recorded outcome is "correct" given that the answer was revealed is below one in a thousand. The recorded label is therefore the product of two events: that the learner attempted the item unaided, and that they solved it given the attempt. Measured on the full stream, the between-learner spread of the first exceeds that of the second, so a substantial part of the between-learner variance in "correct" is help-seeking temperament rather than knowledge. This is the collapse of *cannot* into *does not* that Section 3.2 separated with engagement, arriving as the dominant statistical fact of the primary corpus.

Two consequences follow, and both are conventions the rest of the thesis obeys. First, the system predicts a five-class outcome — unaided correct, unaided wrong, hint used, answer revealed, and both — so that engagement is a predicted quantity rather than a confound. Second, every correctness comparison is reported on one of two **courts**. The *marginal* court is the dataset's own benchmark, scoring the recorded label as it stands; it is inflated by the predictability of help-seeking by roughly 0.10 of AUC, for every model alike. The *unaided* court is restricted to events where no help was used, and is where knowledge is actually being predicted. Think of a race time reported with and without the runners who dropped out: both numbers are meaningful, they answer different questions, and one is never compared with the other. A number quoted from one court is never set against a number from the other anywhere in this document.

Baselines. The dataset ships its own benchmark. Four frontier language models of 70 to 120 billion parameters are given a learner's full interaction history as text and asked, zero-shot, to predict the correctness of the next response and to produce the learner's exact answer. They reach an AUC of 0.512 to 0.559 against a majority-class floor of 0.513, and exact-answer accuracy of 35.5 to 44.3% on the marginal court (Worden et al., 2026). Their evaluation set is rebalanced to 51.3% correct and requires at least fifty prior interactions; where a like-for-like comparison is made in Chapter 7, that regime is reproduced, and the comparable rows are the per-stratum exact-match rates rather than pooled accuracy, because the base rates differ. The dataset's authors additionally

propose two learner-free tasks — judging from a problem's text alone which of two problems is harder, and which is more discriminating — and evaluate the same models on them; those numbers are the comparators of Chapter 6.

Under a free-first policy, which uses published numbers and released artifacts wherever they exist so that compute is spent only where no such number does, exactly one knowledge-tracing model is trained from scratch in this thesis: attentive knowledge tracing (Ghosh et al., 2020), the strongest of the standard deep family, fitted on the clean stream with the dataset's skill tags and again with the tags removed, so that the value of the curriculum is measurable as a single-variable change. A majority-class baseline completes the family.

4.3 CodeWorkout: programming

CodeWorkout is a first-year university Java course, and its unit of behaviour is not an answer but an episode. A learner reads a short problem, writes a method and submits it; the system compiles it and runs a fixed test suite; the learner sees what failed and submits again. The logs were collected on Virginia Tech's CodeWorkout platform and are distributed in ProgSnap2 format through the PSLC DataShop (Virginia Tech CodeWorkout, 2019), with per-test-case outcomes and test suites recovered by Duan et al. (2025) for seventeen of the problems. The DataShop terms permit research use and forbid redistribution, and the code text is handled under the same locality rule as the mathematics corpus.

Two semesters are used — Spring 2019 with 413 learners and Fall 2019 with 506 — over a shared catalogue of 50 problems, giving 919 learners and 195,205 program-run events. Every event carries the full source snapshot, the compile result, the score, and a server timestamp, so a learner's trajectory on one problem is a chronological episode: attempt, feedback, revision, and either success or abandonment. The unit is the submission, not the keystroke. Within an episode the feedback is a deterministic function of the submission — a compiler and a fixed suite — which is the one place in either corpus where an intervention's assignment mechanism is fully known.

Every attempt is assigned one of four outcome classes: compile error; compiles but passes no test case; partial pass; full pass. The second of these — code that compiles and runs and is wrong on every test — is the class a tutor most needs to anticipate, and the one that item difficulty alone predicts worst; ranking it against the other three is the wrong-detection instrument used throughout the programming results. For the seventeen problems with recovered test suites, the full per-test-case pass vector, up to 32 test cases, is also available and is predicted directly.

Four published models supply the comparison: the test-case-informed knowledge-tracing model of Duan et al. (2025) and the open-ended model it extends, together with KCGen-KT (Duan et al., 2026) and Code-DKT. Their processed data and code are in hand, so the comparisons of Chapter 7 are computed on their rows. Because those models were trained on a cohort partly overlapping the one used here, the comparison is also run zero-shot on the external cohort as a transfer test.

4.4 What a learner actually produces

The two corpora differ in one respect that shapes every result in Chapter 7: the size and the freedom of the thing a learner makes. Table 2 makes that concrete, and it is worth reading before any number.

A mathematics answer is one or two tokens, and its space of plausible values is small: a fraction, a decimal, a small integer. A Java submission is tens of lines with free choice of variable names, control flow and structure. Two learners can write the same number for quite different reasons and

TYPE	WHAT THE LEARNER IS ASKED	WHAT THE LEARNER PRODUCES
FoundationalASSIST — mathematics, grades 6–8		
Fill in the blank (2,188 problems)	There are 12 inches in 1 foot and 5,280 feet in 1 mile. Jada ran $3\frac{1}{4}$ miles. How many feet is that?	a typed string: `17160`
Multiple choice, one answer (795)	A line contains the points $(-1, 4)$ and $(5, -3)$. Without doing any calculations, determine whether the slope of this line is positive or negative.	one of `Positive`, `Negative`
Multiple choice, select all (380)	Which scales are equivalent to 1 inch to 1 foot? Select all that apply. *Options:* 1 to 12 · $\frac{1}{12}$ to 1 · 100 to 12 · 5 to 60 · 36 to 3 · 9 to 108	a subset of the six options
CodeWorkout — introductory Java		
Programming problem (50)	Given two integers `a` and `b`, return their sum. However, sums in the range 10 to 19 inclusive are forbidden, so in that case return 20.	a complete Java method, submitted, compiled and run against a fixed test suite; then revised and resubmitted
Programming problem	Your cell phone rings. Return true if you should answer it. Normally you answer, except in the morning you only answer if it is your mother calling. In all cases, if you are asleep you do not answer.	the same, over a median of two attempts

Table 2: *Example problems from each corpus. On mathematics a production is a short string drawn from a narrow set of plausible values; on programming it is a whole program, revised across attempts in response to compiler and test feedback.*

the record cannot tell them apart; two learners rarely write the same program. Chapter 7 finds precisely this asymmetry in its measurements of personal identity, and the explanation is on this page rather than in the results.

4.5 Splits, courts, protocols, discipline

Two axes of generalisation. Every claim in this thesis is a claim about generalisation, and there are two different things to generalise to: learners the model has never seen, and problems it has never seen. They are separate axes and they are split separately. On mathematics the 5,000 learners are partitioned into 3,500 for training, 500 for validation, 500 held as *development-exposed* — an earlier test partition whose numbers were disclosed during development and which is therefore never used as a test again — and 500 in a sealed lockbox. On programming the 919 learners are partitioned into 686 for training, 110 for validation and 123 in the lockbox, the last being the challenge's own test partition, recovered exactly. On the item axis, 498 problem groups are withheld from all mathematics training, grouped so that near-duplicate variants of one problem fall on the same side of the split; on programming, seven problems are withheld, chosen disjoint from the seventeen with recovered test suites so that the two instruments do not overlap. Those seven are also late-course problems: for 87.6% of the learners who touch one, it is among their last events, so that stratum measures late-course transfer rather than random item generalisation.

The lockbox. A learner-by-item lockbox — 26,484 events over 500 mathematics learners, and the 123 programming learners of the release's own test partition — was sealed before the first model of this thesis was trained, and its hash was recorded. It has not been opened. **Every number in this document is a validation number.** The lockbox is opened once, at the end, on the pre-registered claims and on no others.

One rule about inputs. No model input may be normalised by, or derived from, an episode-level aggregate. The rule is absolute because the failure it prevents is invisible to an ordinary causal-masking check: a quantity computed over a whole episode and then attached to each step of it makes the episode's future recoverable from a prefix that is, step by step, perfectly causal. Two split canaries — a planted duplicate and a deliberately leaked row — are asserted caught by the split builder on every rebuild. Appendix A records the two leaks that produced the rule, together with the results they affected.

Approximating an interactive setting on frozen logs. The framework of Chapter 3 needs the one thing a log cannot give: answers to questions the modelling agent chose. Two protocols approximate it, and their costs are measured rather than hidden. In **held-out-pool replay**, a prefix of a validation learner's history is revealed and the belief is computed from it; a *menu* of later logged items is presented; the policy chooses one to ask; the learner's logged response on that item is the answer; the belief is re-filtered with it; and the process repeats for a budget of B asks. The policy is then scored on how well its final belief predicts a fixed set of *target* items that are strictly later than everything on the menu. This is how adaptive-testing policies are evaluated offline. The construction is governed by rules stated with the planner in Section 5.7, of which the most important is that the menu is recency-bounded and its geometry is printed beside every result, because a menu drawn from a learner's first week cannot inform predictions about their fortieth. In the **positive control**, every policy is first run on a synthetic world with a known answer key — simulated learners whose latent is planted and whose items have known diagnostic value — where the correct question is unambiguous. A policy that cannot find the diagnostic item there is broken as an instrument and its real-data numbers are not interpreted. The synthetic world is a control, never a setting in which a claim about learners is made.

Two regimes for generation. The simulator can be evaluated with nothing from the future in its prompt — the *blind* regime, comparable to any outside baseline — or with the true outcome of the event supplied so that only the production is scored — the *outcome-given* regime, which isolates the text channel for diagnosis. Every generation number in this thesis names its regime, and only blind numbers are compared with published ones.

Metrics. Outcome prediction is scored by AUC where a ranking is what matters, and by log-loss in nats and expected calibration error where the probabilities themselves are consumed by a planner. Production is scored on mathematics by exact match of the generated answer against the learner's typed answer and by the negative log-likelihood of the true answer; on programming by CodeBLEU against the published models, and by normalised edit distance and similarity in the space of meanings, because a single renamed variable fails a program that behaves identically and exact match is not a meaningful instrument at the level of a whole method. The value of asking is scored by the target-set log-loss after B asks, relative to a random policy on the same menus.

The four arms. Every personalisation claim in this thesis is decomposed by evaluating the same model on the same rows under four conditions: the learner's own belief; the adapter switched off, which is *no adaptation*; a history-length-matched *stranger's* belief; and the learner's own belief collapsed to its centre, which is *no uncertainty*. The difference between the second and the first is what adapting to *some* learner's history is worth. Between the third and the first is what knowing *which* learner is worth. Between the fourth and the first is what carrying a region rather than a point

is worth. Each term is also reported as a function of how much history the belief has consumed. This decomposition is the standing format of Chapter 7.

Intervals. A learner's events are correlated, so resampling rows overstates certainty. Every interval in this thesis is a learner-clustered bootstrap over 1,000 draws, and every paired comparison resamples learners rather than events.

Pre-registration and the three-round rule. Every claim was written down, with its gate expressed as a script containing the numbers it had to clear, before the run that tested it. A claim was allowed at most three declared attempts on the same split; a fourth round is a write-up rather than another experiment. Gates that failed as written are recorded as failed and were not rewritten afterwards. Nothing measured was deleted: a superseded number is moved, with its date and its reason, to an append-only history, and runs that produced no usable number are recorded with their cost.

Compute. All text was encoded once and cached. Everything that could run on a laptop — an Apple M4 with 24 GB — did; the personalised language model and the programming simulator were trained on single-GPU cloud instances. The whole programme cost 457 US dollars of cloud compute across 183 recorded actions, roughly a fifth of which went on runs that produced nothing.

What this hands to the next chapter. The two corpora between them supply meaning-bearing item text, real learner productions, a graded outcome with a separable help-seeking channel, order, population scale, and — on programming — an attempt loop with deterministic feedback. They do not supply an identified teaching intervention. Chapter 5 therefore builds an encoder over item text, a filter over ordered outcome-bearing events, a mechanism that turns a belief into a personalised generator, two prediction heads matched to the two corpora's outcome spaces, and a planner that asks; it builds no teaching component, and the reason is Table 1.

5 ATOM: the Architecture

This chapter specifies the system. It is called **ATOM**, and it has two halves that correspond to the two halves of the problem: **ATOM-Sim** reads a person from their history and simulates them, and **ATOM-Act** decides what to ask them next. The chapter describes the configuration that ships, in the order in which a prediction is actually produced, and gives for each component the reason it takes the form it does — what would go wrong otherwise — before any measurement is cited. Alternatives that were built and measured are named in passing and reported in Appendix A; this is a specification, not a history.

5.1 The shape of the machine

Section 3.3 argued that a modelling agent which respects the two-layer structure must compute in two stages: first infer where this person sits among all persons, then read the shared laws at that location. ATOM implements those two stages literally, and every component belongs to one of them.

The **shared layer** is everything learned from the whole population and identical for every learner: a frozen language model that reads the meaning of problems and writes text, a shared basis

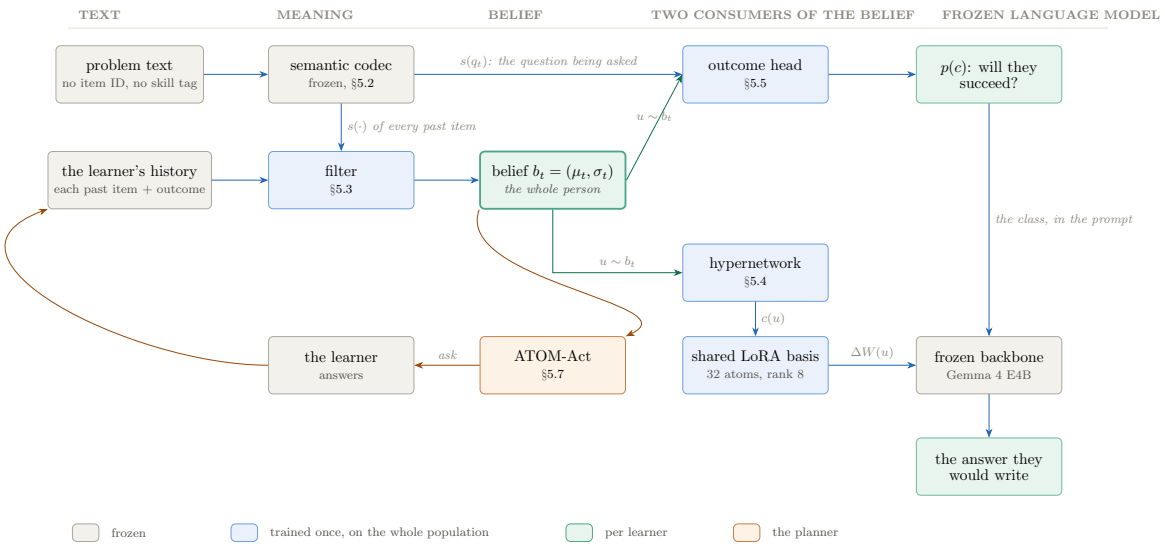


Figure 3: *ATOM, end to end. Two paths meet. Along the top, the problem being asked becomes a point of meaning. Along the second row, the learner's history of past items and outcomes becomes a belief — a centre and a width. The belief has two consumers: an outcome head, which combines it with the question's meaning to say whether this learner will succeed; and a hypernetwork, which turns it into a per-learner edit of a frozen language model, so that the edited model writes the answer this learner would write. Colour marks what is learned where: grey never receives a gradient, blue is trained once on the whole population, green is the only per-learner object, amber is the planner, which reads the belief, asks, and returns the answer to the history.*

of low-rank weight edits that spans the ways minds in this domain differ, and the heads that turn a location into a prediction. This layer is what a single learner's history could never supply, and what a language model reading one transcript does not have. The **personal layer** is one point with a region around it: the belief $b_t = \mathcal{N}(\mu_t, \mathrm{diag}(\sigma_t^2))$ over the person latent u. It is the only thing that is personal. No parameter of the shared layer is indexed by a learner, and no parameter of either layer is indexed by an item.

The bridge between them is a hypernetwork. It takes coordinates in the space of minds and returns a mind, in the form of a weight edit applied to the shared model. Placing the learner is the filter's job; *being* the learner is the edited model's job. Figure 3 shows the whole machine before any of it is explained.

Reading the figure left to right is reading the computation. A problem's text is passed once through a frozen encoder and becomes 64 numbers that stand for what the problem is about; the same encoder has already turned every problem in the learner's past into 64 numbers of its own. A recurrent filter walks that history — each past item paired with what happened on it — and emits a centre and a width, which together are the belief: where this mind probably is, and how sure the system is. From the belief, two things are read. A small head combines a sample of the belief with the meaning of the question now being asked and returns a distribution over outcomes: will this learner get it right, or ask for a hint, or give up. Separately, a hypernetwork turns the same sample into 32 mixing weights, which combine a shared basis of low-rank matrices into one weight edit applied to a frozen 8-billion-parameter language model. That edited model, prompted with the problem and with the predicted outcome, writes the answer this learner would write. Underneath, the planner reads the belief and decides which question to spend next; the learner's answer re-enters the filter, and the belief tightens.

The backbone is Gemma 4 E4B and it is frozen, for three reasons given in order of how much

Component	Parameters	Status	What it is
Language backbone (Gemma 4 E4B)	7,941 M	**frozen**	reads and writes text
Semantic codec transform	—	**frozen**	2,560 numbers $\to$ 64
Belief filter (with its item encoder)	0.61 M	trained	history $\to (\mu, \log \sigma)$
Hypernetwork	0.02 M	trained	$u \to 32$ mixing weights
Shared low-rank basis (66 projections)	72.61 M	trained	mixing weights $\to$ weight edit
Outcome head	1.50 M	trained	$(s(q), u) \to$ class distribution
Total trainable	**74.74 M**		**0.9% of the stack**

Table 3: *The mathematics simulator by parameter count. The programming simulator has the same shape with smaller widths (Table 4).*

they bind. It is frozen because the encoder of Section 5.2 is a fixed linear map read off this model's hidden-state geometry, and any gradient reaching the backbone would move that geometry underneath a transform computed once and cached. It is frozen because the division of labour is the entire proposal: the backbone supplies what no population of learners could teach it about language and about mathematics or Java, and the belief supplies what no amount of pretraining could know about this person; letting the backbone learn the corpus would blur that line and make the personalisation claim unmeasurable. And it is frozen because 7.94 billion parameters cannot be fine-tuned within this project's budget. Table 3 shows what that leaves: 74.74 million trainable parameters, 0.9% of the stack.

The rest of the chapter follows Figure 3. Section 5.2 builds the encoder, Section 5.3 the filter, Section 5.4 the bridge from a location to a mind, Section 5.5 the two heads, and Section 5.6 states how the whole thing is fitted and how it differs on the two corpora. Section 5.7 is the planner, and it is the longest, because it is where the title's reinforcement learning lives.

5.2 Reading meaning instead of looking up an index

Every textual artifact in the system — a problem statement, a typed answer, a code submission — is mapped to a point of the space of meanings by a frozen encoder $s(\cdot)$, which is how ATOM realises Assumption 3.2.

The construction is a fixed linear map read off a language model's internal states. A text is passed once through the frozen backbone; the hidden states at layer 15 of 42 are mean-pooled over the text's tokens, giving 2,560 numbers. The corpus mean is subtracted, the leading eigenvectors of the covariance are computed, and the top five directions are discarded, leaving directions 5 through 68: **64 numbers per item**. The map is computed once over the corpus, cached, hash-checked at every training launch, and never updated by any gradient. On mathematics it is fitted on the 3,323 unique problem bodies. On programming the *same* mathematics-fitted transform is applied, without refitting, to Java problem statements and to code.

Why this, and not an embedding table. Section 2.2 isolated the structural limitation of the classical knowledge-tracing family: the item side of those models is a lookup. Each item, or each skill tag it carries, indexes a row estimated only from responses to that item. Four consequences follow, and the encoder removes all four by construction rather than by regularisation.

1. **A new item has no row.** A problem written this morning cannot be scored by a model whose knowledge of problems is a table, until learners have answered it. Under the encoder,

a new problem has a representation the moment its text exists, because the representation *is* a function of the text. This is a property of the parameter count, not a claim about generalisation.

2. **Relatedness has to be asserted by a human.** A table has no notion of two items being similar unless a curriculum says so. Here relatedness is proximity, and the curriculum's own grouping turns out to be recoverable from the geometry without ever being supplied (Chapter 6).
3. **The learner's representation inherits the table's coverage.** If items are indexed, what the model knows about a learner is expressed in terms of the particular items that learner met. A belief built over item *meanings* is expressed in terms that transfer to items nobody has met.
4. **The scarcity is in the wrong place.** A table spends capacity per item and therefore needs many responses per item. A shared function over meaning spends its capacity once and needs many items.

There is a price, and it is stated here as a fact rather than defended later. Roughly two-thirds of the explainable variance in a problem's empirical difficulty is not expressed in its text at all — it lives in how the problem was presented, where it sat in a lesson, and everything else a table can silently memorise. An item-agnostic representation cannot recover that. It therefore loses something on items the model has seen, and loses less on items it has not; Chapter 6 measures both halves of that trade.

The 64 numbers do not enter the model raw. A small **item encoder** — $\text{Linear}(64 \to h) \to \text{ReLU} \to \text{Linear}(h \to h)$, a shared function with no item-indexed parameter — reads them, and its output is what the filter and the heads consume. It is nonlinear because the frozen geometry is organised by meaning and is not axis-aligned with difficulty: the directions that distinguish a ratio problem from a slope problem are not the directions along which either gets harder, so an affine read has to spend one linear map on both jobs. That the encoder is shared is the architectural commitment; that it is nonlinear is a measured choice, and the measurement is in Chapter 6.

Provenance. The encoder is adopted from a companion paper of the author's (Godoy Ortiz, 2026), which establishes across four model families that removing the dominant high-variance directions of a frozen language model's hidden-state distribution exposes a semantic ordering invariant to phrasing, format and language. Those properties are cited here, not re-derived, and no result of that paper is presented as a result of this thesis. What is new here is the use: this is the first system to make such a representation the item side of a learner model, and thereby to do without a curriculum entirely.

5.3 From a history to a belief

What an event is. Chapter 4 described two corpora that record different things, and the filter's input reflects that. On mathematics the event at position t carries the encoded meaning of the item faced at $t-1$ and five binary channels describing what happened on it: whether it was scored correct, whether a hint was requested, whether the answer was revealed, whether the position is valid at all, and whether the score was missing. On programming an event is a whole episode — one problem's attempt loop — summarised by a within-episode encoder described below. Skill tags never appear on either corpus. What the learner produced enters the two corpora differently, and the difference is a consequence of what each records. On mathematics the typed answer is

supervised at the output and never consumed at the input, so the belief there is built from what a learner achieved rather than from a copy of what they wrote. On programming the production is the evidence: the attempt encoder reads the encoded meaning of each submission and of its diff from the previous one, because a Java method carries far more about its author than a two-token answer does.

The filter. A gated recurrent network runs over that sequence and emits two vectors,

$$h_t = \mathrm{GRU}(h_{t-1},\, e_t), \qquad \mu_t = W_\mu h_t, \qquad \log \sigma_t = W_\sigma h_t \;\in\; [-5,\, 2], \tag{12}$$

so that the belief is a diagonal Gaussian,

$$b_t \;=\; q(u \mid h_t) \;=\; \mathcal{N}\big(\mu_t,\, \mathrm{diag}(\sigma_t^2)\big), \qquad u \in \mathbb{R}^{256}, \tag{13}$$

which is the amortised form of Definition 3.4. Samples are drawn by reparameterisation, $u = \mu_t + \sigma_t \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$: one sample during training, eight at evaluation, and thirty-two for the classification panels, with common random numbers shared across every arm of a comparison so that a difference between arms is never noise in ϵ.

The width is not decoration. Three things depend on σ_t and none of them survives its removal. It is the object the planner reads: a question is worth asking precisely to the extent that the belief is wide along the directions a prediction depends on (Section 3.5), and a planner given only μ_t has no way to tell a confident belief from an ignorant one. It makes every prediction an average over plausible versions of the learner rather than the prediction of one guessed learner, which is what Definition 3.5 asks for. And it is what a cold start looks like from the inside: with no history the filter returns the prior, and the region is wide because the population is wide.

Why a Gaussian. The choice of family is a choice about what the coordinates mean, and three mechanical properties decide it. First, a Gaussian is the family whose samples are a differentiable function of its parameters and a fixed noise draw, which is what allows K versions of a learner to be drawn, turned into K networks, and run in one batch — without that, the region cannot reach the output at all. Second, its width is one number per axis, which is a quantity a policy can read as a feature; a posterior represented by particles or by a flow has no comparable summary to hand a planner. Third, its divergence from a fixed reference is available in closed form, which is what makes it possible to *force* the latent space to be smooth rather than hope that it is.

That last property matters more than it sounds. Section 3.3 argued that a belief over the person latent is only interpretable if nearby coordinates denote nearby minds. Were the population to occupy isolated islands, a Gaussian region would place mass on coordinates corresponding to no possible person, the width would mean nothing, and averaging over samples would average over fictions. The architecture therefore enforces coherence through the regulariser

$$\mathcal{L}_{\mathrm{KL}} \;=\; \sum_{d=1}^{256} \max\Big(\mathrm{KL}\big(\mathcal{N}(\mu_{t,d}, \sigma_{t,d}^2) \,\|\, \mathcal{N}(0,1)\big),\; \lambda_{\mathrm{fb}}\Big), \qquad \lambda_{\mathrm{fb}} = 0.05, \tag{14}$$

which pulls every learner's belief toward one shared unit-variance prior. This is the discipline of a variational autoencoder applied to persons: the population is compelled to fill a single smooth region rather than to scatter, so that interpolation between two learners passes through learners. The per-dimension floor λ_{fb} — "free bits" — prevents the opposite failure, in which the regulariser

wins outright and the belief collapses to the prior, carrying no person at all. Its weight is annealed linearly from zero to 0.01 over the first thousand optimisation steps, so that the model learns to use the latent before it is asked to economise on it.

The covariance is diagonal, which asserts that the axes of u are independent given the history. This is a computational commitment rather than a claim about minds: a full covariance would be 256×256 numbers per learner per step, recomputed at every event.

The belief is query-independent. The belief at time t is a function of the history only: the item about to be asked never enters the recurrent path. This is enforced by construction, since the recurrent input at position t carries the item at $t-1$, and it is verified by a canary that swaps the next question while holding the history fixed and asserts that the belief moves by exactly zero. Section 3.4 gave the reason: the value of asking a question is the change it would produce in a belief that did not already know it was coming, so a belief that peeks is a prediction in disguise and the planner built on it is incoherent. Relatedly, every prediction at step t is made from events strictly before t, so cold-start curves report what a deployed system would actually experience.

The programming variant runs on two clocks. A learner does not answer a Java problem once; they attempt it repeatedly, with compiler and test feedback in between, which is the framework's distinction between a stable disposition and a moving state showing up in the data. The *fast clock* is a recurrent attempt encoder inside one problem. It consumes each attempt as the encoded meaning of its code, the encoded meaning of its diff from the previous attempt, and four scalars — did it compile, what fraction of tests passed, how many compile errors, which attempt this is — and it emits two things: the state entering each attempt, built only from strictly earlier attempts, which the outcome head reads as local evidence; and a 16-number summary of the whole episode. The *slow clock* is the same recurrent filter as (12), consuming at each episode the *previous* episode's problem meaning and summary, producing one belief per episode that is broadcast to its attempts.

Two further differences are consequences of the corpus. The outcome head additionally reads the within-episode prefix state, so on programming it predicts $p(c \mid u, s(q), h^{\text{prefix}})$. And the belief's width is **monotone** by construction: rather than emitting $\log \sigma$ freely, the filter accumulates precision, $\sigma_t^{-2} = 1 + \sum_{s \le t} \text{softplus}(w \cdot e_s)$, so evidence can only narrow the region. This is a deliberate restriction of the general form and it encodes a fact about the setting: within a single course, over a few weeks, there is no forgetting channel that should widen a belief again. The unconstrained parameterisation is retained on mathematics, where histories are much longer.

Think of the belief as a satellite fix. Each observation is a reading; the fix tightens as readings accumulate; and a fix with no error ellipse is useless to anyone planning a route.

5.4 From a location to a mind

The shared layer is materialised as a low-rank edit of a frozen language model, generated from the belief. Three pieces do it.

A low-rank adapter, in three sentences. Fine-tuning a large model means changing its weight matrices, which is expensive and destroys the model one started from. A low-rank adapter instead leaves every weight matrix W untouched and adds a correction BA, where A and B are thin matrices whose product has the same shape as W but far fewer free parameters (Hu et al., 2022). The model's behaviour can then be changed substantially by a small object, and reverted exactly by setting the correction to zero.

A shared basis. For each adapted weight matrix the model holds $m = 32$ low-rank pairs $\{A_i, B_i\}$ of rank $r = 8$, trained on the whole population. These are not one adapter per learner; they are a basis that spans *the ways a learner can be*. Adapted projections are every query and value projection of the text decoder — 42 and 24 respectively, 66 in total. Think of the frozen model as a piano and the basis as a set of 32 pedals: nobody rebuilds the instrument, and a person is a particular pressure on each pedal.

A hypernetwork. A small map $H_\xi : \mathbb{R}^{256} \to \mathbb{R}^{32}$, Linear$(256 \to 64) \to$ GELU $\to$ Linear$(64 \to 32)$, turns one belief sample into the 32 mixing weights $c(u)$. It runs once per learner rather than once per layer, which is the point: one location in the space of minds produces one *coherent* edit across the entire network, rather than 66 independent adjustments that need not describe the same person. The edit is

$$\Delta W^\ell(u) \;=\; \frac{\alpha}{r} \sum_{i=1}^{32} c_i(u)\, B_i^\ell A_i^\ell, \qquad y \;=\; W^\ell_{\text{frozen}}\, x \;+\; \Delta W^\ell(u)\, x. \tag{15}$$

The product ΔW is never materialised as a matrix. It is applied as a sequence of contractions against the activations, which is what makes running a different network for every learner affordable inside a single batch — materialising 66 per-learner matrices for eight sampled learners would exhaust the device before any forward pass began.

Zero initialisation, and what it is not. The B matrices are initialised at exactly zero, so before training the edit is a verified no-op and the model reproduces the unedited backbone bit for bit. Personalisation therefore grows out of a state that is provably not personalised, which is what makes the adapter-off arm of the four-arm decomposition an exact control rather than an approximate one. Two things that look alike must be separated here. **Before any evidence, the belief is the population prior; before any training, the mind is the bare backbone.** The prior belief $\mathcal{N}(0, I)$ pushed through a *trained* hypernetwork gives a distribution over edits, not the zero edit — a cold-start learner is still placed in the middle of a population that the basis has learned about. The zero edit is a fact about initialisation; the prior belief is a fact about inference. They coincide only at the start of training, and personalisation is the distance between them.

Why weights and not context. The literature offers three carriers for a belief about a person (Section 2.3): activations, neighbours, weights. ATOM uses the first as its *filter*, compressing a history into a vector, and the third as its *binding*, and it does not use the second at all. The reason to edit weights rather than to extend the prompt is the objection of Section 2.2: a context window is a belief with no sufficient statistic. It grows without bound; it truncates just when a history has become most informative; it collapses the posterior to whatever the transcript happens to say; and it offers no place to put a width. A weight edit generated from a fixed-size belief costs the same at inference whether the learner has ten events or four hundred, carries a region because the belief does, and keeps the personal state in an object the planner can read.

What the three pieces together produce is the object Chapter 3 called the simulator: the modelling agent's stand-in for this learner's world model, made runnable. The frozen backbone supplies what any competent writer of this domain knows; $\Delta W(u)$ moves it to the point in the space of minds the filter has inferred; and what comes out generates as that learner would.

The region survives to the output. Drawing K samples from b_t yields K personalised models in one batched pass. Their average is the prediction; their disagreement is the epistemic uncertainty of that prediction, which is the person term of (7) made computable. This is the point at which the belief stops being a representation and becomes a quantity a decision can be made from.

5.5 What the person does: two heads

The outcome head reads the belief, not the language model. A small network takes the current item's encoded meaning and one belief sample and emits a distribution over outcome classes:

$$f = \mathrm{ReLU}\big(\mathrm{Enc}(s(q_t))\big), \quad \gamma = 1 + W_\gamma u, \quad \beta = W_\beta u, \quad p(c \mid u, q_t) = \mathrm{softmax}\big(W_o\,\mathrm{ReLU}(\gamma \odot f + \beta)\big), \tag{16}$$

a feature-wise modulation of the item's representation by the person: the learner scales and shifts the problem. It is initialised near the identity, so an uninformative belief leaves the item's own difficulty intact and personalisation is a departure from a sensible default rather than from noise.

This head never touches the language model, and the reason is mechanical rather than empirical. Estimating a probability of success is a calibration problem: the number that comes out must be a frequency over a population, and it must remain one after the model is confident. A text generator exposes no such quantity. Its hidden states are optimised to make the next token likely, and the margin by which one token beats another is not a calibrated statement about the world; reading a probability off them requires a fitted map that has no population statistics behind it. Predicting whether a learner will succeed and predicting what they will type are different estimands, and giving each its own output path is what allows the first to be calibrated against a population while the second remains a language model. The claim can be stated in one line: **estimation lives on the belief; the language model renders text.** Appendix A reports the ablation that agrees.

On mathematics the classes are the five of Section 4.2 — unaided correct, unaided wrong, hint used, answer revealed, and both — so that help-seeking is predicted rather than confounded, and every correctness quantity in Chapter 7 is derived from this one distribution: the probability of being marked correct is the first class; the probability of solving given an unaided attempt is the first renormalised against the second; and the probability of seeking help is the remaining mass. On programming the classes are the four of Section 4.3, and the generative simulator carries an additional *test-vector head* predicting, for each of up to 32 individual test cases, whether this submission will pass it.

The production head is the backbone's own vocabulary head. The learner's actual answer is generated by the personalised model, and what goes into its prompt is the whole design. The prompt contains the problem's cleaned text, its options where it has any, and one phrase naming the outcome class — and nothing else. It contains no learner identifier, no history rendered as text, no reference answer and no skill tag, all of which appear in the prompts of the published baselines. Everything the model knows about this learner arrives through $\Delta W(u)$, which is what makes the comparison in Chapter 7 a comparison of mechanisms rather than of prompt engineering. Training is teacher-forced on the learner's real first response, with the prompt tokens masked from the loss so that the model is scored only on the answer. On programming the prompt is the problem statement, the previous attempt, and the feedback that attempt drew; a first attempt has no prior code, so its prefix state is a fixed zero vector and its prompt carries a fixed phrase saying so.

The two predictions, jointly. The class and the text form one distribution,

$$p(c, a \mid h_t, q_t) \;=\; \mathbb{E}_{u \sim b_t}\big[\, p(c \mid u, q_t)\; p(a_{1:L} \mid c, u, q_t)\,\big], \tag{17}$$

estimated as a mixture over both the K belief samples and the classes. Equation (17) is what makes the two regimes of Section 4.5 precise. In the *blind* regime the head predicts the class, that prediction is placed in the prompt, and the generator writes the answer; nothing from the future is used, and only these numbers are compared with published baselines. In the *outcome-given* regime the true class is supplied and only the production is scored, which isolates the text channel for diagnosis.

5.6 Training, and the two instantiations

The trainable stack — filter, hypernetwork, basis, outcome head — is fitted against one objective per learner sequence,

$$\mathcal{L} \;=\; \underbrace{\mathcal{L}_{\text{outcome}}}_{\text{cross-entropy on the class}} \;+\; \alpha \underbrace{\mathcal{L}_{\text{prod}}}_{\text{token loss on the answer}} \;+\; \beta_{\text{KL}}(\text{step}) \underbrace{\mathcal{L}_{\text{KL}}}_{\text{Equation (14)}}, \tag{18}$$

with the belief entering both likelihood terms through the *same* sample, so that gradients from the text a learner wrote and from the outcome they achieved reach the same latent and cannot describe two different people.

What is trained together, and what is initialised from where. On mathematics everything trainable is optimised jointly, in one optimiser, from the first step: the filter, the hypernetwork, the adapter basis and the outcome head all receive gradient from (18) throughout, with the backbone frozen. The one thing that is not started from scratch is the filter, whose weights are initialised from a checkpoint of a separate, much cheaper run that trained the filter and its item encoder against the outcome objective alone, with no backbone forward pass anywhere in it. That initialisation is a warm start and not a first phase: the filter is not held fixed afterwards, and the hypernetwork, basis and outcome head all begin from scratch inside the joint run. The reason for warm-starting rather than beginning cold is that the token loss is large and noisy next to the outcome loss, and a filter that has never yet learned to encode a person would spend its early gradient steps encoding whatever makes the next token likely; starting it at a point where the latent already separates learners means the joint objective refines a belief rather than inventing one.

The shipped mathematics configuration is **FA-SIM3**: $\alpha = 0.3$; Adam at 10^{-4} with gradients clipped at unit norm; micro-batches of two sequences with eight-fold accumulation; one full shuffled pass over 239,181 supervised events drawn from all 3,500 training learners, which is 14,949 optimisation steps; and the belief always computed over each learner's *entire* valid history rather than over the sampled subset, so that a sampled event is scored under the belief a deployed system would actually hold at that moment. The run occupies a single 48 GB GPU for about nineteen hours.

The programming stack ships as two checkpoints with a clean division of labour, because the two corpora put their difficulty in different places. The **classifier** — attempt encoder, cross-episode filter and outcome head — is trained alone for 60 epochs with an eight-sample mixture likelihood and no backbone forward pass at all. It is the production predictor for every classification number

in Chapter 7 and the belief substrate the planner of Chapter 8 runs on, and it is trained separately because a planner needs a belief whose meaning is not entangled with a token loss. The **generative simulator** then loads that checkpoint and adds the hypernetwork, a 16-atom rank-8 basis over the same 66 projections, and the test-vector head; all of these, and the loaded components, train jointly for two epochs with the token loss at weight 1.0, the outcome loss at 0.3 and the test-vector loss at 0.1. One further step is part of the shipped configuration rather than a repair: the outcome head used at generation time is refreshed — retrained alone on the frozen belief — because the head that emerges from joint training is measurably worse at the class that matters most on this corpus, code that compiles and fails everything.

Two failure modes are designed against. Posterior collapse, in which the regulariser wins and every learner receives the prior, is prevented by the free-bits floor of (14) and monitored throughout training by the between-learner variance of μ. Degeneracy of the shared basis, in which all 32 atoms drift toward one direction and the hypernetwork's output stops distinguishing people, is prevented by the zero initialisation of B together with the gradient path through $c(u)$, and monitored by a canary asserting that the mixing vector actually varies across learners by more than it varies across noise draws at a fixed learner.

	Mathematics (FA-SIM3)	Programming
Belief dimension	256	32
Width parameterisation	direct, clamped	precision-accumulating (monotone)
Event unit	one item, one answer	one episode of attempts (two clocks)
Outcome classes	5 (help-seeking separated)	4 (compile / zero / partial / full)
Production	the learner's typed answer	the learner's Java submission
Extra head	—	per-test-case vector (32 slots)
Basis atoms × rank	32×8	16×8
Token-loss weight α	0.3	1.0

Table 4: *One architecture, two instantiations. Every difference is a consequence of what the corpus records, stated in Chapter 4; none of them is a separate design.*

5.7 ATOM-Act: the planner

The simulator exists to be acted upon. This section states the decision problem the planner solves, then the six families of policy implemented for it. The framework's teaching operation is not among them, for the reason given in Section 4.1: no corpus supplies interventions with a known assignment mechanism.

5.7.1 The environment

State. The planner's state is the belief and the budget:

$$x_t \;=\; \big(\mu_t,\; \sigma_t,\; \text{step},\; \text{budget},\; \text{asks remaining}\big), \tag{19}$$

summarised for the trained policies as eight scalars, together with a per-candidate description of every question on the menu: its encoded meaning, the predicted distribution over its outcome under the current belief, that distribution's entropy, and the spread of that prediction across the K belief samples. The width σ_t and the remaining budget are both mandatory, and for reasons that do not

overlap. Without the width, a policy cannot behave differently because it is unsure — it has no access to the quantity that makes a question valuable. Without the remaining budget, it cannot know whether this is its last question, and the best question to ask when five remain is not the best question to ask when one does.

Actions. A question is an action through its meaning. A policy scores each candidate as $f(x_t, s(q))$ and takes a softmax over the menu, so the same policy handles menus of any size and any content and never learns an item identifier. The action space is therefore the space of meanings itself. Whether a question can be *constructed* in that space rather than only selected from it is a separate question, taken up at the end of this section.

Transition. Selection is hypothetical; execution is real. While scoring candidates the planner imagines their answers under its own belief and no update occurs. Once it commits to a question, the answer comes from the learner — in replay, their real logged response — and the belief is updated by re-running the frozen filter over the prefix and everything revealed so far, in true chronological order. The policy cannot decline an update or edit one. The belief, the heads and the backbone are frozen throughout: **the planner trains a policy, never the simulator.**

An episode. An episode is a start prefix, a budget, and a target set. "Start 3, budget 8" means that the learner's first three logged events are revealed to form the initial belief, that the policy may then choose eight questions from the menu, and that it is scored on a fixed set of the learner's later interactions that lie strictly after everything on the menu. That vocabulary is used unchanged in Chapter 8. Figure 4 walks one through.

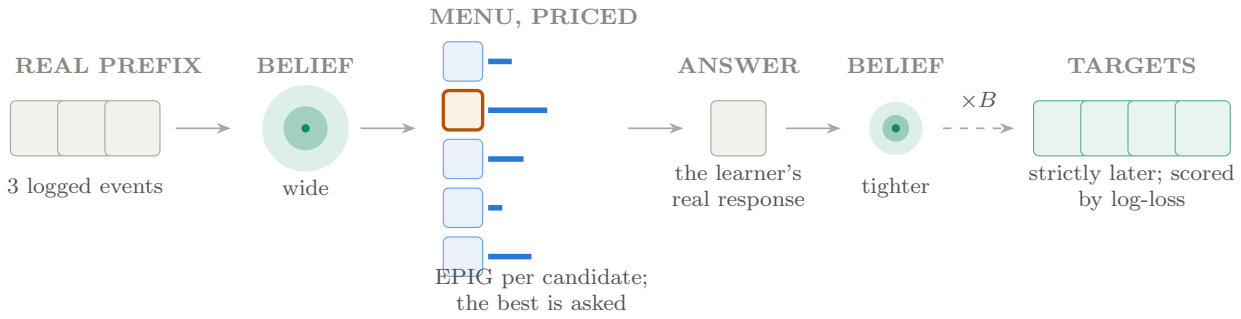


Figure 4: *One planner episode. A prefix of the learner's real history forms a wide belief; every question on the menu is priced by how much its answer would sharpen the prediction on the targets; the chosen question is asked and the learner's real logged answer is revealed; the belief tightens; and after B such asks the belief is scored on a fixed set of the learner's later interactions. Bar length is the price of a question, and no target is ever inside the menu.*

The reward. Two currencies are used and both are reported. The *external* one is the target set's log-loss under the resulting belief: what the planner actually bought. The *internal* one is convergence — the distance between the current belief's predictions on those targets and the predictions of the belief formed from the learner's entire history, which is the best any amount of asking could achieve. Trained policies are rewarded by the per-step decrease in that distance rather than by the final value alone. This is a potential-shaped reward: the total reward over an episode telescopes to a function of the start and end states only, so adding it changes what a policy learns from but not

which policy is optimal. It is the difference between paying a climber for each metre gained and paying only at the summit; the summit is unchanged, and the climber learns faster.

Pricing a question. The value of a candidate before it is asked is the expected value of the information it would supply. Section 3.5 showed that when the decision is a prediction over a target set, this reduces to the mutual information between the candidate's outcome and the targets' outcomes. Instantiating that with the simulator's own committee of K sampled learners gives an estimator that needs no re-filtering and no language-model call:

$$\widehat{\mathrm{EPIG}}(e) \;=\; \hat{I}(Y_e; S) \;+\; \hat{I}(Y_T; S) \;-\; \hat{I}\big((Y_e, Y_T); S\big), \tag{20}$$

where S indexes the committee, Y_e is the candidate's outcome and Y_T the targets' outcomes. Each of the three terms is a Jensen gap — the entropy of the mixture minus the mean entropy of its members — computed from the same K predictions the simulator has already produced, so the whole price of a menu is one forward pass. Two properties are worth stating. The estimator is non-negative for finite K by construction. And it satisfies the invariant $\mathrm{EPIG}(e) \leq H(p_e)$: a question cannot be worth more information than its own answer contains. That invariant is asserted at every evaluation, and a violation raises rather than warns, because a violation means the committee has been mis-assembled and every number downstream of it is meaningless.

Ask or act. Equation (8) makes stopping the same arg max as asking, given a cost per question. No defensible cost exists in these corpora — nobody recorded what a learner's patience is worth — so the cost is **swept** rather than assumed: policies run to a fixed budget, and the deliverable is the stopping curve, showing where the marginal question stops being worth its price over a grid of costs and two fatigue schedules. No policy in this thesis has a learned stop action.

Five rules bind the replay harness , and they are reported with every result. Every target is strictly later than every candidate. The menu is recency-bounded, and its geometry — its position in the history, its distance from the targets, its overlap with the target items — is printed beside each result. An event where the learner revealed the answer is never askable, because no policy could target it at selection time and its outcome carries no knowledge. Every policy sees the same menu and the same random draws, so comparisons are paired. And a random policy is always present, with its own improvement measured *first*, as the headroom any policy is competing for.

5.7.2 Six families of policy

The simulator is an environment, and the thesis's question is which kind of policy can use it. Six families were implemented. The first three need no learning and set the bar; the fourth is the title's reinforcement learning; the fifth and sixth ask whether a policy can be trained without real data at all, and whether a language model can do the job.

Blind heuristics. *Random* prices the harness and defines the headroom. *Maximum entropy* asks the question whose own outcome is least predictable, which is the classical uncertainty-sampling comparator; it cannot tell genuine ignorance about the person from inherent randomness in the item. *Maximum epistemic spread* asks the question on which the committee of sampled learners most disagrees, which does separate the two; it is the stronger of the pair and the one that must be beaten. *Difficulty-matched* selection asks a question of the same predicted difficulty as

the value-of-information choice but a different question, which rules out the explanation that an information-seeking policy is merely picking hard items.

Greedy value of information. At each step, score every remaining candidate by (20) and take the best. This is the decision-theoretic policy of Chapter 3 implemented directly, with no learning at all, and it is the natural incumbent: it is what a correct one-step calculation recommends, and a trained policy that cannot beat it has not earned its training. A pre-registered variant chooses uniformly among the top five rather than taking the maximum, as a test of whether taking an arg max over many noisy estimates is itself the problem.

The privileged oracle. At each step, actually reveal each candidate's logged outcome, re-filter, and keep the one that moves the belief closest to the fully-informed belief. It reads the answers before choosing and is therefore not deployable. It is not a baseline but an existence proof: it measures how much of the gap between random and perfect a policy that saw the future could close, and therefore says whether the ceiling is set by the policy or by the questions the menu happens to contain.

The trained actor. A pointer network scores each candidate from the state of (19) and its per-candidate features, and is trained by policy gradient with a learned baseline: actions are sampled from the softmax, the per-step reward is the reduction in distance to the fully-informed belief, and returns are undiscounted over the episode. Budgets are drawn per episode from a grid, so a single actor is budget-conditioned rather than one actor per budget. The critic is *privileged*: it may see the fully-informed belief, which the actor may not. This is legitimate because the critic is discarded after training and never participates in a decision — it exists only to reduce the variance of the gradient by telling the actor how good a state was, and a lower-variance gradient is worth a great deal on episodes of length three. The actor sees only what a deployed system would see, and that boundary is enforced in code rather than by convention. It is the reason the actor is a claim about deployment rather than about replay.

Training inside the simulator. The strongest possible use of a behavioural twin is to train a policy entirely inside it and deploy the result on real learners. The simulator is run as an environment: outcomes are sampled from a frozen learner's own outcome head, the content of each answer is drawn from a corpus the simulator itself generated, and checkpoint selection uses simulated validation only, so that not one real interaction enters fitting or selection. The policy is then examined on real learners. A weaker relative — imagination as *augmentation*, one imagined branch added to each real episode — was also implemented. One rule governs both: **imagination never cold-starts**. Training episodes begin from a belief carrying at least three real episodes, because at an empty history the filter returns the prior, every learner's first decision is therefore identical, and a policy trained there is learning from a person-blind state — it would discover which question is best on average and nothing about matching a question to a person. The analogy is a flight simulator: a pilot may be trained entirely inside one, but the test is a real flight, and a simulator that always starts on an empty runway teaches nothing about landing.

Language models as policies. Three tiers, in increasing ambition. In the first, a frozen model is placed in the same replay harness as every other policy and asked to *choose* from the menu, given the learner's revealed history in prose and the remaining budget; its decision is read as the

arg max over the menu's identifier tokens, so an off-menu answer is structurally impossible, and an ablation replaces the history with a fixed sentence to isolate what the history contributes. A second tier asks the model to *write* a diagnostic question conditioned on the history, and a third to write novel problems for a simulated evaluation; both are gated, before any downstream measurement, on construct-validity floors asking whether the generated questions occupy the same region of the space of meanings as real ones and whether they are diverse. Finally, the selecting model is fine-tuned by low-rank adaptation against the exact one-step advantages computed inside the frozen simulator — the same reward the pointer actor receives, delivered to a different kind of policy. A companion utility model is trained on the same signal but built to score a question from its *meaning* rather than its identity, which is what makes it possible to ask whether a price learned on one set of items transfers to items it has never scored.

Question *generation* in the strict sense — constructing a probe as a point of the space of meanings that no item realises — was implemented only as a demonstration in the planted world, where a candidate is interpolated between two real menu items and rescored. It measures what a constructed question could be worth; it produces no text and makes no claim about learners.

5.8 What the architecture computes, and what it does not

Chapter 3 defined an ideal. Table 5 states, object by object, how much of it this system computes, so that the results chapters can be read against the ideal rather than against an impression of it. Every entry in the right-hand column is a limit this thesis reports rather than a component it omits quietly, and Chapter 9 takes them up in order.

Framework object	As computed here	What is missing
Belief $b_t = p(u \mid h_t)$	amortised diagonal Gaussian over one undivided u, query-independent	traits and moving state are not separately identified; the shared laws are a point estimate, not a posterior
Simulator $\hat{\pi}(\tau \mid q, b)$ (the learner's world model, made runnable)	the personalised language model, marginalised over K belief samples	the endpoint is modelled, not the path: no trajectory of reasoning is predicted
Success map $\hat{K}_g(b)$	the outcome head's class distribution	competence at best effort is separated from elicited performance only as far as the help-seeking classes allow
Transition law T_ϕ	**not implemented**	no effect model; the belief moves by evidence only, never by an action's consequence
Three-way variance	the person term, as K-sample spread	the law term is not computed; there is no uncertainty gate
EVOI(e)	Equation (20), targeted at a prediction	the general decision-targeted form is not used
Discovery policy	greedy value of information, and a trained budget-conditioned actor	one-step lookahead; no search, no critic trained inside imagination
Extension policy	**not implemented**	no teaching action; the data cannot identify one
Population prior over persons	$\mathcal{N}(0, I)$ with free bits	no learner graph, no cold-start prior from peers

Table 5: *The ideal of Chapter 3 against the system of this chapter. The middle column is what ships; the right-hand column is what the results may not be read as evidence about.*

What is new here. Individually, every mechanism above has precedent: hypernetworks that generate adapters, low-rank weight mixtures, variational recurrent filters, amortised experimental design. What does not have precedent is their coupling under one discipline. A variational filter over a person latent, trained on a whole population, is decoded through a shared adapter basis into a per-learner edit of a frozen language model; the representation of every item is a function of its text, so no parameter is indexed by an item and no curriculum enters anywhere; the belief is a region and carries its width from the first observation through to the rule that decides what to ask next; and the policy that asks is trained inside the simulator the belief defines, with the boundary between what it may see and what only its critic may see enforced in code. Section 2.2 described two families of learner model with complementary failures — an explicit, population-trained state with no access to content, and content with no state. This chapter is the proposal that they are one machine. The next three chapters measure whether the machine works.

6 Results I: Meaning in Place of a Curriculum

Chapter 5 removed two things every deep knowledge-tracing model relies on: the table of per-item parameters, and the human-authored curriculum that groups items into skills. In their place it put a frozen encoder and a shared function of item meaning. This chapter asks the one question that decides whether that substitution was affordable: **can meaning do the job the curriculum did?**

The question has two parts, and each is answered by a measurement that does not involve the simulator at all. Does the geometry organise problems the way a curriculum does? Does it carry the pedagogical properties that decide which item is worth asking? The chapter closes with the reciprocal test — what the curriculum is worth to a model that has one — because that is what puts the size of the substitution in context.

The instrument throughout is a *probe*: a small classifier trained to recover a natural label from the frozen representation, where the representation was never shown that label. A probe that succeeds shows the information is present and linearly or nearly-linearly accessible; a probe that fails shows it is not present in a form any downstream head could use. The encoder was frozen before any model in this thesis was trained, so none of these measurements can have influenced it.

6.1 The geometry organises problems the way a curriculum does

Table 6 reports four probes, each against the comparator that matters for it.

What is probed	AUC	Comparator	What it licenses
Curriculum skill: same skill?	**0.819**	raw hidden states 0.749	the no-curriculum commitment
Task identity: which problem does this code solve?	**0.995** [0.989, 0.999]	edit distance 0.917	the positive control
Person: who wrote this code?	0.874 [0.838, 0.907]	**edit distance 0.868**	style is present but ties a surface baseline
Behaviour: do two submissions pass the same tests?	0.523 [0.469, 0.573]	gate 0.70	behaviour must be supervised

Table 6: *What the frozen representation carries. Each row trains a classifier to recover a natural label the encoder never saw; chance is 0.5. Intervals are learner-clustered bootstraps where the row has a learner unit. The comparator column gives the baseline that row has to beat to mean anything: a curriculum probe must beat the untreated hidden states, an identity probe must beat surface string similarity.*

The first row is the one the architecture rests on. Given two problems and no labels of any kind, a probe reading only their encoded meaning recovers whether the curriculum assigns them to the same skill at 0.819 AUC. The curriculum's own grouping is therefore substantially present in the text, recoverable without ever being supplied. Stripping the dominant directions from the hidden states, which is the step the encoder performs, is worth 0.07 of that AUC: the untreated states reach 0.749. This is the concrete form of the substitution — what a human labelled, the geometry already knew — and it is why no skill tag appears anywhere in ATOM.

The second row is a positive control, and it is here because the fourth row is a failure. Asked which of fifty programming problems a piece of code solves, a probe on the same frozen representation reaches 0.995, well clear of a string-similarity baseline at 0.917. The instrument works. So when the fourth row reports that the same representation cannot tell whether two submissions pass the same test cases — 0.523, against a pre-registered floor of 0.70 — that is a fact about the geometry and not about the probe. The representation encodes what a program is *about* and not what it *does*, which is unsurprising for a semantic encoder and decisive for the architecture: behavioural prediction cannot be read off the geometry and must be carried by a supervised head. Chapter 5's test-vector head exists because of this row. A failed probe that changes a design is the most useful kind.

The third row is reported with its limitation, which is the reason it is not a headline. Personal style is present — a probe identifies the author of a code submission at 0.874 — but a plain edit-distance baseline reaches 0.868, and the intervals overlap. The geometry carries authorship, and it does not carry it better than string similarity does. The claim that survives is that style is present in the representation, not that the representation is the best available detector of it.

6.2 The geometry carries what makes an item worth asking

Recovering a curriculum's grouping shows the representation knows what a problem is about. It does not show that it knows anything about the problem *as an item* — how hard it is, and how sharply it separates learners who know the material from those who do not. Discrimination is the property that decides which question is worth asking, and it is what a planner needs to be able to read from text.

Ground truth here is psychometric rather than editorial. A two-parameter item-response model was fitted on the training learners' 1.7 million responses, giving each problem an empirical difficulty and an empirical discrimination; split-half reliabilities are 0.976 and 0.936, so the targets are stable. A probe was fitted on 2,019 problems and evaluated on 356 it had never seen. The comparators are the four frontier language models of Section 4.2, on the dataset's own learner-free task: shown two problems, say which is harder, and say which is more discriminating.

Pairwise accuracy, 356 held-out problems	codec probe	best frontier model	worst frontier model
Which problem is harder?	**72.4%**	68.6%	—
Which problem is more discriminating?	**65.4%**	46.9%	31.1%

Table 7: *Judging an item's psychometric properties from its text alone. Ground truth is a two-parameter item-response model fitted on 1.7 million training-learner responses. The probe is supervised on other problems' response data and the frontier models are zero-shot, so this is not a like-for-like contest; what it establishes is that the property is recoverable from text, and where in the text it is.*

On difficulty the probe is ahead but the gap is modest, which is consistent with the price stated in Section 5.2: most of what makes a problem hard is not in its wording. On discrimination the

gap is not modest, and the direction of the frontier models' error is the finding. **Every frontier model tested is below chance.** A 120-billion-parameter model asked which of two problems better separates strong learners from weak ones does worse than a coin, consistently, in a way that a systematic misreading rather than noise would produce. The probe, reading 64 numbers, is at 65.4%. Its accuracy also rises monotonically with how far apart the two problems really are — 52.5%, 58.9% and 71.6% across terciles of the true gap — which is what a genuine signal looks like and what a lucky classifier does not.

This is the strongest evidence in the thesis that meaning can stand in for a curriculum. The information a curriculum designer uses to sequence a course is in the problem's text; it is not in a form a language model exposes when asked directly; and it is in a form a small probe on the frozen geometry can read.

What the curriculum was worth. The reciprocal question decides how much of a concession the no-curriculum commitment is. Attentive knowledge tracing was trained on the same data twice, once with all 462 skill tags and once with every tag stripped. Removing them costs 0.0060 of AUC, from 0.6982 to 0.6922 — which leaves it essentially level with ATOM's shipped filter at 0.6913 on the same court.

Two readings follow and both are worth stating. The narrow one is that ATOM gives up almost nothing by refusing the curriculum, because on this corpus the curriculum is worth six thousandths of AUC to a model that has one. The broader one is that a human-authored skill taxonomy, at least at this granularity, is largely redundant with what a deep model can extract from the response data itself — and, from Table 6, largely recoverable from the problem's text in any case. The commitment of Chapter 5 is therefore cheap where a curriculum exists, and it is the whole difference where one does not.

What the rest of the thesis inherits. Three things carry forward. Item meaning is a usable substitute for item identity: it recovers the curriculum's own grouping and it carries the psychometric property a planner needs. Item meaning is not a substitute for behavioural supervision: whether a program passes its tests is not in the geometry, and the architecture's supervised heads exist for that reason. And the substitution's price is real and is one-sided: about two-thirds of a problem's empirical difficulty is not in its text, so a meaning-based model gives up a little on the problems everyone has seen.

This chapter supports the second contribution of Section 1.4. The next chapter puts the representation to work: it is the item side of a model that has to say what a particular learner will do.

7 Results II: Simulating a Learner

Chapter 2 left two families of learner model with complementary failures: one that learns a state from a whole population but cannot read a question, and one that reads everything but keeps no state. Chapter 3 claimed they are not rivals but two stages of one computation — locate the mind, then simulate it. This chapter measures both stages, on both corpora, with the same architecture.

It is organised by stage rather than by corpus, because the stages are the claim. Section 7.1 measures locating: given a history and a question, will this learner succeed? That is the task knowledge tracing was built for, and it is where the specialists are strong and the language models are weak. Section 7.2 measures simulating: what will they actually write? That is the task no

knowledge-tracing model can attempt at all. Section 7.3 asks the internal question the outside comparisons cannot answer — how much of what the model does is about *this* learner rather than about learners in general — and finds two different answers on the two corpora, for a reason that is in the data rather than in the model.

Every number is computed on validation learners the model never trained on: 500 on mathematics, 110 on programming. The lockbox remains sealed. Intervals are learner-clustered bootstraps.

7.1 Stage one: locating the learner

7.1.1 FoundationalASSIST: Mathematics

Table 8 puts the deployable system against the corpus's own published benchmark and against two specialists trained here.

Model	Acc. (%)	Lift over own baseline	AUC	When correct	When wrong
AKT (trained; uses skill tags)	**75.3**	**+13.4**	**0.810**	**85.3**	**59.2**
ATOM (4B)	73.7	+11.8	0.786	83.3	57.9
GPT-OSS-120B	56.2	+4.9	0.559	68.2	43.4
Qwen3-80B-Think	55.8	+4.5	0.555	63.6	44.6
Qwen3-80B-Instruct	54.0	+2.7	0.539	56.5	51.0
Llama-3.3-70B	52.3	+1.0	0.512	85.4	12.6

Table 8: *Predicting whether a learner will get the next problem right. Accuracy is quoted against each model's own majority baseline, because the courts differ: frontier models are scored on the dataset's rebalanced set at 51.3% correct, the trained models on the corpus's natural rate. The last two columns give accuracy on the events where the learner was in fact correct and where they were in fact wrong. Frontier numbers are the dataset's own published benchmark (Worden et al., 2026).*

A 256-number belief, computed before the question is seen, reaches 0.786 AUC and 73.7% accuracy. Four frontier models of 70 to 120 billion parameters, given the same learner's history as text and asked the same question, reach 0.512 to 0.559 — between one and five points of accuracy over their own majority baseline, against ATOM's 11.8. **Compression beats context.** A history rendered as prose is not a state: the model must re-derive what a learner is like from a transcript, at inference time, with no population to compare them against, and it does not manage it.

The one model ahead of it is attentive knowledge tracing, at 0.810 AUC against 0.786. Two things are worth saying about that comparison, and they point in different directions. AKT was trained here, on this corpus, from scratch — to the author's knowledge the first AKT baseline reported on FoundationalASSIST, whose release paper benchmarks frontier language models rather than knowledge-tracing architectures. Paired on the same 4,914 rows its lead is 0.0237 AUC [0.0149, 0.0324], and it is given the skill tags ATOM is denied, which Chapter 6 prices at 0.0060 AUC.

That comparison is not a choice that was open and lost. ATOM's filter could have been built on an attentive body; it was built on a recurrent one because what the rest of the system consumes is not a prediction but a belief — a mean and a width over the person — and the first question was whether the two-stage architecture works at all. The body is a component rather than the claim: an attentive variant was built and compared under identical heads and loss, and Appendix A reports why the recurrent one ships. What is established here is that a model reading no curriculum, indexing no item, and generating the learner's answer besides comes within 0.024 AUC of a specialist handed all three.

The split by outcome is where the frontier models break rather than merely underperform. Llama-3.3-70B is 85.4% accurate on the events the learner got right and 12.6% on the events they got wrong: it is close to predicting "correct" regardless, which is a strategy and not a model. ATOM is 83.3% and 57.9% — weaker on the easy half and more than four times better on the half that matters, since a tutoring system's entire value is in anticipating failure. Attentive knowledge tracing reaches 59.2% on that half.

Two further measurements say the signal is about the person rather than about the problem. Restricting to comparisons *within a single problem*, where difficulty is held exactly fixed and a learner-blind model scores 0.500 by construction, ATOM reaches 0.753 [0.732, 0.777] over 741 problems. And the belief's own width tracks its own error: the correlation between the width of the belief and the size of the prediction error is 0.484, so the model is not only accurate but knows where it is not.

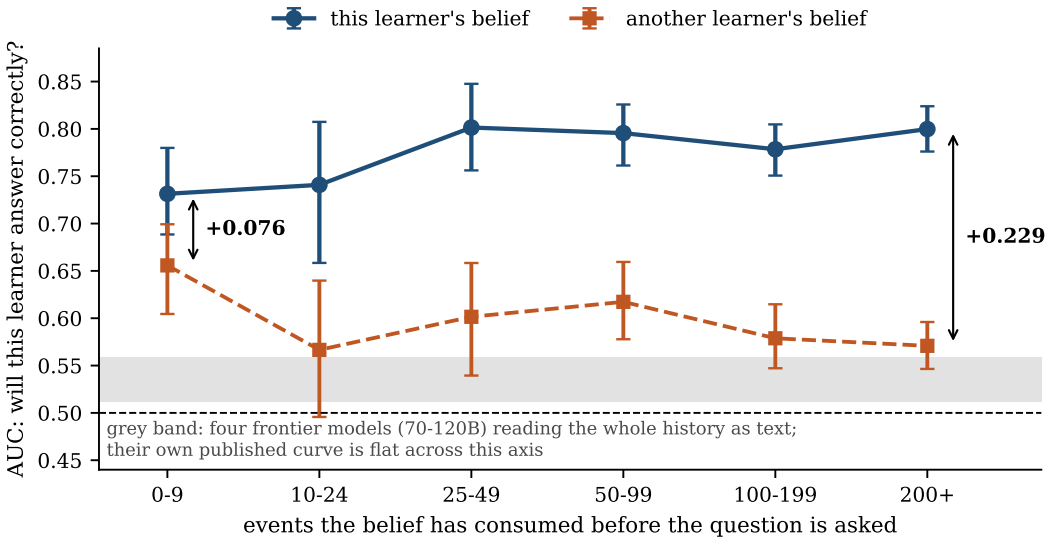


Figure 5: *How the value of knowing* this *learner grows with evidence. Each curve is AUC on the same events, once with the learner's own belief and once with the belief of a different learner matched for history length. What to read: the vertical gap between the curves, which is what identity is worth. The upper curve plateaus by about twenty-five events while the lower curve does not climb at all, so the gap is opened by the belief and not by the passage of time. Bands are learner-clustered 95% intervals; history buckets are cross-sectional, so the trend rather than the level is the readable quantity.*

Figure 5 separates the two things a long history could be doing. With fewer than ten events the learner's own belief is worth 0.076 of AUC over a length-matched stranger's; beyond two hundred it is worth 0.229. Identity accumulates, and it accumulates in the belief: the stranger's curve, built from just as much history, stays flat. The upper curve's plateau at around twenty-five events is the practical reading — most of what a belief will ever know about a learner is available within a few dozen interactions.

7.1.2 CodeWorkout: Programming

On programming the same stage is measured against four systems built for this corpus, on their metric: predicting whether a learner's *first* submission to a problem passes, over all fifty problems, 4,593 first submissions with a 35.4% pass rate.

Model	AUC
ATOM	**0.833**
KCGen-KT, generated knowledge components	0.816
KCGen-KT, human-written knowledge components	0.797
TIKTOC, adapted to overall correctness	0.788
Code-DKT	0.766
Majority class	0.500

Table 9: *First-submission correctness on CodeWorkout. Published rows are taken from KCGen-KT's own comparison table (Duan et al., 2026), computed on 246 students and 10,834 submissions with an instruction-tuned language model and a code parser in the pipeline.* The comparison is not row-for-row. *KCGen-KT is evaluated on random splits of its population, with all fifty problems represented in training, under the five-split protocol; ATOM is evaluated on learners excluded entirely from training, and its aggregate additionally includes seven problems withheld from all training. No spread is quoted on either side: the published figures are point values in the source table, and ATOM was trained once.*

ATOM leads at 0.833. Two facts about where that number comes from matter more than the ranking. Item difficulty and learner ability alone — two scalars, no learned representation, fitted under the same causal rule on ATOM's own rows — reach 0.795, so ATOM's margin over classical psychometrics is 0.038 rather than the 0.067 the table implies against the best deep comparator. And a recurrent network given only problem identifiers, with no text at all, reaches 0.772: *below* the two scalars. Most of what is predictable here is item difficulty and learner ability, which is a fact about the task and not a criticism of any model on the list; what a learned representation adds is real and is a third of what the ranking suggests. Within a single problem, difficulty held fixed, ATOM reaches 0.796 across the fifty problems, so the personal signal survives the control. On the seven problems held out of training, ability alone reaches 0.805 and a recurrent network trained without them 0.783; no published model holds problems out, so there is no external comparison for that stratum.

Figure 6 makes the cold start explicit. On a learner's very first problem the model scores 0.489 [0.331, 0.655] — chance, with an interval that contains it comfortably. After a handful of problems it reaches 0.746, and by the sixteenth 0.855, after which it is flat. This is a limitation reported as one, and it is also the argument for the next chapter: a system that is uninformative until it has watched sixteen problems has an obvious incentive to choose which problems it watches.

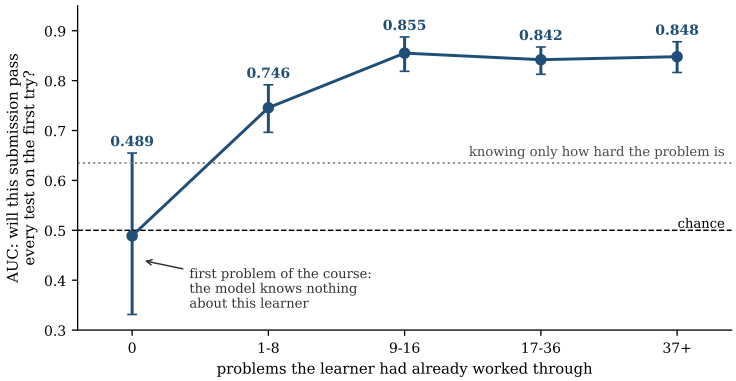


Figure 6: *What a belief is worth as it matures. First-submission AUC against the number of problems the learner has already completed. What to read: the left-hand end, where the model is at chance. The 110 points at zero are each learner's very first problem, where the filter returns the population prior and the model has nothing personal to say. Bands are learner-clustered 95% intervals.*

7.2 Stage two: writing what the learner would write

Knowledge tracing stops at stage one. It never reads the problem and it produces no text, so the whole of this section is outside its reach; the comparators here are language models, which is the other half of Chapter 2's division.

7.2.1 FoundationalASSIST: The exact answer

Model	Outcome class from	Overall	Student correct	Student incorrect	Answer incorrect
ATOM (4B)	**its own filter**	**46.0**	65.1	**21.4**	**13.2**
ATOM (4B)	the true label	57.3	78.1	30.5	34.0
Llama-3.3-70B	—	44.3	72.6	14.5	3.7
GPT-OSS-120B	—	41.2	62.9	18.2	10.1
Qwen3-80B-Think	—	38.5	57.6	18.3	12.9
Qwen3-80B-Instruct	—	35.5	50.3	19.8	13.5
AKT, item response theory	—	*cannot produce an answer*			

Table 10: *Reproducing the learner's exact typed answer, per cent. The first ATOM row is the deployable system: the outcome class is predicted by its own filter and nothing from the future enters the prompt. The second supplies the true class and scores only the text. "Student correct" and "student incorrect" split by what the learner did; "answer incorrect" is the stratum where the model itself produced a wrong answer. The two ATOM rows are computed on 300 events; the frontier rows on approximately 27,000 (Worden et al., 2026). Frontier models are scored with a 1% numerical tolerance on fill-in items; ATOM is scored on exact tokens throughout, which is conservative against ATOM.*

Given the problem's text and nothing else about the learner but a weight edit, ATOM reproduces the exact string a learner typed 46.0% of the time, against 44.3% for the best of the four frontier models, each of which is given the learner's history in the prompt. The aggregate hides the pattern that matters, and no single comparator survives it: a different frontier model is strongest in each column, so each is quoted against its own best rival below. Where the learner was *correct*, ATOM is behind — 65.1% against Llama-3.3-70B's 72.6%. Where the learner was *wrong*, which is the

stratum a tutoring system exists for, ATOM leads: 21.4% against 19.8% for the best of the four there, Qwen3-80B-Instruct, and against 14.5% for Llama. And on the narrower stratum where the generated answer is itself wrong, ATOM is behind again, at 13.2% against Qwen3-80B-Instruct's 13.5%. The profiles differ in kind and not only in level: Llama-3.3-70B is the strongest of the four where the answer is the correct answer and the weakest by a wide margin where it is not, at 3.7%, which is a model solving the problem rather than simulating a learner.

The second row is the measured value of locating the mind. Supplying the true outcome class and changing nothing else raises exact reproduction from 46.0% to 57.3% overall, and from 13.2% to 34.0% on wrong answers. That is also where the deficit above comes from. On the answer-incorrect stratum ATOM trails the best of the four by three tenths of a point while its own filter is proposing the wrong regime to write in; told the right one, the same generator reaches 34.0%, two and a half times Qwen3-80B-Instruct's 13.5%. What limits ATOM on that stratum is the head that conditions the language model, not the language model. The mechanism is visible in the filter's own numbers: sampling its predicted class, the model agrees with the truth 53.7% of the time overall and 22.6% on the wrong-answer stratum, so it proposes the wrong-answer regime for only a fraction of the events where it belongs.

This is the cleanest evidence in the thesis for the two-stage claim, and what it shows deserves to be stated precisely. The bottleneck in simulating a learner is not the language model's ability to write like one; it is knowing which learner to be. Locate the mind correctly and the generator delivers. That is the division of labour Chapter 5 proposed, measured as a gap of 11.3 points overall and 20.8 on wrong answers, and it is the strongest argument the thesis has for joining the two families rather than choosing between them.

The same bottleneck is visible from the other side. On the unaided stratum the joint simulator's outcome head reads 0.669 AUC against an item-difficulty table's 0.703 on the same rows, while the identical filter trained *without* the language model attached reads 0.049 above its own floor. Attaching a token loss to the belief costs something on the classification side. That trade is disclosed here rather than in an appendix because it is the same fact as the oracle row, seen from the other direction: the two stages compete for one latent, and the shipped configuration favours the generator. A head-only factored variant, predicting help-seeking and then correctness given it, recovers 0.036 on detecting wrong answers in the bottom ability tercile; it is not what ships, for the reason given in Appendix A.

7.2.2 CodeWorkout: The whole program

On CodeWorkout the compared quantity is a whole Java method rather than a two-token string, and the comparators are systems built for this task and scored by CodeBLEU. Before any number, one thing has to be said about how those numbers are produced, because the literature measures this task in two different ways and the two are not interchangeable.

The first way is the one the field quotes: all fifty problems, first submissions, with the model fitted and scored on random splits of the same population of 246 students — five splits, every problem represented in training. The second is TIKTOC's own benchmark: the seventeen problems whose test suites were recovered, the same 246 students, 3,714 first submissions, scored row for row. ATOM is evaluated under a discipline stricter than either. Its learners are excluded from training entirely, and seven of the fifty problems are withheld from all training and still counted inside its aggregate. Table 11 therefore reports three separate measurements rather than one ranking, and the rest of this section reads them in the order of how much they can be trusted.

Protocol	Model	CodeBLEU	Rows
All 50 problems first submissions	KCGen-KT, generated components	0.580 ± 0.018	10,834
	KCGen-KT, human-written components	0.557 ± 0.028	10,834
	ATOM	**0.526**	4,587
	TIKTOC, adapted	0.507 ± 0.015	10,834
TIKTOC's benchmark 17 problems, 246 students	**ATOM** (zero-shot, cross-cohort)	**0.561**	3,709
	TIKTOC	0.554	3,714
	OKT-TC	0.522	3,714
7 held-out problems	ATOM	0.479	591

Table 11: *Writing the program a learner would submit. The three blocks are three different measurements and not three parts of one ranking; what separates them is set out in the text. "Rows" is the number of first submissions each figure is computed over.*

The contest that is row for row. On TIKTOC's own benchmark the two sides are scored on the same submissions: the same seventeen problems, the same 246 students. ATOM reaches 0.561 against TIKTOC's 0.554 and OKT-TC's 0.522, and it is the only arm there that is zero-shot. The comparators were fitted on this cohort; ATOM was fitted on a different semester and reads these learners for the first time at evaluation. This is the one place in the programming results where nothing has to be assumed about comparability, and it is the strongest form the generative claim takes anywhere in this thesis: a model that has never met a student writes that student's code as well as a model trained on them.

The aggregate protocol, and why its two figures are not a head-to-head. Across all fifty problems ATOM reaches 0.526, where the two KCGen-KT variants report 0.580 and 0.557 and the TIKTOC adaptation 0.507. ATOM's figure is the lower of the two leading ones, and it should not be read as a result in either direction, because the two are not measurements of the same thing. They are computed over different populations under different rules: KCGen-KT scores random splits of its own 246 students with every problem represented in training, while ATOM scores 110 learners held out of training entirely and carries inside its aggregate seven problems that no training run ever saw — the stratum that costs it most, as the next paragraph shows. They carry different kinds of uncertainty: the published figures are the mean over five retrainings and their spread is the spread of that mean, whereas ATOM was trained once, so no comparable quantity exists, and a learner-clustered bootstrap over its rows would report sampling variance rather than variance across training runs. And they are different systems on different budgets: KCGen-KT's pipeline is an eight-billion-parameter instruction-tuned Llama-3 with a code parser, trained on the population it is scored against, against a frozen backbone with about four billion parameters active per token and 74 million trainable. What would settle the question is ATOM run under KCGen-KT's protocol, or KCGen-KT run under ATOM's; neither exists, and until one does the two figures are reported side by side and left there. One thing is worth recording about ATOM's: before the run it was pre-registered to land below the thirteen-problem figure and plausibly near the pooled value or lower, and it did.

The problems nothing was trained on. Withholding a problem from every training run costs 0.127 of CodeBLEU: from 0.606 on the thirteen problems whose test-case joins could be verified to 0.479 on the seven never-trained ones. That is the largest single drop anywhere in the programming results, and it is the same coverage price Chapter 6 named, paid in generation rather than in

classification. It is also why the aggregate above is not a like-for-like number: that stratum sits inside ATOM's 0.526 and inside nobody else's. No published model on this corpus holds problems out, so the figure stands without a comparator rather than favourably.

Where the generative performance comes from. Three measurements locate it. Turning the adapter off entirely drops CodeBLEU from 0.564 to 0.407 on the pooled validation rows, so the personalised weight edit is worth about 0.157. Replacing the learner's belief with another learner's costs almost nothing — 0.564 against 0.563 on the same rows — so what the adapter carries is the register of a novice Java programmer on this course rather than the identity of a particular one, which is a real limit on the twin claim and is taken up in Section 7.3. And the decoding mode is immaterial: argmax, conditional and ancestral sampling land within 0.004 of one another, so nothing here is an artifact of how the text was drawn.

The finer-grained metric reverses the ordering. Predicting which individual test cases a submission will pass, on the same 246 learners, ATOM reaches 0.675 against TIKTOC's 0.764 and Code-DKT-TC's 0.661. The ordering is the reverse of the code-writing result and the reason is the one that runs through this whole section: TIKTOC and Code-DKT-TC are trained inside this population, at a target finer than anything ATOM's classification head predicts, while ATOM writes code for these learners without having trained on one of them. A model optimised for a target beats one that was not, on that target. Appendix A reports that refreshing the test-vector head alone on the frozen belief raises all-attempt AUC from 0.744 to 0.802 and first-attempt AUC from 0.584 to 0.677, which locates the shortfall in the head rather than in the belief.

7.3 How much of this is *this* learner?

Everything above compares ATOM with other systems. The remaining question is internal, and no external baseline can answer it: of everything the model does, how much is about *this* learner rather than about learners in general? The four-arm decomposition of Section 4.5 answers it directly, by evaluating one model on one set of rows under four conditions that differ only in what belief it is given.

What the model is given	NLL	Cost	95% interval
this learner's belief	**0.4492**	—	—
another learner's belief	0.4660	−0.0168	[−0.0205, −0.0133]
this learner's belief, collapsed to a point	0.4512	−0.0020	[−0.0037, −0.0003]
no belief at all (adapter off)	3.3007	−2.8515	[−2.8982, −2.8071]

Table 12: *The four arms, mathematics: 4,977 events over 500 held-out learners, paired, learner-clustered. Negative log-likelihood of the learner's true answer, in nats; lower is better. Each row differs from the first only in which belief the hypernetwork is given.*

The three costs differ by orders of magnitude, and the summary is worth memorising: **2.85 nats for having a belief, 0.017 for whose, 0.002 for the region.** Almost all of the value is in being adapted to *someone*: the shared basis, driven by any plausible learner, turns a general-purpose language model into a model of a middle-school mathematics learner, and that is worth 2.85 nats. Knowing *which* learner is worth 0.017 nats — small, and cleanly separated from zero. Carrying a region rather than a point is worth 0.002, positive and barely clear of zero on the production

channel, which is the fair reading of what the width buys *here*; its value in this thesis is to the planner of Chapter 8, which consumes it directly, and not to the token likelihood.

Figure 7 shows the identity term is present from the beginning and grows: +0.0105 [+0.0022, +0.0184] with fewer than ten prior events, and +0.0196 [+0.0139, +0.0254] beyond two hundred. Adaptation is flat across the same axis, which is what one would expect if it is carrying the population and not the person.

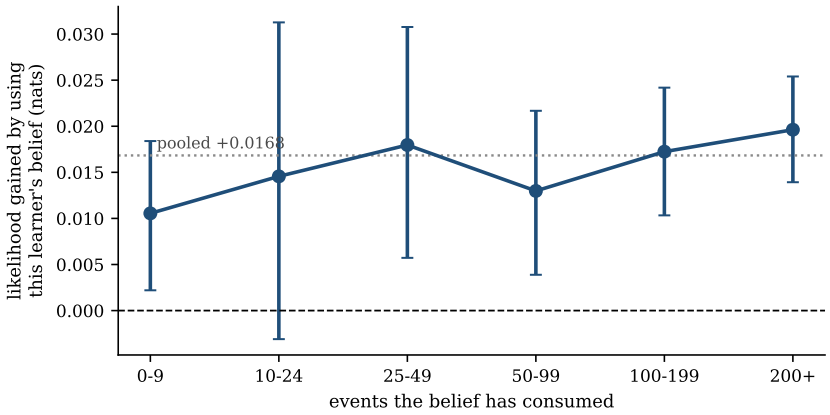


Figure 7: *The identity effect as the belief matures. What to read: the identity curve, which is the cost of swapping in a stranger's belief, against how much history the learner's own belief has consumed. It clears zero from the first handful of events and roughly doubles thereafter, while the adaptation effect stays flat at about 2.85 nats. Bars are learner-clustered 95% intervals; buckets are cross-sectional, so the trend rather than the level is the readable quantity.*

Figure 8 explains why the identity term is so small, and the explanation is about mathematics rather than about the model. Measured in probability space — the likelihood the model assigns to the true answer — every readout detects the individual. Measured on the tokens the model actually emits, almost none of them does. The one discrete instrument that does detect identity is exact reproduction restricted to *open* responses, and it gains nothing on either kind of multiple-choice item. The mechanism follows immediately: a mathematics answer is a short string drawn from a narrow set of plausible values, so a belief that shifts the probability of "17160" against "17,160" by a few per cent changes the likelihood and does not change the argmax. **The identity signal is present in the distribution and quantised away at the moment of emission.**

That is a fact about the production channel of this corpus, and it is the reason the same architecture was run where a production is a whole program.

On programming the identity signal is visible, and it is visible differently under the two instruments. Against the average of random classmates' real submissions, ATOM leads by 0.078 [0.067, 0.089] in character-level similarity, clearly separated from zero. Against the *medoid* — the one real submission per problem that is closest on average to all the others, selected with hindsight, and therefore the best score any learner-blind strategy could possibly achieve — ATOM falls short by 0.015 [0.006, 0.025]. A system that beats a random classmate and loses to the optimal generic answer is carrying personal signal without carrying enough of it to overcome the fact that novice solutions to a small Java exercise are mostly alike.

Figure 9 shows the two instruments diverging with maturity, and the divergence is informative rather than contradictory. Controlling for composition and measuring in the frozen space of meanings, ATOM's lead over the random-classmate average grows from 0.044 at a cold belief to 0.146 past thirty-seven episodes, both clear of zero. The classmate average itself gets *worse* with maturity,

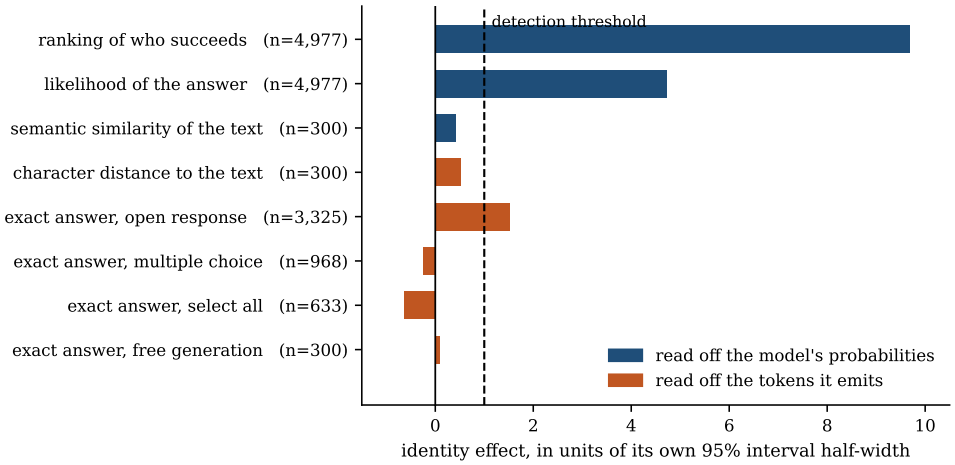


Figure 8: *The same identity effect, measured eight ways. Each bar is one readout, expressed in units of its own interval half-width so that instruments of different scales can be compared; a bar longer than one is an effect its own interval clears. What to read: the split between the probability-space readouts on one side and the token-space readouts on the other, and the single exception among the discrete instruments. The two readouts computed on 300 events are the weakest evidence in the panel.*

because later problems are harder and a generic answer serves them less well, while ATOM's semantic closeness holds flat — the model keeps up with the difficulty and the generic answer does not. In raw characters the story is a cold-start advantage that a mature belief loses: ATOM produces a near-exact reproduction 7.0% of the time against the medoid's 2.9% at one to eight episodes, and 7.4% against 3.4% at nine to sixteen, both clear; the ordering reverses at seventeen to thirty-six and reaches 21.3% for the medoid against 8.2% for ATOM past thirty-seven. The reversal does not move when the similarity threshold is changed, so it is not an artifact of where the line was drawn. The two instruments agree that a personal signal exists and disagree about which comparison reveals it: semantic closeness is where personalisation shows, and character-exactness past the mid-course is where the convergence of real learners' solutions to easy problems shows.

7.4 Summary

- **Locating works, and compression beats context.** A 256-number belief computed before the question is seen reaches 0.786 AUC on mathematics against 0.512–0.559 for four frontier models reading the same history as text, and 0.833 on programming against 0.816 for the best published system.
- **Simulating works, and it is bottlenecked by locating.** ATOM reproduces a learner's exact typed answer 46.0% of the time against 44.3% for the best frontier model, and 21.4% against 19.8% on the events where the learner was wrong. Supplying the true outcome class raises those to 57.3% and 30.5%: the generator is not the limit, the belief is.
- **On code, the zero-shot arm wins its own benchmark and loses the field's.** 0.561 CodeBLEU against TIKTOC's 0.554 on identical rows without training on those learners; 0.526 against KCGen-KT's 0.580 across all fifty problems, which is reported as a loss.
- **Almost all the value of personalisation is being adapted to somebody; a small, real part is being adapted to this person.** 2.85 nats, 0.017 nats and 0.002 nats for having a belief, whose it is, and how wide it is.

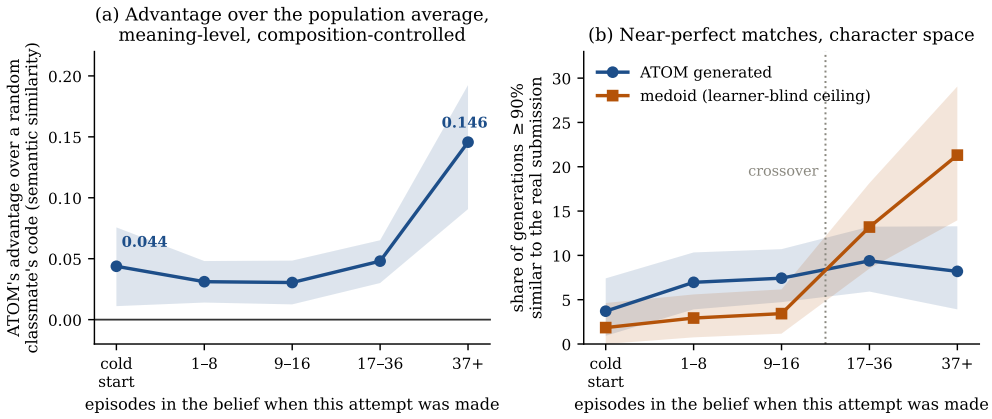


Figure 9: *Personal identity in generated code, against two learner-blind ceilings. Panel (a) is similarity in the frozen space of meanings; panel (b) is the share of generations that reproduce the real submission almost character for character. What to read: the direction of each panel with maturity — (a) widens in ATOM's favour, (b) crosses over. The medoid is the single real submission per problem closest on average to all the others, chosen with hindsight, and is the best any learner-blind strategy could do. Bands are learner-clustered 95% intervals; bins were chosen after seeing the distribution and the learner count falls from 108 to 64 across them, so the trend rather than the level is the claim.*

- **Identity survives where productions are rich and is quantised away where they are short.** It is detectable in probability space on mathematics and almost never in emitted tokens; on code it is worth 0.078 over a random classmate and grows with maturity in semantic space.
- **The cold start is real and is measured.** On a learner's first programming problem the model is at chance, and it reaches its ceiling only by their sixteenth.

This chapter supports the third, fourth and fifth contributions of Section 1.4. It also states the case for the next one: a model that knows nothing about a learner until it has watched them for sixteen problems, and whose bottleneck is locating rather than generating, has every reason to choose what it watches.

8 Results III: Acting on the Belief

Chapter 7 ended on a finding and an opening. The finding is that what limits the simulator is locating the learner rather than writing text for them: give it the right belief and the generator delivers. The opening is that locating takes time — on CodeWorkout the model is at chance on a learner's very first problem and reaches its ceiling only around their sixteenth — so a system that has to wait that long has a reason to choose what it waits for. This chapter measures what choosing buys. What makes choosing possible at all is that the belief carries its own width: it records not only where the learner probably is, but how sure the system is, and that is the quantity against which a candidate question is priced.

Everything below is measured by the replay protocol of Section 4.5, and one piece of its vocabulary is used on every page. A policy is given the first few events of a held-out learner's real history — the *start* — and forms a belief from them. It is then shown a *menu* of questions that learner answered later, chooses one, and receives the answer they really gave; the belief is recomputed with it, and the process repeats until the *budget* of questions is spent. The policy is scored at the end on how

well that belief predicts a fixed set of the learner's interactions lying after everything on the menu. A **scenario** is one combination of start and budget: "start 3, budget 8" means three real events revealed and eight questions to spend.

Results are reported over a grid of scenarios rather than at a single operating point, because the value of asking depends on both halves of it. A policy that helps where the system knows nothing about a learner may add little where it has already watched them for ten problems, and eight questions are not eight times one. Reporting the grid is what makes it possible to say *where* a policy helps rather than only that it helps somewhere. The grids are not all the same size, and the reason is cost. A policy that needs no training can be swept over many scenarios cheaply, and the untrained arms were; every trained arm has to be trained, and then have its questions generated and scored, separately in each scenario, which is the expensive part of this programme, so those arms were run on a smaller grid. Scenarios starting from an empty belief are excluded from the trained arms in any case, by the rule of Section 5.7.2 that no policy is trained or judged from a cold start. Because the grids differ, proportions rather than raw counts are the quantity to compare across arms.

The chapter opens with a question that has to be answered before any policy is run, because it decides what a planner could be worth on each corpus at all. It then measures a trained actor against the decision-theoretic incumbent, a policy trained without a single real interaction, and a language model taught to select by the simulator itself. Every comparison is paired — the same learners, the same menus, the same random draws — and every result is quoted relative to a random policy asked the same questions, because the improvement a random policy already makes over asking nothing is the headroom any policy is competing for.

8.1 Where a planner can matter at all

A planner buys information about the person. If the outcome variance belongs to the item rather than to the person, there is nothing for it to buy, and no policy — however good — can beat a random one by much. This is measurable before any policy exists, and it was measured first.

Corpus	Learners	Items	% person	% item	person : item
CodeWorkout, first submissions	864	50	**23.6**	6.2	**3.82**
FoundationalASSIST, first exposures	5,000	3,359	12.3	18.1	0.68
FoundationalASSIST, give-ups removed	4,999	3,335	**4.6**	18.7	**0.25**

Table 13: *How outcome variance divides between the person and the item, difficulty removed first, both corpora on one code path. The last column is the ratio, and it swings by a factor of fifteen between the top row and the bottom.*

Table 13 is the most consequential measurement in this chapter and it involves no planner. On CodeWorkout the person owns about four times more outcome variance than the item. On FoundationalASSIST, once events where the learner revealed the answer are removed, the item owns about four times more than the person. The ratio swings fifteen-fold between them.

The same swing appears in the planner's own quantity. Table 14 asks how well a person-estimate built from k observations predicts that person's remaining items — which is exactly what a budget of k asks can buy.

At a budget of three, three observations are worth 0.660 on CodeWorkout and 0.334 on the clean mathematics slice: roughly half the information for the same number of questions.

The comparison between the second and third rows of both tables is worth stating on its own,

Corpus	$k = 1$	$k = 3$	$k = 5$	$k = 10$
CodeWorkout	0.466	**0.660**	0.746	0.825
FoundationalASSIST, all events	0.365	0.561	0.658	0.776
FoundationalASSIST, give-ups removed	0.203	**0.334**	0.411	0.537

Table 14: *Correlation between a person-estimate built from k observations and that person's remaining items. At the operating point used throughout this chapter, $k = 3$, three observations are worth roughly twice as much on programming as on mathematics.*

because it is the sharpest illustration in the thesis of the label problem of Section 4.2. Removing the events where a learner revealed the answer drops the person's share of variance from 12.3% to 4.6% and the value of three observations from 0.561 to 0.334. **Roughly 62% of what looked like person signal, and roughly 40% of the apparent value of asking, was help-seeking temperament rather than knowledge.** A planner scored on the raw label would have been rewarded for identifying who gives up, which is a real and useful thing to know and is not what a knowledge-diagnosing system claims to do.

None of this says the person is invisible on mathematics. Split-half reliability of the person estimate, disattenuated, is 0.912 on the clean mathematics slice against 0.929 on CodeWorkout: the personal signal is measured almost as precisely on one corpus as the other. It is simply small next to the item. This is why every planner result below is stronger on programming, and why the mathematics result is a negative one rather than a failure to measure.

8.2 Reading the belief beats asking at random

The question this section answers is not which policy is best. It is whether the belief the simulator maintains can be turned into better questions than chance would ask — whether, given a simulator of a person, there is a planner to be had at all. The evidence is a dividing line, and it does not fall where a leaderboard would put it.

At the registered headline scenario on CodeWorkout — a learner at the start of the course, a budget of three questions — the policies separate into two kinds. The two that price a candidate by what its answer would reveal about this person reach target-set log-losses of 0.924 and 0.944 nats: the trained actor and greedy value of information. The two that do not consult the belief in that way — maximum entropy and difficulty-matched selection — reach 1.028 and 1.031, barely distinguishable from a random policy's 1.072. What separates the policies is *whether* they read the belief, not how they read it.

The line holds across the grid rather than at one convenient point. Over four starting points and eight budgets, the trained actor's advantage over random is interval-clear in 66 of 128 log-loss scenarios and it never reverses: there is no scenario in which it is worse than random with an interval clear of zero. Re-run on three different underlying beliefs, it stays interval-clear against both random and maximum entropy in about a third of scenarios on the strongest of the three. A policy that reads a belief this simulator produced improves the predictions that belief supports — repeatedly, at different operating points, on learners held out of training.

What this cohort cannot say is which of the two belief-reading policies is better. The trained actor is nominally ahead of the greedy calculation at the headline scenario and never separates from it at interval level anywhere. With roughly 110 held-out learners most policies here are not separable from one another, which is a statement about the power of the study rather than about the policies, and it is why the figure reads every policy against random rather than against its

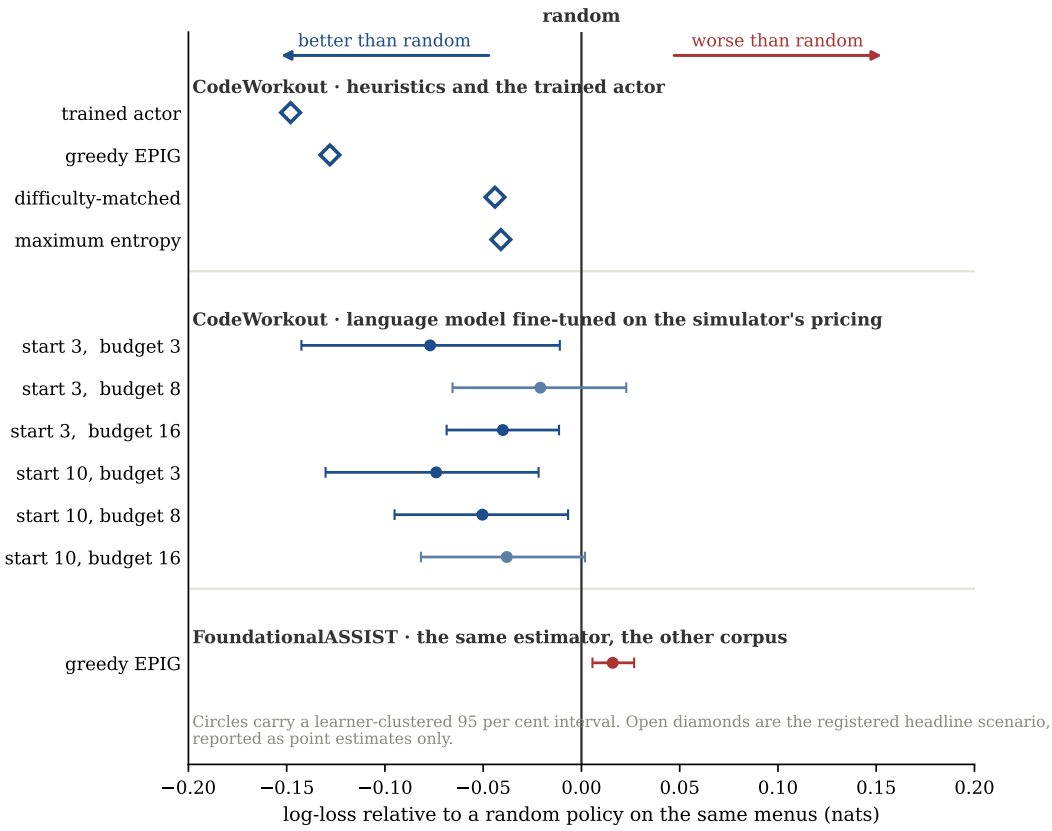


Figure 10: *Every policy against a random one, on the same menus. What to read: distance from the vertical line at zero, which is a random policy on the same learners and the same draws; left is better. The three bands are separate contests and their rows are not comparable across bands. Circles carry a learner-clustered 95 per cent interval; open diamonds are the registered headline scenario, for which point estimates were pre-registered without intervals.*

nearest rival. The claim this section supports is that the room exists, not a ranking inside it.

The negative on the other corpus is what makes that claim credible rather than an artifact of the harness. On FoundationalASSIST the same estimator, run through the same code, is *worse* than random by 0.0159 nats [0.0056, 0.0268] — interval-clear in the wrong direction. That is the mirror image of Section 8.1: where the item owns four times more variance than the person, and three observations are worth half what they are worth on programming, a policy that spends its budget learning about the person is spending it on the smaller quantity. A harness that manufactured planner gains would have manufactured them here too, and it did not. The pre-registered variant that chooses uniformly among the top five candidates reproduces the same loss, which argues against taking a maximum over many noisy estimates being the problem.

8.3 A policy trained without a single real interaction

The strongest possible use of a behavioural twin is to train a policy entirely inside it and then deploy it on real people. This is the claim that would make a simulator worth building even where interaction is expensive, and it is testable here because the simulator and the real learners are both available.

The policy was trained for eight epochs of policy gradient against imagined rollouts, with outcomes

sampled from the frozen simulator's own head, answer content drawn from a corpus the simulator generated, and checkpoint selection on simulated validation. The audit field `real_rows_in_training` is zero. In-simulator validation improved for three epochs, from 0.539 to 0.477, and degraded thereafter; the epoch-3 checkpoint was carried forward, chosen without reference to any real learner.

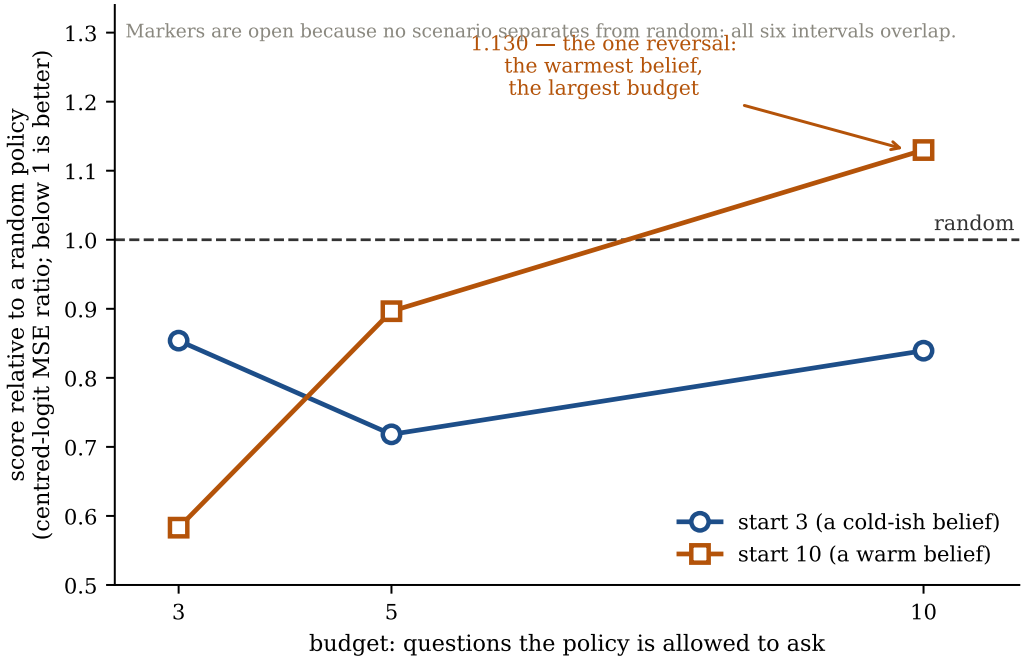


Figure 11: *A policy trained on zero real interactions, examined on real learners. What to read: whether each line sits below the dashed line at 1.0, which is a random policy on the same learners. The x-axis is the number of questions the policy may ask; the two lines differ in how much real history formed the belief it starts from. Markers are open because no scenario separates from random: with 102 to 109 learners per scenario every interval overlaps.*

Figure 11 shows the transfer. Five of the six scenarios favour the simulator-trained policy, with ratios from 0.583 to 0.897 — in the best scenario it achieves the same predictive quality on real learners' real later answers as a random policy achieves, using a little over half the loss. The sixth scenario reverses to 1.130. **No scenario clears the pre-registered bar**, because at 102 to 109 learners per scenario the intervals are wide enough to contain both a substantial effect and none at all.

Two things are worth saying about a result of that shape. The direction is not noise-shaped: five of six in the same direction, with the two largest effects in the two smallest-budget scenarios, is the pattern a real but underpowered effect produces, and it is not the pattern six coin flips produce. And the single reversal falls where a simulator-trained policy has least to add — the warmest belief and the largest budget, where ten real events have already been seen and ten more questions are about to be asked, so the marginal value of choosing well is at its lowest and a random policy is nearly as good as any. The claim this licenses is that training a questioning policy entirely inside a learned simulator is *feasible and transfers in the right direction*, and that this cohort cannot establish that it transfers.

8.4 Teaching a language model to ask

The last family asks whether a language model can be the policy, and it is the only place in this thesis where the simulator is used not as something to be measured but as something that *supplies a training signal*. The section states what the model was asked to do, why it cannot do it unaided, what signal the simulator can hand it, and what changes when it does.

The task, and how a choice is read out. The model is placed in the same replay harness as every other policy. Its prompt contains the learner's revealed history in prose, the menu of candidate questions, and the number of questions remaining; it must name one. Reading that answer back reliably is not trivial, and the way it is done matters for everything that follows. Each menu item is labelled with a *single-token* identifier drawn from a pre-registered alphabet, and single-tokenness is asserted for every item. The policy is then the softmax over those identifier tokens, $\pi(j \mid s)$, which makes three things true at once: the read-out is a proper distribution over the menu rather than a parse of free text, an off-menu answer is structurally impossible, and no candidate is favoured for being shorter than another. A canary asserts that the identifier set appears in the rendered prompt for every state.

Cold, the model cannot do this, and the way it fails is diagnostic. Read out as free text, its answer parses as a menu choice only 42.5% of the time overall and 30% on the longest histories, which alone would sink the comparison. Under the constrained read-out the parsing problem disappears and the real failure becomes visible: **the model reads position, not content**. It picks the first-listed candidate 90.8% of the time with a full history in the prompt and 99.7% of the time with no history at all, and it loses to a random policy in 36 of 36 scenarios. A frontier model handed this task zero-shot is not doing diagnosis badly; it is not doing diagnosis.

The signal the simulator can supply, which nothing else can. What makes this setting unusual is that the value of every possible action is known exactly, and for free. In replay, asking menu item j at state s reveals the answer the learner actually gave; the frozen filter recomputes the belief; and the one-step reward is how much closer that belief comes to the belief formed from the learner's entire history,

$$r(s, j) \;=\; d(b_s,\, b_{\text{full}}) \;-\; d(b_{s\circ j},\, b_{\text{full}}),$$

with d the centred-logit distance registered in Section 5.7. No critic has to be learned, no reward model has to be fitted, and nothing has to be sampled: at any state the simulator can price *every* candidate on the menu by simply performing each update. That is a far richer signal than the one a policy-gradient method usually gets, which is the return of the single action it happened to take. Here the whole distribution over the menu is available as a target.

The fine-tuning uses it directly. Fifteen thousand states are drawn from *uniform-random-policy* trajectories over the fitting split — deliberately not from the trained policy's own trajectories, so that the states the model learns on do not depend on the model being learned. At each state the exact rewards are z-scored across the menu, $A(s, j) = (r(s, j) - \overline{r})/\mathrm{sd}(r)$, and turned into a target distribution $p^\star(j \mid s) \propto \exp(A(s, j)/\tau)$ at $\tau = 1$. The model is then trained to match it, under

$$\mathrm{KL}\big(p^\star \,\|\, \pi_\theta\big) \;+\; \beta\, \mathrm{KL}\big(\pi_\theta \,\|\, \pi_{\text{base}}\big), \qquad \beta = 0.1,$$

where the second term anchors the policy to the untrained model and bounds how far reward pressure can drag it. Only a low-rank adapter is trained — rank 16 on the query and value projections —

so the backbone that writes the text is the same frozen one used everywhere else in this thesis. A polarity canary on a hundred held-out states asserts that the highest-advantage action really does leave the belief closer to the fully-informed one than the lowest-advantage action, before any result is read.

What this amounts to is distillation of a decision rule that the simulator can compute and the language model cannot: the simulator knows what each question would be worth because it can try it; the language model learns to recognise, from the question's text and the learner's history, which questions those tend to be.

What changes. Fine-tuned this way, the same backbone beats a random policy in all six scenarios tested, interval-clear in five of them, recovering between 25% and 74% of the headroom between random and the privileged oracle. Against maximum epistemic spread — the strongest blind heuristic — it separates at interval level in three of six, all of them warm-belief scenarios, which is the first time in this programme that a trained policy cleared that bar against anything other than random. It is a genuinely different policy and not the same one rescored: the trained and untrained models agree on only 2.0% to 5.5% of their choices.

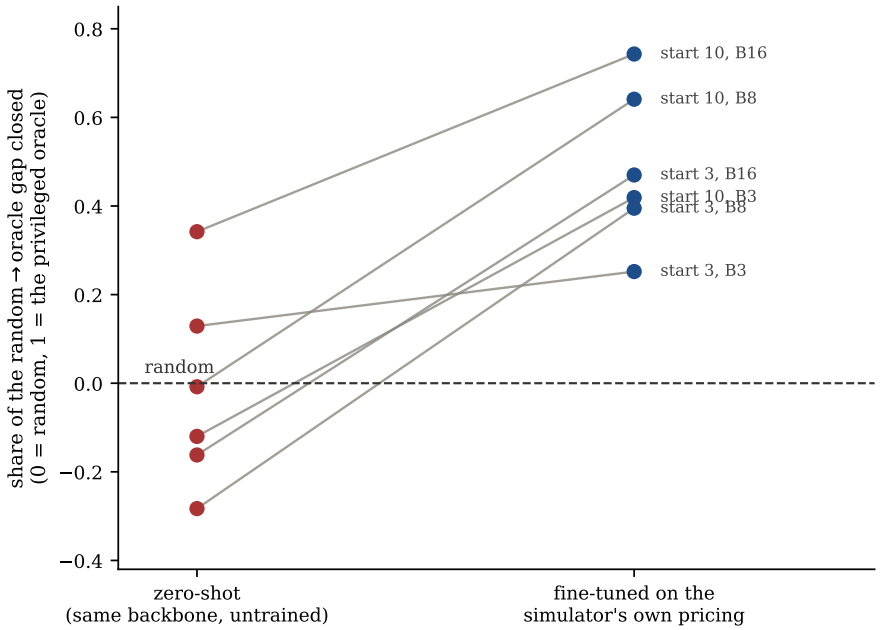


Figure 12: *The same backbone, before and after being fine-tuned on the simulator's own pricing. What to read: the direction and length of each line, one per start-and-budget scenario. The y-axis is the share of the gap between a random policy and the privileged oracle that the policy closes, so 0 is random and 1 is a policy that saw the answers before choosing. The oracle is a bound, not a competitor.*

The claim this licenses is about *selection* and not yet about personalisation, and the distinction is worth keeping. A pre-registered test asked whether the trained model's advantage over its untrained self is larger when it is given a learner's history than when it is not. The answer came back inconclusive — one scenario in the hypothesised direction, one against, four unclear — and the no-history arm is strong in its own right. Part of what the model learned from the simulator is what a good diagnostic question looks like on this corpus in general, which is a real and transferable thing to have learned, and is not the same as learning to match a question to a person.

Pricing a question by its meaning, so that unseen questions can be priced. The selector above scores menu identifiers, so it can only rank questions it has been shown. A companion *utility model* was trained on the same exact-advantage signal but reads a candidate's codec vector instead of its label, which means it can put a price on a question it has never scored. On items it was trained on it beats the greedy value-of-information calculation outright: rank correlation with the true advantage of 0.1923 [0.1632, 0.2231] against EPIG's 0.0690, a realised improvement of +0.0450 [0.0181, 0.0720], and 34.2% of the headroom recovered against EPIG's 21.3%. On items it has never scored the surplus disappears and it *ties*: 21.5% against 22.2%, with both comfortably above a shuffled-identity null. The split is informative rather than disappointing. What transfers is a general sense of what makes a question informative on this corpus; what does not transfer is the item-specific part, which is precisely the part that could only have been memorised.

Writing the question, rather than choosing it. The last experiment in this chapter asks the hardest version: can the simulator teach the model to *write* a question, as text, from the learner's history alone? The signal is the one used throughout this section. For each state a target problem is drawn in proportion to $\exp(A(s, j)/\tau)$ — the same advantage distribution the selector was trained on — and the model is fine-tuned to produce that problem's statement verbatim, given only the history; the menu and the budget are absent from the prompt. Two changes are made to the machinery: the adapter is mixed through a per-learner hypernetwork, so the belief has a channel into what is written, and a contrastive term prefers this learner's own next question over another learner's. Eighteen million low-rank parameters are trained; the backbone is frozen, as everywhere else.

It works on the half of the problem that was expected to be hardest, and Figure 13 measures that against the only fair control: the same weights, on the same states, with the adapter switched off. Untrained, the model writes fluent text that is not really CodeWorkout: only 27.2% of its questions land on the manifold of real problems, a third project nowhere near any real item, and a fifth simply propose something the learner has already done. Fine-tuned on the simulator's signal, 95.6% land on the manifold, the mean distance to the nearest real problem falls by 3.47 units [3.24, 3.71], and the already-seen failure drops to 5.0%. The cost is novelty, and it is paid honestly: verbatim copies of a real problem rise from none to 29.7%, and unique generations fall from 97.5% to 74.1%. A frozen simulator can therefore supply a signal rich enough to teach a language model what a question on this course looks like, and the model pays for that by copying about a third of the time.

What that buys in value is the second panel, and it divides. Judged by the value-of-information scorer, the generated questions improve a great deal: from the 27.5th percentile of their own menu to the 49.3rd, a movement of +0.218 [0.172, 0.263] that is clear of zero. Judged by what the questions are actually worth — the realised diagnostic value of the menu item each generation lands nearest — they reach 0.5079 [0.4739, 0.5428], where 0.50 is what picking uniformly from the menu scores by construction, and the movement of +0.031 over the untrained model contains zero. The model learned to write questions that the pricing instrument likes; it did not demonstrably learn to write questions that pay off. Part of that gap belongs to the instrument rather than to the model: within a single state, the value-of-information score correlates with realised reward at only 0.07 to 0.10, so a model trained to please it has been aimed at a noisy proxy of the thing that matters.

That negative is reported at the scale it was actually run, because the scale is the reason it is an open question rather than a closed one. The registered state draw held 14,370 states, and training it at the registered epoch cap across three seeds was measured, before any run, at about 111 hours and $112 — outside this project's band. A stride-8 subsample was therefore taken, sized

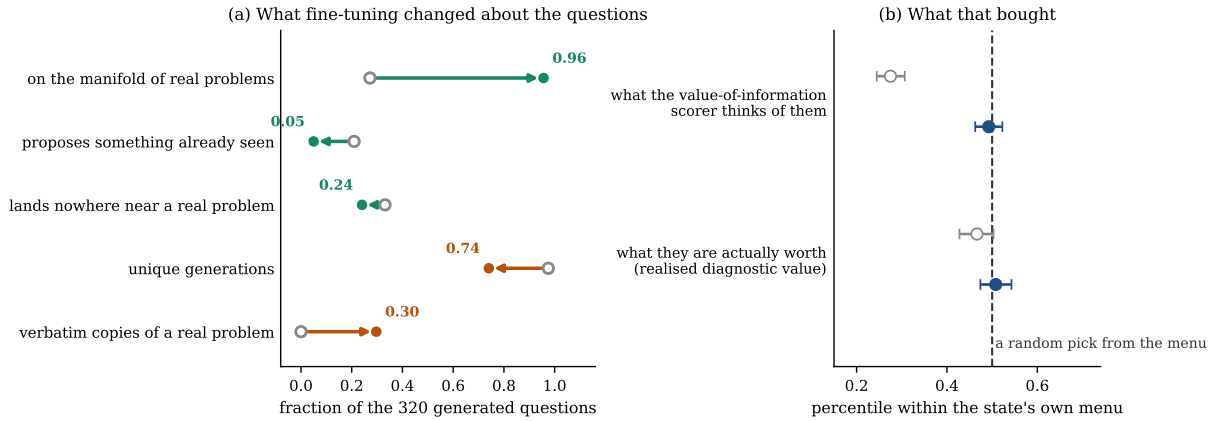


Figure 13: *Teaching the model to write a question, against its own untrained control on the identical harness: 320 states, 110 learners, 1,920 generations. What to read: in panel (a), the direction and length of each arrow, from the untrained model (hollow) to the fine-tuned one (filled); green is the direction that helps and orange is the direction that costs. Panel (b) places the two pointing instruments against the dashed line at 0.50, which is what picking uniformly from the menu scores by construction; bars are 95 per cent intervals.*

from measured throughput and fixed in advance: the model saw **1,400 training rows for a single epoch on a single seed**, with the held-out loss already worse on the second pass. Overfitting after one pass over a tenth of the registered data, with eighteen million trainable parameters, is what that cut predicts. Whether the value barrier is intrinsic to the approach or an artifact of training scale is not answered here, and it is answerable at a known price.

What this tier establishes, then, is the space rather than the result. A learned simulator of a person can supply a generative training signal; the signal moves the model's output, and moves it onto the manifold of real questions. Making those questions informative is a separate obstacle from making them plausible, the two are now measurably separable, and the experiment that would test whether more of the same signal closes the second gap was priced and not run.

8.5 Summary

- **Whether a planner can help is a property of the corpus, and it is measurable in advance.** The person-to-item variance ratio swings fifteen-fold between the two corpora, and every planner result follows that swing.
- **Most of the apparent person signal on mathematics was help-seeking.** Removing give-ups cuts the person's variance share by 62% and the value of three observations by 40%.
- **Policies that read the belief beat asking at random; policies that do not, barely do.** The two belief-reading policies reach 0.924 and 0.944 nats at the headline scenario against random's 1.072, where the two blind heuristics reach 1.028 and 1.031. The trained actor is interval-clear against random in 66 of 128 scenarios and never regresses; which of the two belief-reading policies is better this cohort cannot say.
- **On the corpus where the item owns the variance, the same estimator loses to random**, by 0.0159 nats with the interval clear the wrong way.
- **A policy trained on zero real interactions transfers in the right direction and is not established.** Five of six scenarios favour it at ratios 0.583–0.897; the sixth reverses to

1.130; no scenario clears its bar at this cohort size.

- **A language model cannot do this task cold and can be taught it by the simulator.** From losing to random in 36 of 36 scenarios to beating it in six of six, interval-clear in five, and separating from the strongest blind heuristic in three.
- **A price learned from the simulator transfers to questions it has never scored**, matching the principled calculation there (21.5% of headroom against 22.2%) while beating it on questions it has (34.2% against 21.3%).
- **A simulator can teach a language model to write questions that belong on the course; making them *informative* is a separate and unsolved problem.** Fine-tuning moves generations onto the manifold of real problems, from 27.2% to 95.6%, at a cost of 29.7% verbatim copies. It also moves them from the 27.5th to the 49.3rd percentile of what the value-of-information scorer likes, clear of zero — but their realised diagnostic value reaches only 0.5079 [0.4739, 0.5428], where a random pick scores 0.50 by construction. It was trained on a tenth of the registered draw for one epoch, the subsample forced by cost, so the scale at which that might change is untested rather than ruled out.

This chapter supports the sixth and seventh contributions of Section 1.4. It also completes the argument the thesis set out to make: a belief that can be located can be simulated, and a belief that can be simulated can be acted upon — on the corpus where there is something to act on.

9 Discussion and Conclusion

9.1 What was shown

Section 1.4 made eight claims. They are taken here in order, each against the evidence that supports it and with the scope that evidence licenses.

The framework (C1) states what an ideal mind-reader computes: two spaces, competence as the probability that best-effort thinking reaches a target, a belief that carries a region, information priced by the decision it changes, and discovery and extension held apart. Its claim is assembly rather than invention, and its yield is concrete: the personalisation literature's three carriers become coordinate choices for one operation, and Table 5 states object by object what the built system does not compute.

Meaning can do the work a curriculum did (C2). On FoundationalASSIST the encoder recovers the curriculum's own grouping at 0.819 AUC without ever being shown it and judges which of two problems is more discriminating at 65.4%, where every frontier model tested is below chance; stripping all 462 skill tags from a baseline trained on the same corpus costs it 0.0060 AUC. On CodeWorkout the same representation identifies which of fifty problems a submission solves at 0.995 and cannot say whether two submissions pass the same tests at all, at 0.523 — a failure that put the test-vector head into the architecture.

The simulator locates a learner better than models many times its size (C3, C4). On FoundationalASSIST, 0.786 AUC against 0.512–0.559 for four frontier models reading the same history as text, with attentive knowledge tracing ahead at 0.810; and the exact typed answer reproduced 46.0% of the time against 44.3%. On CodeWorkout, 0.833 AUC on a strictly learner-held-out cohort against a best published 0.816, and 0.561 CodeBLEU on TIKTOC's own rows against 0.554, zero-shot on learners it never trained on. Across all fifty programming problems it reaches 0.526 where KCGen-KT reports 0.580, under a protocol that is not the same measurement.

The gap between the two stages is measurable (C5), and it is the cleanest evidence in the thesis. Supplying the correct outcome class and changing nothing else raises exact reproduction from 46.0% to 57.3%, and from 21.4% to 30.5% on the events where the learner was wrong. The generator was never the bottleneck; locating the mind was.

There is room for reinforcement learning on a simulator of a person (C6). Policies that price a question by what its answer would reveal reach 0.924 and 0.944 nats against a random policy's 1.072, where policies that do not consult the belief reach 1.028 and 1.031; the advantage holds across the grid and never reverses. On the corpus where the item owns the variance, the same estimator loses — which is what tells us the harness is measuring something real.

A simulator can supply a training signal a language model has no other way to get (C7). Given the task cold, a frontier model reads position rather than content and loses to random everywhere; fine-tuned on the exact pricing the simulator computes over real events, it beats random in every scenario and separates from the strongest blind heuristic in three of six. Taught to write questions rather than choose them, it moves its output onto the manifold of real problems, from 27.2% to 95.6%, and does not yet make those questions more informative than a random pick.

The evaluation discipline (C8) changed the conclusions rather than decorating them: without the two courts, most of the apparent person signal on FoundationalASSIST would have been reported as knowledge when it was help-seeking.

9.2 Limitations

These are stated once, at their true strength, and in the order in which they bind.

Traits and moving state are not separately identified. The framework distinguishes stable dispositions from what a learner currently knows; the implementation carries one undivided latent, and any claim about which of the two moved is unavailable.

The endpoint is modelled, not the path. The simulator predicts what a learner would write, not the reasoning that produced it. Chapter 3 distinguished an idea from a thought precisely here, and the thesis measures only the former.

There is no transition law, and therefore no teaching result. The belief moves by evidence and never by an action's consequence. This is a data limitation before it is a design one — no public corpus records who assigned which lesson to whom by a known mechanism — but the consequence is unambiguous: nothing here is evidence that ATOM can teach.

The posterior family was never contested. The belief is a diagonal Gaussian everywhere, defended in Section 5.3 on mechanical grounds, and no heavier-tailed, mixture, full-covariance or flow posterior was implemented. The smoothness the regulariser enforces was also never verified by a direct interpolation probe.

The planner does not separate from the decision-theoretic incumbent. A trained actor matches greedy value of information and does not beat it, in any scenario, on either corpus. With roughly 110 held-out learners the study is underpowered to establish a difference of the size that would be expected.

The identity signal is quantised away where productions are short. On mathematics, personal identity is detectable in the model's probabilities and almost never in its emitted tokens, because a mathematics answer is drawn from a narrow set. The behavioural-twin claim is therefore much stronger on code than on arithmetic.

The model's confidence does not track its errors as well as it should. The coupling between the belief's width and the size of its errors sits at a ratio of 1.08 against a pre-registered bar of 1.2. The

uncertainty channel is the component the planner depends on most, and it is the component this thesis validates least.

The generative results were run at a fraction of their registered scale. The tier that teaches a language model to write questions saw a tenth of its registered state draw for a single epoch on a single seed, the subsample forced by cost and fixed before any result existed. Its negative on question value is therefore a statement about what was run, not about what the method can do.

The oblivious-subject assumption excludes strategic learners. Assumption 3.1 rules out test-wiseness and gaming. Relaxing it makes every answer a strategic report and changes the emission law; the entry point is clear and the work is not done.

Replay pools are confounded by the platforms' own assignment policies. A policy evaluated on logged data may only ask what the platform once offered, which is why the menu's geometry is printed beside every planner number. Only a live deployment removes it.

9.3 Conclusion

When we say a friend knows medicine, we mean something entirely operational: if you asked him about the heart, he could answer. This thesis took that everyday practice literally and built a machine that does the thing the practice stands for.

The machine reads a mind backwards from what it wrote. A frozen encoder turns every problem into a point of meaning rather than an identifier, so nothing in the system indexes an item and no curriculum enters anywhere; a filter walks a learner's path through that space and returns not a guess but a region, a place among five thousand other people together with how sure it is; and a hypernetwork turns that region into an actual model, editing the weights of a frozen language model along axes learned from the whole population, so that the result does not merely know about the learner but for a moment writes as them. The same region is then what the planner reads, because a belief that carries its own width is the one object that can serve both halves of the problem: it can be turned into a person, and it can be asked what it still does not know.

None of the three is finished. The belief does not separate what a person is from what they currently know. The simulation reproduces what a learner writes and not how they arrived at it. The planner asks well where the person owns the variance and no better than a correct one-step calculation. And the language model, taught by the simulator, learned to write questions that belong on the course before it learned to write questions worth asking.

But the parts hold together, and the seam between them is measurable. Supply the right belief and the generator's accuracy on the answers a learner got wrong rises by nine points. That number is the thesis in one line: reading a mind is the hard half, and everything downstream — simulating, asking, eventually teaching — is waiting on it.

Much of what is here can be improved and much of it will be changed. The scale of every generative result was set by a budget rather than by a hypothesis; the posterior is the simplest one that works; the planner's advantage over a principled calculation is not yet established. What the complete picture was always for is something else. It was to show a door: that a machine can hold a model of another mind well enough to run it in imagination, and that imagining a person before acting on them is a usable engineering principle rather than a metaphor — for personalisation in artificial intelligence generally, and, for this author, for education, where being understood before being taught is the whole of what a good teacher does.

A Ablations and Retired Variants

Chapter 5 specifies the configuration that ships and gives the mechanism behind each choice. This appendix is where the evidence and the history live: the alternatives that were built and measured, one line each, so that every choice in that chapter can be read as a decision on evidence rather than on taste. Numbers here are validation numbers under the discipline of Section 4.5; none is a headline claim, and none appears in the main text as one.

A.1 The representation

Codec width. Ranks 45, 64, 128, 256 and 512 were compared. On mathematics, a nonlinear read of 45 dimensions beats a linear read at any width, and predictive quality reverses past about 128 as the training fit keeps rising. On programming, widening from 45 to 256 raises in-pool AUC by +0.0088 and *lowers* unseen-item AUC by −0.0102; under a half-resample refit the in-pool gain erodes to +0.0027. **64 dimensions ship on both corpora** (Section 5.2).

Whitening. Measured as a no-op behind a standardiser on mathematics, identical to four decimal places; left off there, on for programming.

Pooling and layer. Mean-pooling at layer 15 was selected on curriculum recovery, 0.819 against 0.749 for the raw hidden states. A last-token readout is the worst readout on every probe, and diagnosing that was what resolved an earlier null result on code semantics: the information was present and the readout was discarding it.

Two views of the codec. The hypothesis that a raw view carries person-style while a deflated view carries item geometry, so that the model should maintain both, was tested and **refuted**: the two are not separated by their intervals. One view ships.

Item encoder. Linear against a two-layer network, crossed with belief width. The nonlinear read is worth +0.0073 AUC on mathematics and halves calibration error; on programming a leave-items-out cross-validation over seven folds gives it +0.0118 [+0.0052, +0.0192] on unseen items. **Nonlinear ships** (Section 5.2).

The person-style probe. The 0.874 figure in Table 6 was computed with a readout whose off-by-one alignment was corrected afterwards, and the probe was not re-run. It is reported as evidence that style is present, and it is not used to support any ranking.

A coverage test that was not run. The decisive test of the coverage claim of Chapter 6 would plot held-out AUC against a problem's distance, in the space of meanings, to its nearest training item. It was specified and never run, and with the unseen-item comparison no longer reported in Chapter 6, no measurement of coverage stands in the thesis at all.

A.2 The belief

Belief width (mathematics). 32, 64, 256 and 512 hidden units give 0.652, 0.669, 0.678 and 0.680 on the unaided court. The last quadrupling, to 2.15 million parameters, buys +0.0012. **256 ships**; the gap to the published baseline was closed by the item encoder, not by capacity.

Belief dimension (programming). 64 against 32: +0.0060 [−0.0027, +0.0177] on unseen items, straddling zero, and −0.0009 overall. **32 ships** on parsimony.

Filter body. A causal event-transformer over the learner's own events, with attention biased by elapsed time and by semantic distance between items, was built and compared against the recurrent filter under identical heads and loss. It does not win. The informative decomposition is that the *temporal* half of the bias carries about 96% of the body's value and the *semantic* half about 11%, with the learned time weight five times the distance weight — convergent with the published attentive baseline's own design, which has temporal decay and no distance term. **The recurrent filter ships** (Equation (12)).

Width parameterisation. Emitting $\log \sigma$ freely was measured to *widen* the belief as evidence accumulates on programming, which is the wrong direction; the precision-accumulating form was adopted there. On mathematics the free form is retained, and a precision-accumulating variant narrows correctly but loses between-learner identity (Section 5.3, Table 4).

Time and forgetting. A mean-reverting Ornstein–Uhlenbeck forgetting kernel over gaps, with learned per-dimension rates, improved post-gap calibration from 0.0183 to 0.0153 and cost 0.061 of AUC. It was retired on evidence, and the reason is a property of the corpora rather than of the idea: the median gap between consecutive events is 1.4 minutes, and only about 3% of events follow a gap longer than a week. Explicit time and session features were screened separately and are worth -0.0029 nats — real, and too small to ship. **Time enters only through chronological order.**

The posterior family. The belief is a diagonal Gaussian throughout, and this is the ablation's largest gap. Variants were compared *within* the family: the width parameterisation above; a K-sample mixture likelihood in place of a single sample; structured committee sampling with antithetic and sigma-point draws; and a joint-set objective scoring ten to twenty of a learner's later answers under one shared sample. A latent-free read-out of epistemic spread by predictive resampling was run as an exploratory probe on both corpora and moved in the expected direction. **No heavier-tailed, mixture, full-covariance or normalising-flow posterior was implemented at all**, so the case for the Gaussian in Section 5.3 is mechanical and is not supported by a comparison against another family.

Latent smoothness. No interpolation probe between two learners' beliefs was run, so the smoothness the regulariser is designed to enforce is not verified directly. The evidence is indirect: predictions respond continuously as the belief is widened along its own axes, and the effect of perturbing one early event decays smoothly with history.

Structural variants specified and not built. A belief carried as several attentional slots rather than one vector; a particle or sequential-Monte-Carlo belief; a learner graph supplying a cold-start prior and peer context. The prior remains $\mathcal{N}(0, I)$.

A.3 The heads and the binding

Where estimation lives. An earlier design read outcome predictions off the backbone's hidden state at the prompt boundary. It was built and measured: success prediction scored 0.5833 cross-entropy against a 0.4802 constant floor — worse than predicting the base rate — and support prediction lost to a per-learner frequency table. **Estimation moved onto the belief and the language model was kept for text alone**, which is the configuration of Section 5.5.

Factored outcome head. Predicting help-seeking, and then correctness given it, rather than one five-way distribution, was trained head-only on the shipped filter. It *wins* on detecting unaided

wrong answers, by 0.036 in the bottom ability tercile, and fails the uncertainty-coupling gate decisively — its confidence stops tracking its error. **The joint head ships**, and the trade is disclosed in Section 7.2 rather than left here.

Token-loss weight. The weight α on the production loss was swept over $\{0.3, 0.75, 1.0, 1.5, 2.0\}$ at pilot scale under a pre-registered rule: take the largest value that neither loses the personalisation gap nor is decisively worse at detecting wrong answers. Values 1.5 and 2.0 fail; 0.75 maximises the personalisation gap; the rule selects 1.0. A full retrain at $\alpha = 1.0$ exists and the thesis nevertheless reports FA-SIM3 at $\alpha = 0.3$, on the author's instruction. The two are statistically indistinguishable on identity gap, within-item AUC, outcome log-loss, wrong-answer detection and free generation; the later run is +0.009 nats on likelihood and +1.5 points on the easy exact-match cell, and FA-SIM3 is +0.1 points on the hard cell.

Hypernetwork complexity. Identity, linear and small nonlinear maps from belief to mixing weights were specified as an ablation; only the nonlinear map was run.

Global adapter control. A second basis driven by a learner-independent mixing vector, isolating what personalisation adds over merely having an adapter, was constructed but not trained on the shipped runs. Its role is filled by the adapter-off arm of Table 12.

Test-vector head refresh (programming). The per-test-case head trained inside the joint objective was undertrained on first attempts. Retraining it alone on the frozen belief, with first attempts oversampled, raises all-attempt AUC from 0.744 to 0.802 and first-attempt AUC from 0.584 to 0.677. **The refreshed head is what ships** (Section 5.6). A base-rate blend was also fitted and is worse; it is reported and not shipped.

Null-prefix convention. Replacing the fixed zero vector standing for "no previous attempt" with a learned embedding changes nothing, 0.677 against 0.677. The convention is kept.

A.4 The planner

The value estimator. The original acquisition score valued a question by the total entropy it would remove, which mixes irreducible randomness into an epistemic quantity and therefore rewards questions that are merely unpredictable. It was replaced by the Jensen-gap form of Equation (20). The old form is retained as a configuration flag, so that the earlier failure remains reproducible, and both are run side by side under common random numbers wherever it matters.

Replay construction. Three defects in earlier harnesses were found and repaired, each having produced a number. A menu drawn earliest-first placed the askable questions in the first 13% of the learner's history and roughly 287 events before the targets, which asks a policy to choose first-week questions that predict problems a term later. A candidate pool was filtered on outcomes that are unobservable at the moment of selection. And an acquired observation revealed help-seeking channels along with correctness, so the policy was paid for information it could not have had. **Results computed under the unrepaired harness are precursors and are never quoted as floors anywhere in this thesis**; the rules that prevent all three are stated as rules in Section 5.7.

Optimiser's curse. Choosing uniformly among the top five candidates rather than taking the maximum reproduces the greedy policy's losses, which argues against the explanation that the failure on mathematics is an argmax over noisy estimates.

The earlier imagination attempt. A first simulator-trained policy was trained from a permanently empty belief, with the imagined questions left inside the candidate pool. It was worse than random in 31 of 32 cells. Repairing the belief's starting point and the candidate pool — not the training method — produced the result of Section 8.3. The failure was environment construction, and the rule it produced is the one stated in Section 5.7.2: imagination never cold-starts.

Search and imagination as specified. Short-horizon rollout search in latent space, a critic trained inside imagination, and distillation of successful probes back into a proposal head were specified as an escalation ladder and **not built**. What exists is a critic trained on replay, imagination used as augmentation, and a policy trained purely inside the simulator.

Stopping. A learned stop action and a cost-conditioned actor were registered and not run; every actor asks exactly its budget, and the cost is swept afterwards.

Comparators not run. Fisher-information item selection — the classical adaptive-testing policy — is the one obvious comparator that was specified and never implemented. Batch selection, choosing all B questions at once rather than sequentially, was likewise not run.

Belief-conditioned question generation, the discarded first design. Before the run reported in Section 8.4, the same advantage-weighted generative fine-tune was attempted with a *single global* adapter rather than a per-learner one and without the contrastive term. It collapsed onto rote memorisation of the training pool — 96% verbatim copies, 13% unique generations — and its history-dependence fell relative to the untrained model, interval-clear in the wrong direction. It is recorded here as the design that motivated the two changes, and no number from it is quoted in the main text.

A.5 Data defects found and closed

Two leaks were found during the construction of the manifests of Section 4.5, and both are recorded here because each had produced numbers before it was caught. The first was exact-duplicate rows in the mathematics release — 31,441 of them, in 15,677 groups — which allowed an identical event to appear on both sides of a learner split. The second was a normalisation computed over a whole episode and then attached to each of its steps, which made the episode's future recoverable from a step-wise causal prefix; this is the failure the leakage rule of Section 4.5 is written to forbid, and it is why that rule is absolute rather than a matter of judgement. Split canaries — a planted duplicate and a deliberately leaked row — were added afterwards and are asserted caught on every rebuild of the splits.

B Notation, for reference

The table of Section 1.5 is repeated here so that it can be consulted while reading the results chapters without turning back to the introduction.

OBJECT	SYMBOL	IN PLAIN WORDS	INTRODUCED
Meanings			
Space of meanings	$\mathcal{S}$	where the meaning of any piece of text lives: a problem, an answer, a program	§3.2
Meaning of a text	$s(x) \in \mathcal{S}$	what this text is about; in the implementation, 64 numbers	§3.2, built §5.2
Raw text	$x \in \mathcal{X}$	the characters themselves, before encoding	§3.2
Idea, thought, target	z, τ, g	a point of $\mathcal{S}$; a path through $\mathcal{S}$; where a path is meant to end	§3.2
Minds			
Person latent	$u \in \mathcal{U}$	the coordinates of one mind: what it knows and how it tends to work	§3.2
Space of minds	$\mathcal{M}$	every configuration the personalised model could take; a person is a point $W(u)$ in it	§3.3, built §5.4
Shared laws	$\Lambda = (\omega, \phi, \psi)$	what all minds have in common: the prior over persons, how a person moves, how a person shows in behaviour	§3.3
Engagement	η_t	how hard the person is actually trying; separates *cannot* from *does not*	§3.2
Competence	$K_g(u)$	the probability that best-effort thinking reaches target g	Def. 3.3
Evidence and belief			
History	h_t	everything observed about this person up to round t	§3.1
Belief	$b_t = p(u \mid h_t)$	where the mind probably is, and how sure we are; realised as $\mathcal{N}(\mu_t, \mathrm{diag}(\sigma_t^2))$	Def. 3.4, built §5.3
Belief samples	$u^{(1)}, \ldots, u^{(K)}$	K plausible versions of the person, drawn from b_t	§5.3
Questions, answers, moves			
Question	q	what the person is asked; its meaning is $s(q)$	§3.2
Outcome class	c	which of the corpus's outcome categories occurred (5 on mathematics, 4 on programming)	§4.2
Production	w	what the person actually wrote: the typed answer, or the program	§5.5
Probe	$e \in \mathcal{E}$, cost $c(e)$	a move that only informs the modelling agent — the only kind measured here	§3.1
Control action	$a \in \mathcal{A}$	a move that changes the person: teaching. Specified, never measured	§3.1
Deciding			
Value of information	EVOI$(\cdot)$	how much better a decision gets because of what a move reveals	Def. 3.6
The estimator used	EPIG(e)	the special case actually computed: how much a question's answer would sharpen the current prediction	Eq. (10), built Eq. (20)
Weight edit	$\Delta W(u)$	the per-person change to the frozen model's weights	Eq. (15)
Mixing weights	$c(u) \in \mathbb{R}^{32}$	how far to press each of the 32 shared adapter axes for this person	§5.4

References

Alain, G. and Bengio, Y. (2017). Understanding intermediate layers using linear classifier probes. In *ICLR Workshop*.

Bai, Y. et al. (2024). A survey on explainable knowledge tracing. *Artificial Intelligence Review*.

Baker, C. L., Jara-Ettinger, J., Saxe, R., and Tenenbaum, J. B. (2017). Rational quantitative attribution of beliefs, desires and percepts in human mentalizing. *Nature Human Behaviour*, 1:0064.

Baker, C. L., Saxe, R., and Tenenbaum, J. B. (2009). Action understanding as inverse planning. *Cognition*, 113(3):329–349.

Belkin, M. and Niyogi, P. (2003). Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396.

Burns, C., Ye, H., Klein, D., and Steinhardt, J. (2023). Discovering latent knowledge in language models without supervision. In *International Conference on Learning Representations (ICLR)*.

Çelikok, M. M., Peltola, T., Daee, P., and Kaski, S. (2019). Interactive AI with a theory of mind. *arXiv preprint arXiv:1912.05284*.

Chaloner, K. and Verdinelli, I. (1995). Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.

Chen, Z. et al. (2024). ToMBench: Benchmarking theory of mind in large language models. *arXiv preprint arXiv:2402.15052*.

Christiano, P., Cotra, A., and Xu, M. (2021). Eliciting latent knowledge: How to tell if your eyes deceive you. Technical report, Alignment Research Center.

Corbett, A. T. and Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278.

Da Costa, L., Parr, T., Sajid, N., Veselic, S., Neacsu, V., and Friston, K. (2020). Active inference on discrete state-spaces: A synthesis. *Journal of Mathematical Psychology*, 99:102447.

Dai, D., Sun, Y., Dong, L., Hao, Y., Ma, S., Sui, Z., and Wei, F. (2023). Why can GPT learn in-context? language models implicitly perform gradient descent as meta-optimizers. *Findings of ACL*.

de Finetti, B. (1931). Funzione caratteristica di un fenomeno aleatorio. *Atti della R. Accademia Nazionale dei Lincei*, 4:251–299.

Diaconis, P. and Freedman, D. (1980). De Finetti's theorem for Markov chains. *Annals of Probability*, 8(1):115–130.

Ding, X. and Larson, E. C. (2019). Why deep knowledge tracing has less interpretability than bayesian knowledge tracing. *Knowledge-Based Systems*.

Duan, Y., Schulman, J., Chen, X., Bartlett, P. L., Sutskever, I., and Abbeel, P. (2016). RL^2: Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*.

Duan, Z., Fernandez, N., Hicks, A., and Lan, A. (2025). Test case-informed knowledge tracing for open-ended coding tasks. In *Learning Analytics and Knowledge Conference (LAK)*. arXiv:2410.10829.

Duan, Z., Scarlatos, A., Malik, R., and Lan, A. (2026). Automated knowledge component generation and knowledge tracing for coding problems. In *Findings of the Association for Computational Linguistics*. arXiv:2502.18632.

Finn, C., Abbeel, P., and Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning (ICML)*.

Foster, A., Ivanova, D. R., Malik, I., and Rainforth, T. (2021). Deep adaptive design: Amortizing sequential bayesian experimental design. In *International Conference on Machine Learning (ICML)*.

Friston, K. (2010). The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138.

Gandhi, K., Fränken, J.-P., Gerstenberg, T., and Goodman, N. D. (2023). Understanding social reasoning in language models with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Gao, W. et al. (2026). Contextual bandits with LLM-generated messaging for behaviour change. *arXiv preprint arXiv:2506.07275*.

Gen-SSD (2026). Generative student-selective distillation. *arXiv preprint arXiv:2604.02819*.

Ghosh, A. et al. (2024). AutoIRT/BanditCAT: Bandit-based computerized adaptive testing with calibrated item parameters. *arXiv preprint arXiv:2410.21033*.

Ghosh, A., Heffernan, N., and Lan, A. S. (2020). Context-aware attentive knowledge tracing. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*.

Godoy Ortiz, A. D. (2026). Meaning contracts: Local semantic geometry in large language models. arXiv preprint arXiv:2610.30715, https://arxiv.org/abs/2610.30715. Results cited here are established in this companion paper, not in the present thesis.

Grant, E., Finn, C., Levine, S., Darrell, T., and Griffiths, T. (2018). Recasting gradient-based meta-learning as hierarchical Bayes. In *International Conference on Learning Representations (ICLR)*.

Greenblatt, R., Roger, F., et al. (2024). Stress-testing capability elicitation with password-locked models. *arXiv preprint arXiv:2405.19550*.

Grimm, C., Barreto, A., Singh, S., and Silver, D. (2020). The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Ha, D., Dai, A., and Le, Q. V. (2017). Hypernetworks. In *International Conference on Learning Representations (ICLR)*.

Ha, D. and Schmidhuber, J. (2018). World models. *arXiv preprint arXiv:1803.10122*.

Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. (2020). Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*.

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. (2019). Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*.

Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2023). Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*.

Hofstätter, F. et al. (2025). The elicitation game: Stress-testing capability elicitation techniques. *arXiv preprint arXiv:2502.02180*.

Howard, R. A. (1966). Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Janner, M., Fu, J., Zhang, M., and Levine, S. (2019). When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Jara-Ettinger, J. (2019). Theory of mind as inverse reinforcement learning. *Current Opinion in Behavioral Sciences*, 29:105–110.

Jha, R. et al. (2025). Towards machine theory of mind with large language model-augmented inverse planning. *arXiv preprint arXiv:2507.03682*.

Jiang, S., Malkomes, G., Abbott, M., Moseley, B., and Garnett, R. (2020). Efficient nonmyopic bayesian optimization via one-shot multi-step trees. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Kaelbling, L. P., Littman, M. L., and Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134.

Karine, K. and Marlin, B. M. (2025). LLM-augmented reinforcement learning for personalised health interventions. *arXiv preprint arXiv:2501.06980*.

Kosinski, M. (2024). Evaluating large language models in theory of mind tasks. *Proceedings of the National Academy of Sciences*, 121(45):e2405460121.

Krishnan, R. G., Shalit, U., and Sontag, D. (2015). Deep kalman filters. *arXiv preprint arXiv:1511.05121*.

LeCun, Y. (2022). A path towards autonomous machine intelligence (v0.9.2). *Position paper, OpenReview*.

Lee, U. et al. (2025). Next-token knowledge tracing. *arXiv preprint arXiv:2511.02599*.

Li, P. et al. (2026). On the consistency and no-regret guarantees of active inference. *arXiv preprint arXiv:2602.06029*.

Liao, P., Greenewald, K., Klasnja, P., and Murphy, S. (2020). Personalized HeartSteps: A reinforcement learning algorithm for optimizing physical activity. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 4(1).

Lindley, D. V. (1956). On a measure of the information provided by an experiment. *Annals of Mathematical Statistics*, 27(4):986–1005.

Liu, Y. et al. (2026). Deep reinforcement learning for non-myopic item selection in computerized adaptive testing. *arXiv preprint arXiv:2502.19275*.

Liu, Z., Liu, Q., Chen, J., Huang, S., Tang, J., and Luo, W. (2022). pyKT: A python library to benchmark deep learning based knowledge tracing models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*.

Liu, Z., Liu, Q., Guo, T., Chen, J., Huang, S., Zhao, X., Tang, J., Luo, W., and Weng, J. (2023). XES3G5M: A knowledge tracing benchmark dataset with auxiliary information. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*.

Lord, F. M. (1980). *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum.

Mallen, A., Brumley, M., Kharchenko, J., and Belrose, N. (2024). Eliciting latent knowledge from quirky language models. *arXiv preprint arXiv:2312.01037*.

Marks, S. and Tegmark, M. (2023). The geometry of truth: Emergent linear structure in LLM representations of true/false datasets. *arXiv preprint arXiv:2310.06824*.

Moreno, P., Hughes, E., McKee, K. R., Pires, B. A., and Weber, T. (2021). Neural recursive belief states in multi-agent reinforcement learning. *arXiv preprint arXiv:2102.02274*.

Mulder, J. and van der Linden, W. J. (2009). Multidimensional adaptive testing with optimal design criteria for item selection. *Psychometrika*, 74(2):273–296.

Müller, S., Hollmann, N., Arango, S. P., Grabocka, J., and Hutter, F. (2022). Transformers can do bayesian inference. In *International Conference on Learning Representations (ICLR)*.

Owen, R. J. (1975). A bayesian sequential procedure for quantal response in the context of adaptive mental testing. *Journal of the American Statistical Association*, 70(350):351–356.

Parr, T., Pezzulo, G., and Friston, K. J. (2022). *Active Inference: The Free Energy Principle in Mind, Brain, and Behavior*. MIT Press.

Phang, J., Mao, Y., He, P., and Chen, W. (2023). Hypertuning: Toward adapting large language models without back-propagation. *International Conference on Machine Learning (ICML)*.

Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., and Sohl-Dickstein, J. (2015). Deep knowledge tracing. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Rabinowitz, N. C., Perbet, F., Song, H. F., Zhang, C., Eslami, S. M. A., and Botvinick, M. (2018). Machine theory of mind. In *International Conference on Machine Learning (ICML)*.

Riemer, M. et al. (2025). Position: Theory of mind benchmarks are broken for large language models. In *International Conference on Machine Learning (ICML)*.

Russo, D. and Van Roy, B. (2018). Learning to optimize via information-directed sampling. *Operations Research*, 66(1):230–252.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T., and Silver, D. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609.

Strachan, J. W. A. et al. (2024). Testing theory of mind in large language models and humans. *Nature Human Behaviour*, 8:1285–1295.

van der Linden, W. J. (1998). Bayesian item selection criteria for adaptive testing. *Psychometrika*, 63(2):201–216.

van der Weij, T., Hofstätter, F., Jaffe, O., Brown, S. F., and Ward, F. R. (2024). AI sandbagging: Language models can strategically underperform on evaluations. *arXiv preprint arXiv:2406.07358*.

Virginia Tech CodeWorkout (2019). CodeWorkout data, Spring 2019. PSLC DataShop, dataset 3458, https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=3458. Terms of use: https://pslcdatashop.web.cmu.edu/ProjectTerms?id=585.

von Oswald, J., Niklasson, E., Randazzo, E., Sacramento, J., Mordvintsev, A., Zhmoginov, A., and Vladymyrov, M. (2023). Transformers learn in-context by gradient descent. In *International Conference on Machine Learning (ICML)*.

Wang, F., Liu, Q., Chen, E., Huang, Z., Chen, Y., Yin, Y., Huang, Z., and Wang, S. (2020). Neural cognitive diagnosis for intelligent education systems. In *AAAI Conference on Artificial Intelligence*.

Wen, Y., Yang, Y., Luo, R., Wang, J., and Pan, W. (2019). Probabilistic recursive reasoning for multi-agent reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

Worden, E., Heffernan, C., Heffernan, N., and Sonkar, S. (2026). FoundationalASSIST: An educational dataset for foundational knowledge tracing and pedagogical grounding of LLMs. *arXiv preprint arXiv:2602.00070*.

Xie, S. M., Raghunathan, A., Liang, P., and Ma, T. (2022). An explanation of in-context learning as implicit bayesian inference. In *International Conference on Learning Representations (ICLR)*.

Xu, H. et al. (2024). OpenToM: A comprehensive benchmark for evaluating theory-of-mind reasoning of LLMs. In *Association for Computational Linguistics (ACL)*.

Xu, S. et al. (2025). Classroom simulacra: Building generative student agents. *arXiv preprint arXiv:2502.02780*.

Yeung, C. H. et al. (2025). Language bottleneck models for knowledge tracing. *arXiv preprint arXiv:2506.16982*.

Yu, T., Thomas, G., Yu, L., Ermon, S., Zou, J., Levine, S., Finn, C., and Ma, T. (2020). MOPO: Model-based offline policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Zhang, Z., Jin, C., Jia, M. Y., and Shu, T. (2025). AutoToM: Scaling model-based mental inference via automated agent modeling. *arXiv preprint arXiv:2502.15676*.

Zhao, X. et al. (2025). Adaptive chain-of-thought distillation. *arXiv preprint*.

Zintgraf, L., Shiarlis, K., Igl, M., Schulze, S., Gal, Y., Hofmann, K., and Whiteson, S. (2020). VariBAD: A very good method for Bayes-adaptive deep RL via meta-learning. In *International Conference on Learning Representations (ICLR)*.

Use of generative AI. I acknowledge the use of Claude (Anthropic, https://www.claude.ai) as a coding assistant during the implementation of this project, including its training and evaluation scripts, and fluency of the written text. The conceptual framework, architecture, experiments, results and analysis in this thesis are my own, and all AI-assisted text was reviewed and revised by me.